SJVIK Labs

Proxmox Homelab: Zero to Production

Build a production-grade homelab that runs like real infrastructure

Steven J. Vik

2026 Edition

© 2026 Steven J. Vik / SJVIK Labs. All rights reserved.

This guide is licensed for personal use only. Do not redistribute.

Legal Notice

© 2026 Steven J. Vik / SJVIK Labs. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except for brief quotations in critical reviews and certain other noncommercial uses permitted by copyright law.

This guide is licensed for **personal use only** by the original purchaser. You may not share, resell, or redistribute this file. Unauthorized distribution is a violation of copyright law and just kind of a crappy thing to do.

Disclaimer: The information in this guide is provided "as is" without warranty of any kind. The author makes no guarantees regarding the accuracy, completeness, or suitability of the content for any particular purpose. Use the commands, configurations, and advice in this guide at your own risk. The author is not responsible for any damage to hardware, data loss, security breaches, or other consequences resulting from following this guide.

Product names, logos, and trademarks referenced in this guide are the property of their respective owners and are used for identification purposes only. Use of these names does not imply endorsement.

2026 Edition — Version 1.0

Contents

Introduction **5**
What This Guide Is (And What It Isn't) . . . 5
Who This Guide Is For . . . 6
Software Versions . . . 6
What You'll Build . . . 6
Prerequisites . . . 7
The Production Mindset . . . 9
How to Use This Guide . . . 10

Chapter 1 — Hardware and Planning **11**
Hardware Selection . . . 11
Network Planning . . . 13
Storage Planning . . . 15
Power and UPS Considerations . . . 17
Planning Artifacts . . . 18
Pre-Installation Checklist . . . 19

Chapter 2 — Proxmox Installation **20**
Creating the Installer . . . 20
Installing Proxmox VE . . . 21
Initial Configuration . . . 22
Creating a Cluster . . . 24
Storage Configuration . . . 25
Post-Install Hardening . . . 26
Verify Your Installation . . . 28
Troubleshooting . . . 29

Chapter 3 — LXC Containers **31**
Why LXC Over VMs . . . 31
Downloading Templates . . . 32
Creating Containers . . . 33
Resource Allocation Best Practices . . . 34
Networking Containers . . . 36
Container Lifecycle Management . . . 37
Tagging and Organization . . . 38
Privileged vs Unprivileged Containers . . . 39

Container Templates: Building Your Own . . . 40
Batch Operations . . . 41
Troubleshooting . . . 42

Chapter 4 — Core Services **45**
DNS: AdGuard Home . . . 45
Reverse Proxy: Traefik . . . 48
Alternative: Nginx Proxy Manager . . . 54
File Synchronization: Syncthing . . . 54
Version Control: Nightly Git Push . . . 57
Service Dependency Map . . . 59

Chapter 5 — Monitoring and Alerting **61**
Architecture Overview . . . 61
Prometheus . . . 62
node_exporter: Metrics on Every Host . . . 65
PVE Exporter: Proxmox-Specific Metrics . . . 66
Grafana . . . 67
Alert Rules . . . 70
Uptime Kuma . . . 75
Log Aggregation with Loki . . . 77
Putting It All Together . . . 80

Chapter 6 — Backups and Recovery **82**
Backup Strategy Overview . . . 82
Proxmox Backup Server (PBS) . . . 83
Restic: Application-Level Backups . . . 85
Backup Verification . . . 89
Email Alerting with msmtp . . . 90
Disaster Recovery . . . 91
Backup Schedule Summary . . . 93
Backup Verification Checklist . . . 93

Chapter 7 — Security Hardening **94**
Security Principles . . . 94
SSH Hardening . . . 94
Firewall Configuration . . . 96
Fail2ban . . . 98
Unattended Security Upgrades . . . 99
Remote Access: Tailscale . . . 101
Certificate Management . . . 103
Security Audit Checklist . . . 104
What We Did Not Cover (and Why) . . . 104

Chapter 8 — Automation and Maintenance **106**
The Cron Schedule . . . 106
Scripting Patterns . . . 107
Lab Health Report . . . 109

Container Update Strategy . . . 112
Infrastructure as Documentation . . . 113
Day 2 Operations . . . 115
Automation Maturity Model . . . 116
Configuration Management with Ansible . . . 116

Chapter 9 — Advanced Topics **119**
AI Inference Server with Ollama . . . 119
Running Web Applications . . . 122
VPN Options Comparison . . . 125
Scaling Your Cluster . . . 127
Lab as a Resume Builder . . . 130
Troubleshooting . . . 132
What to Build Next . . . 133

Chapter 10 — Appendix **134**
A. IP Planning Worksheet . . . 134
B. Container Inventory Template . . . 136
C. Backup Verification Checklist . . . 136
D. Monitoring Dashboard Export Template . . . 138
E. Recommended Hardware Links and Budget Builds . . . 138
F. Quick Reference: Common Commands . . . 140
G. Glossary . . . 141
H. Troubleshooting Decision Tree . . . 143
I. Next Steps After This Guide . . . 144
Closing Thoughts . . . 145
Interview Readiness Rubric . . . 145

About the Author **147**

Introduction

What This Guide Is (And What It Isn't)

Picture this: you're in a DevOps interview and they ask, "Tell me about your infrastructure experience." You could talk about a tutorial you followed once. Or you could describe the 3-node Proxmox cluster you built from scratch — the one with automated backups, Grafana alerting, firewall rules you wrote yourself, and a disaster recovery plan you've actually tested. This guide gives you that answer.

This is a practical, opinionated guide to building a production-grade homelab on Proxmox VE. Not a toy. Not a weekend project you'll abandon. A real infrastructure platform that runs 24/7, backs itself up, monitors its own health, and teaches you skills that translate directly to professional DevOps and systems administration work.

By the end of this guide, you will have:

- A **3-node Proxmox cluster** with high availability and live migration
- **LXC containers** running DNS, reverse proxy, monitoring, file sync, and whatever else you need
- **Grafana + Prometheus monitoring** with alert rules that page you when something breaks
- **Automated backups** with Proxmox Backup Server and Restic, verified weekly
- **Security hardening** including firewalls, fail2ban, SSH lockdown, and encrypted remote access
- **Automation** for maintenance, reporting, and day-2 operations
- **Documentation practices** that keep your future self from cursing your past self

This is not a guide to running Docker Compose on a Raspberry Pi. It is not a guide to setting up Plex. Nothing wrong with those — but they are well-covered territory. This guide assumes you want to go deeper. You want to understand virtualization, networking, storage, monitoring, and security at the level expected of someone who manages infrastructure professionally.

What this guide is not:

- A networking fundamentals course. You should know what a subnet is.
- A Linux tutorial. You should be comfortable at a terminal.
- A Proxmox reference manual. The official docs exist for that. This guide tells you what to do with Proxmox and why.

  Version note: This guide targets **Proxmox VE 8.2+** on Debian 12 (Bookworm). Commands and package names may differ on earlier versions. Where a step is version-sensitive, we call it out.

Who This Guide Is For

You are an IT enthusiast, career changer, student, or junior professional who wants to:

- **Build real infrastructure skills** outside of work or school
- **Prepare for DevOps, SRE, or sysadmin roles** with hands-on experience you can discuss in interviews
- **Run self-hosted services** on something more serious than a single box
- **Understand production patterns** — monitoring, alerting, backups, security — before encountering them in a job

You should already have:

- Basic Linux command-line skills (navigate directories, edit files, install packages)
- A general understanding of networking (IP addresses, DNS, ports, HTTP)
- Willingness to read error messages and troubleshoot
- At least two physical machines (three is ideal; two works for learning)

If you have never touched a terminal before, start with a Linux basics course first. This guide will be here when you get back.

Software Versions

This guide was written and tested against specific software versions. You don't need to pin everything exactly, but if something behaves differently than expected, check here first.

Software	Version	Notes
Proxmox VE	8.2+	Debian 12 Bookworm base
Grafana	11.x	OSS edition
Prometheus	2.x	LTS recommended
Uptime Kuma	1.23.x	Pin to release tag
Traefik	3.x	v3 config syntax used in this guide
AdGuard Home	Latest stable	Auto-updates available
Restic	0.16+	For backup scripts
Proxmox Backup Server	3.x	Matches PVE 8.x
Tailscale	Latest	Auto-updates recommended

These are the versions this guide was tested against. Newer versions should work with minimal changes. Where a step is version-sensitive, we note it.

What You'll Build

Here is the architecture you are building toward:

```
                    ┌──────────────────────────────────┐
                    │            Your Network          │
                    │          192.168.1.0/24          │
                    └──────────────────────────────────┘
                                   │
                                   │
                   ┌───────────────┼───────────────┐
                   │               │               │
                   │               │               │
          ┌────────┴────────┐  ┌────┴────────┐  ┌────┴────────┐
          │  node-01        │  │  node-02    │  │  node-03    │
          │ Proxmox VE      │  │ Proxmox VE  │  │ Proxmox VE  │
          │  .10            │  │  .11        │  │  .12        │
          └────────┬────────┘  └────┬────────┘  └────┬────────┘
                   │               │               │
       ┌───────────┴───┐     ┌─────┴─────┐     ┌─────┴─────┐
       │ LXC 100       │     │ LXC 200   │     │ LXC 300   │
       │ DNS/AdGuard   │     │ Grafana   │     │ PBS       │
       │ .20           │     │ .30       │     │ .40       │
       ├───────────────┤     ├───────────┤     ├───────────┤
       │ LXC 101       │     │ LXC 201   │     │ LXC 301   │
       │ Traefik       │     │ Prometheus│     │ Syncthing │
       │ .21           │     │ .31       │     │ .41       │
       ├───────────────┤     ├───────────┤     ├───────────┤
       │ LXC 102       │     │ LXC 202   │     │ LXC 302   │
       │ App Server    │     │ Uptime    │     │ Git/Scripts│
       │ .22           │     │ Kuma .32  │     │ .42       │
       └───────────────┘     └───────────┘     └───────────┘
```

By the end, every service monitors itself, backs itself up, and alerts you when something goes wrong. That is what "production" means — not perfection, but resilience.

Prerequisites

Hardware: What You Actually Need

The homelab community loves to discuss hardware. Do not let that become a reason to delay starting. You can build a functional cluster with surprisingly modest equipment.

> **Common mistake:** Spending weeks researching the "perfect" hardware before buying anything. The best homelab is the one that exists. Buy two cheap mini PCs today and upgrade later when you hit real limits — not theoretical ones.

Minimum viable lab (2 nodes):

Component	Per Node	Notes
CPU	4+ cores	Intel N100 or equivalent
RAM	16 GB	8 GB is tight; 16 GB is comfortable
Storage	256 GB SSD	NVMe preferred

Component	Per Node	Notes
Network	1 Gbps Ethernet	Onboard is fine

Recommended lab (3 nodes):

Component	Per Node	Notes
CPU	4-8 cores	Intel i5/Ryzen 5 or better
RAM	32 GB	Allows 15-20 containers per node
Storage	512 GB NVMe + optional SATA	NVMe for OS/VMs, SATA for bulk
Network	1 Gbps Ethernet	Dual NIC if budget allows

Budget Tiers

Tier 1 — $200-$300: The Starter

Buy two refurbished mini PCs (Lenovo ThinkCentre Tiny, HP EliteDesk Mini, Dell OptiPlex Micro). These run $80-$150 each on eBay with 16 GB RAM. Add a cheap 8-port gigabit switch ($15-$25). This gets you a 2-node cluster that can run 10-15 containers comfortably.

Best for: Learning Proxmox, understanding clustering, running basic services.

Tier 2 — $500-$800: The Workhorse

Three mini PCs with 32 GB RAM each, or a mix of mini PCs and a used tower for storage duties. Add a managed switch if you want VLANs ($40-$80). Budget $50-$100 for a basic UPS.

Best for: Running a full monitoring stack, multiple applications, and proper backups. This is the sweet spot for learning and is the tier this guide targets.

Tier 3 — $1,000+: The Production Lab

Used enterprise servers (Dell PowerEdge, HP ProLiant), 64+ GB RAM per node, 10 Gbps networking, dedicated NAS or storage node, rack-mounted UPS. At this tier you are building something that could legitimately serve a small business.

Best for: Simulating enterprise environments, running GPU workloads (AI inference, transcoding), hosting services for others.

> **Why this matters:** You do not need Tier 3 hardware to learn Tier 3 skills. Most of what this guide teaches works identically on a $300 setup and a $3,000 setup. Start cheap, upgrade when you hit actual limits — not imagined ones.

Software Prerequisites

All software in this guide is free and open source:

- **Proxmox VE** — Type 1 hypervisor (free, enterprise features included)

- **Proxmox Backup Server** — Backup infrastructure (free)
- **Grafana + Prometheus** — Monitoring and alerting
- **AdGuard Home** — DNS server with ad blocking
- **Traefik** — Reverse proxy with automatic TLS
- **Restic** — Encrypted, deduplicated backups
- **Fail2ban** — Intrusion prevention
- **Tailscale** — Mesh VPN for remote access

Total software cost: $0. This is one of the great advantages of a homelab.

Network Prerequisites

You need:

- A router you control (consumer routers are fine, as long as you can set DHCP reservations)
- A network switch (unmanaged is fine to start; managed if you want VLANs later)
- Ethernet cables (do not run a Proxmox cluster over WiFi)
- A static IP range or the ability to create DHCP reservations on your router

---

The Production Mindset

Here is the single most important idea in this guide, and it has nothing to do with technology:

Treat your homelab like someone is paying you to keep it running.

This does not mean over-engineering everything. It means developing habits that professionals have:

1. **If it is not monitored, it is broken.** You just don't know it yet. Every service gets a health check. Every host gets metrics collection.

2. **If it is not backed up, it does not exist.** Backups must be automated, verified, and stored in more than one place. A backup you have never tested restoring is not a backup — it is a hope.

3. **If it is not documented, you will forget.** Write down your IP scheme. Document your container inventory. Record why you made each decision, not just what you did. Six months from now, you will thank yourself.

4. **Changes are deliberate.** Do not SSH into a box and start changing things without a plan. Know what you are changing, why, and how to roll it back if it goes wrong.

5. **Security is not optional.** Default passwords, open ports, and unpatched systems are not acceptable even in a homelab. Build the habit now so it is automatic when it matters professionally.

None of this is hard. It is just disciplined. And discipline, more than any specific technology, is what separates a homelab that teaches you something from one that just frustrates you.

How to Use This Guide

This guide is designed to be followed in order the first time through. Each chapter builds on the previous one:

Chapter	What You Build	Depends On
01	Hardware and network plan	Nothing
02	Proxmox cluster	Hardware
03	LXC containers	Proxmox cluster
04	Core services (DNS, proxy, sync)	Containers
05	Monitoring and alerting	Core services
06	Backups and recovery	All services running
07	Security hardening	All services running
08	Automation and maintenance	Everything
09	Advanced topics	Everything
10	Appendix / templates	Reference

After your initial build, use individual chapters as reference when you need to add or modify something.

Every command in this guide is designed to be copy-pasted into a terminal. Configuration files are shown in full, not fragments. Where there are choices to make (like which DNS server to use), I give you my recommendation and explain why — then show you how to set up the alternative if you disagree.

Let's build something.

Next: Chapter 1 — Hardware and Planning

Chapter 1 — Hardware and Planning

The most common homelab mistake is buying hardware before having a plan. The second most common mistake is spending three weeks planning and never buying hardware. This chapter helps you do both in the right order and the right amount.

Hardware Selection

Mini PCs: The Sweet Spot

For most homelabs, mini PCs (also called micro PCs, tiny PCs, or SFF — Small Form Factor) offer the best balance of performance, power consumption, noise, and cost.

Recommended models (buy refurbished):

Model	Typical Price	CPU	Max RAM	Notes
Lenovo ThinkCentre M720q/M920q	$80-$150	i5-8500T / i5-9500T	64 GB	Excellent Proxmox support, dual NIC models available
HP EliteDesk 800 G4/G5 Mini	$90-$160	i5-8500T / i5-9500T	64 GB	Reliable, widely available
Dell OptiPlex 7060/7070 Micro	$80-$150	i5-8500T / i5-9500T	64 GB	Good thermals, quiet
Intel NUC 11/12	$150-$300	i5-1135G7 / i5-1240P	64 GB	Compact, decent iGPU for transcoding
Beelink/MinisForum N100	$120-$180	Intel N100	16 GB	Ultra-low power (~6W TDP), great for always-on

Why refurbished enterprise mini PCs?

- Built to run 24/7 (enterprise duty cycle)
- Standardized parts (RAM, SSD, NIC all replaceable)
- Abundant supply keeps prices low
- 8th/9th gen Intel still handles homelab workloads easily
- 10-25W idle power draw (your electricity bill will thank you)

  Common mistake: Buying consumer gaming hardware for a homelab. That RTX-equipped tower pulls 150W at idle and sounds like a jet engine. Unless you specifically need a GPU for AI inference or transcoding, mini PCs are the right answer.

Used Enterprise Servers

If you have space, tolerance for noise, and want to simulate a real data center environment:

Model	Typical Price	CPU	Max RAM	Notes
Dell PowerEdge R720/R730	$100-$300	Dual Xeon E5 v2/v3	768 GB	Loud, power-hungry, but massive capacity
HP ProLiant DL360/DL380 Gen9	$150-$350	Dual Xeon E5 v3/v4	1.5 TB	Same trade-offs as Dell
Dell PowerEdge T620/T630	$150-$400	Dual Xeon E5 v2/v3	768 GB	Tower form factor, slightly quieter

The honest trade-offs:

- 200-400W power draw per server (vs 10-25W for a mini PC)
- Fan noise ranges from "noticeable" to "uninhabitable room"
- IPMI/iDRAC/iLO remote management is genuinely useful for learning
- Way more RAM and CPU than you will use for a homelab
- Great if your lab is in a basement or garage, terrible if it is in your office

My recommendation: Start with mini PCs. Upgrade to enterprise gear only if you hit actual resource limits or specifically want enterprise management experience. Three mini PCs with 32 GB RAM each gives you 96 GB total — enough for 30-40 containers running real workloads.

NAS / Storage Node

If you have data to store (media, backups, archives), consider a dedicated storage node:

Option	Price	Notes
Old mini PC + USB SATA dock	$50-$100	Quick and dirty, works for backups
Refurbished NAS (Synology DS920+)	$200-$400	Purpose-built, but runs its own OS

Option	Price	Notes
Mini PC + SATA expansion	$150-$250	Run Proxmox on it, add a PBS container
Used server with drive bays	$150-$400	Maximum flexibility, maximum noise

For this guide, we will use one of the three Proxmox nodes as the storage/backup node. A dedicated NAS is a nice-to-have, not a requirement.

Network Planning

Plan your network before you install anything. Changing IP addresses and subnets after services are running is painful and error-prone.

Flat Subnet vs VLANs

Flat subnet (recommended for starting):

All devices on one network, e.g., `192.168.1.0/24`. Simple to set up, simple to troubleshoot, works with any router and switch.

VLANs (recommended for advanced setups):

Separate networks for management, services, IoT, guests. Requires a managed switch and a router/firewall that supports VLAN tagging (pfSense, OPNsense, or a VLAN-capable consumer router).

Approach	Pros	Cons
Flat subnet	Simple, works everywhere, no special hardware	Less isolation, all devices can see each other
VLANs	Network segmentation, traffic isolation, professional practice	Requires managed switch, more complex troubleshooting

My recommendation: Start flat. You can add VLANs later without rebuilding. The networking concepts in this guide work the same either way. Do not let VLAN planning become a reason to delay building.

IP Address Scheme

This is your most important planning artifact. Write it down. Keep it updated.

Here is a proven scheme for a 3-node cluster on `192.168.1.0/24`:

Infrastructure (static IPs):

IP	Device	Role
192.168.1.1	Router/Gateway	Default gateway, DHCP server
192.168.1.2	Network switch	Management IP (if managed switch)
192.168.1.10	node-01	Proxmox host
192.168.1.11	node-02	Proxmox host
192.168.1.12	node-03	Proxmox host / storage

Containers on node-01 (.20-.29):

IP	Container	Service
192.168.1.20	ct-dns	AdGuard Home
192.168.1.21	ct-proxy	Traefik reverse proxy
192.168.1.22	ct-app-01	Application server
192.168.1.23	ct-vpn	Tailscale subnet router

Containers on node-02 (.30-.39):

IP	Container	Service
192.168.1.30	ct-grafana	Grafana
192.168.1.31	ct-prometheus	Prometheus
192.168.1.32	ct-uptime	Uptime Kuma
192.168.1.33	ct-app-02	Application server

Containers on node-03 (.40-.49):

IP	Container	Service
192.168.1.40	ct-pbs	Proxmox Backup Server
192.168.1.41	ct-sync	Syncthing
192.168.1.42	ct-git	Git / automation scripts

Client devices (.200-.254):

Range	Use
192.168.1.200-249	Workstations, laptops, phones (DHCP reservations)
192.168.1.250-254	Reserved for temporary/test devices

Why this matters: A consistent IP scheme means you can look at any address and immediately know what it is. `.10-.12` are hosts. `.20-.29` are on node-01. `.30-.39` are on node-02. When something goes wrong at 2 AM, this structure is worth its weight in gold.

> **Common mistake:** Using DHCP for infrastructure and hoping addresses don't change. They will change, at the worst possible time. Use DHCP reservations on your router for everything that matters.

DHCP Reservations

Log into your router and create DHCP reservations for every Proxmox host and every container that runs a service. The process varies by router, but the concept is universal: you bind a MAC address to a fixed IP.

On most consumer routers: 1. Navigate to LAN > DHCP settings 2. Find the "Static IP / DHCP Reservation" section 3. Add each device's MAC address and desired IP

If your router does not support DHCP reservations, you can assign static IPs directly in each Proxmox host and container. DHCP reservations are preferred because they keep IP management centralized.

DNS Planning

You will set up a DNS server in Chapter 4, but plan your naming convention now:

```
node-01.lab          → 192.168.1.10
node-02.lab          → 192.168.1.11
node-03.lab          → 192.168.1.12
dns.lab              → 192.168.1.20
proxy.lab            → 192.168.1.21
grafana.lab          → 192.168.1.30
prometheus.lab       → 192.168.1.31
uptime.lab           → 192.168.1.32
pbs.lab              → 192.168.1.40
sync.lab             → 192.168.1.41
```

Using a `.lab` domain (or `.home`, `.internal`, or `.local`) keeps your internal DNS separate from public DNS. Avoid `.local` if you plan to use mDNS/Avahi, as it conflicts.

> **Common mistake:** Using a real public domain (like `.com` or `.io`) for internal DNS without owning it. This causes split-horizon DNS issues and confusing behavior. Use a non-routable TLD.

Storage Planning

Local Storage (Per Node)

Every Proxmox node needs local storage for the OS and for container root disks. The default Proxmox installation creates:

- `local` — Directory storage for ISOs, templates, backups
- `local-lvm` — LVM thin pool for container and VM disks

For a single NVMe or SSD, this default layout works well. A typical 512 GB drive gives you:

Partition	Size	Purpose
Boot (ESP)	512 MB	EFI boot
Root (ext4/xfs)	~100 GB	Proxmox OS, ISOs, templates
LVM thin pool	~380 GB	Container and VM disks
Swap	8-16 GB	Swap space

ZFS: When and Why

ZFS is a filesystem and volume manager that provides:

- **Data integrity** — Checksums on every block, detects silent corruption
- **Snapshots** — Instant, space-efficient point-in-time copies
- **Compression** — Transparent LZ4 compression (often 1.5-2x space savings)
- **RAID-like features** — Mirror, RAIDZ1, RAIDZ2 without a hardware RAID card

When to use ZFS:

- You have multiple drives in a node and want redundancy
- Data integrity matters to you (it should)
- You want to use ZFS snapshots for backup integration with PBS

When local-lvm is fine:

- Single drive per node (no redundancy possible anyway)
- You are just starting and want simplicity
- Your nodes are mini PCs with one NVMe slot

ZFS memory requirements:

ZFS uses RAM for its ARC (Adaptive Replacement Cache). Rule of thumb: 1 GB base + 1 GB per TB of storage. On a 32 GB system with 1 TB of ZFS storage, ZFS will happily use 2-4 GB of RAM. This is fine — ARC memory is released under pressure.

> **Common mistake:** Configuring ZFS RAIDZ1 with only two drives. RAIDZ1 needs a minimum of three drives to provide any redundancy benefit. For two drives, use a ZFS mirror.

Shared Storage and NFS

For a small cluster, local storage per node is sufficient. If you want to enable live migration of containers between nodes, you need shared storage — storage that all nodes can access simultaneously.

Options:

Method	Complexity	Performance	Notes
NFS share from one node	Low	Moderate	Easy to set up, single point of failure

Method	Complexity	Performance	Notes
Ceph (Proxmox-integrated)	High	High	Distributed storage, needs 3+ nodes with spare drives
iSCSI from NAS	Medium	Good	Requires a NAS appliance

My recommendation for starting: Skip shared storage. Use local storage on each node and do offline migration (stop container, move, start) when needed. Add Ceph or NFS later if live migration becomes important to you.

Power and UPS Considerations

Why UPS Matters

A power outage without a UPS means:

- Proxmox hosts lose power instantly (potential filesystem corruption)
- ZFS is resilient but not immune to power loss during writes
- Running containers get killed without shutdown (data loss risk)
- Your router and switch go down too (cluster loses quorum)

UPS Sizing

Calculate your load:

Device	Typical Wattage
Mini PC (idle)	10-25W
Mini PC (load)	30-65W
Network switch	5-15W
Router	10-20W
Enterprise server (idle)	150-300W

A 3-node mini PC lab with switch and router draws roughly 60-100W at idle. A 600VA/360W UPS ($60-$80) gives you 20-40 minutes of runtime — enough to ride out brief outages or execute a clean shutdown.

Recommended UPS models:

Model	Capacity	Price	Notes
APC BE600M1	600VA/330W	$60-$75	Basic, no network management

Model	Capacity	Price	Notes
CyberPower CP1500AVRLCD	1500VA/900W	$150-$180	AVR, LCD, USB for NUT
APC SMT1500RM2U	1500VA/1000W	$200-$400 (refurb)	Rack-mount, network management card

NUT (Network UPS Tools)

If your UPS has a USB or serial connection, you can monitor it from Proxmox and trigger automatic shutdowns:

```
# Install NUT on one Proxmox host (the one connected to the UPS)
apt install nut

# Other hosts connect as NUT clients to trigger their own shutdown
```

We will not cover full NUT setup in this guide, but be aware it exists. Plugging the UPS USB into one Proxmox host and installing NUT takes about 15 minutes and protects your entire lab from unclean shutdowns.

Planning Artifacts

Before you proceed to installation, you should have these documents ready. Use the templates in the Appendix (Chapter 10) or create your own:

1. IP Address Plan

A table mapping every IP to a device and purpose. You started one above — complete it with your actual addresses.

2. Hardware Inventory

Hostname	Role	CPU	RAM	Storage	IP	MAC	Location
node-01	Proxmox host	i5-8500T	32 GB	512 GB NVMe	192.168.1.10	AA:BB:CC:DD:EE:01	Shelf, rack
node-02	Proxmox host	i5-8500T	32 GB	512 GB NVMe	192.168.1.11	AA:BB:CC:DD:EE:02	Shelf, rack
node-03	Proxmox + storage	i5-9500T	32 GB	512 GB NVMe + 2TB SATA	192.168.1.12	AA:BB:CC:DD:EE:03	Shelf, rack

3. Container Plan

CTID	Name	Node	IP	Service	CPU	RAM	Disk
100	ct-dns	node-01	.20	AdGuard Home	1	512 MB	4 GB
101	ct-proxy	node-01	.21	Traefik	1	512 MB	4 GB
200	ct-grafana	node-02	.30	Grafana	2	1 GB	8 GB
201	ct-prometheus	node-02	.31	Prometheus	2	2 GB	20 GB
300	ct-pbs	node-03	.40	PBS	2	2 GB	500 GB

4. Naming Convention

Decide on a convention and stick with it:

- **Hostnames:** `node-01`, `node-02`, `node-03` (or use a theme — planets, cities, whatever helps you remember)
- **Containers:** `ct-<service>` or `<node>-<service>`
- **DNS:** `<service>.lab`
- **Tags:** Functional tags on every container (e.g., `infra`, `monitoring`, `app`, `backup`)

  Why this matters: Planning feels tedious when you are eager to install things. But 30 minutes of planning now saves hours of renumbering, re-addressing, and debugging later. Every production environment has an IP scheme, a naming convention, and an inventory. Starting with one is a professional habit worth building.

Pre-Installation Checklist

Before you proceed to Chapter 2:

- ☐ Hardware purchased/available (minimum 2 nodes, ideally 3)
- ☐ Network switch connected and powered on
- ☐ IP address scheme documented
- ☐ DHCP reservations created on router (or plan for static IPs)
- ☐ USB drive ready for Proxmox installer (8 GB+)
- ☐ Proxmox VE ISO downloaded from proxmox.com/downloads
- ☐ Monitor and keyboard available for initial install (or IPMI if using enterprise gear)

Next: Chapter 2 — Proxmox Installation

Chapter 2 — Proxmox Installation

This is the foundation. Get this wrong and everything you build on top wobbles — cluster joins fail, containers misbehave, and you spend hours debugging problems that trace back to a hostname typo on day one.

This chapter takes you from bare metal to a working 3-node Proxmox cluster. By the end, you will have a cluster with shared management, proper storage pools, and a hardened SSH configuration. Take your time here. Every minute spent getting the install right saves you ten minutes of troubleshooting later.

Creating the Installer

Download and Verify

Download the Proxmox VE ISO from the official site:

```
https://www.proxmox.com/en/downloads/proxmox-virtual-environment/iso
```

Always verify the checksum. The download page lists SHA256 hashes.

On Linux/macOS:

```
sha256sum proxmox-ve_8.x-x.iso
```

On Windows (PowerShell):

```
Get-FileHash proxmox-ve_8.x-x.iso -Algorithm SHA256
```

Compare the output to the hash on the download page. If they do not match, re-download.

Flash the USB Drive

Using Etcher (cross-platform): 1. Download balenaEtcher 2. Select the Proxmox ISO 3. Select your USB drive 4. Flash

Using dd (Linux/macOS):

```
# Identify your USB drive (BE CAREFUL — wrong device = data loss)
lsblk

# Flash the ISO (replace /dev/sdX with your actual USB device)
```

```
sudo dd if=proxmox-ve_8.x-x.iso of=/dev/sdX bs=4M status=progress
sudo sync
```

Using Rufus (Windows): 1. Download Rufus 2. Select your USB drive 3. Select the Proxmox ISO 4. Use "DD Image" mode (not ISO mode) 5. Start

---

Installing Proxmox VE

Repeat this process on each node. The installation is straightforward — here are the decisions that matter.

Boot from USB

1. Insert the USB drive into the first node
2. Boot and enter BIOS/UEFI (usually F2, F12, Del, or Esc during POST)
3. Set USB as the first boot device, or use the one-time boot menu
4. The Proxmox installer should appear

Installation Wizard

Step 1: Target Disk

Select the disk where Proxmox will be installed. For a single-disk system, this is your only option.

Click "Options" to choose the filesystem:

Filesystem	When to Use
ext4	Single disk, simple, reliable. Default choice.
xfs	Single disk, slightly better for large files.
ZFS (RAID0)	Single disk with ZFS benefits (checksums, compression).
ZFS (Mirror)	Two disks — data integrity + redundancy.
ZFS (RAIDZ1)	Three+ disks — one disk can fail without data loss.

For a single NVMe/SSD, choose **ext4** or **ZFS (RAID0)**. If you have two disks, choose **ZFS Mirror**.

Step 2: Location and Timezone

Select your country and timezone. This matters for log timestamps and cron schedules.

Step 3: Password and Email

Set a strong root password. You will use this for the web UI and SSH. The email is for alert notifications — use a real address.

Step 4: Network Configuration

This is the critical step:

Field	node-01	node-02	node-03
Management Interface	Select your Ethernet NIC	Same	Same
Hostname (FQDN)	node-01.lab	node-02.lab	node-03.lab
IP Address	192.168.1.10/24	192.168.1.11/24	192.168.1.12/24
Gateway	192.168.1.1	192.168.1.1	192.168.1.1
DNS Server	192.168.1.1	192.168.1.1	192.168.1.1

Common mistake: Setting the hostname without a domain. Proxmox requires a fully qualified domain name (FQDN) like `node-01.lab`, not just `node-01`. Clustering will fail without this.

Step 5: Install

Review the summary and click Install. The process takes 3-5 minutes per node.

After installation, remove the USB drive and reboot. You should see the Proxmox console screen showing the web UI URL.

Initial Configuration

Access the Web UI

From your workstation browser, navigate to:

`https://192.168.1.10:8006`

Accept the self-signed certificate warning. Log in as `root` with the password you set during installation.

Remove the Enterprise Repository Nag

Proxmox VE is free, but the default installation includes an enterprise repository that requires a paid subscription. Without a subscription key, `apt update` will show errors, and the web UI will display a subscription nag dialog.

On each node, via SSH or the web UI shell:

```
# Disable the enterprise repository
cat > /etc/apt/sources.list.d/pve-no-subscription.list << 'EOF'
deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription
EOF

# Comment out the enterprise repository
sed -i 's/^deb/# deb/' /etc/apt/sources.list.d/pve-enterprise.list

# Update package lists
apt update && apt full-upgrade -y
```

Remove the subscription nag popup (optional but recommended):

This modifies a JavaScript file in the web UI. It is cosmetic only and needs to be reapplied after Proxmox updates. The exact regex changes between Proxmox versions, so **verify the sed worked** before moving on.

```
# Back up the original file
cp /usr/share/javascript/proxmox-widget-toolkit/proxmoxlib.js \
   /usr/share/javascript/proxmox-widget-toolkit/proxmoxlib.js.bak

# Remove the subscription check (Proxmox VE 8.2+)
sed -Ezi.bak \
  "s/(Ext\.Msg\.show\(\{[^}]*title: gettext\('No valid subscription)/void\(\{ \/\/ \1/g" \
  /usr/share/javascript/proxmox-widget-toolkit/proxmoxlib.js

# Restart the web UI service
systemctl restart pveproxy
```

> **Version warning:** This sed pattern works on PVE 8.1–8.3. If the nag still appears after running it, Proxmox has likely changed the JavaScript. Check the `proxmoxlib.js.bak` diff, or search for community-maintained one-liners for your version. The worst case is a cosmetic popup — nothing breaks if the sed fails silently.
>
> **Why this matters:** The enterprise repo and nag screen are not harmful, but they generate noise — failed apt updates, dialog boxes every login. Cleaning this up immediately means fewer distractions during the rest of your setup.

Set the Correct Timezone

Verify the timezone on each node:

```
timedatectl
```

If incorrect:

```
timedatectl set-timezone America/New_York  # Replace with your timezone
```

Configure NTP

Accurate time is essential for cluster operations, TLS certificates, and log correlation:

```
# Verify chrony is running (Proxmox default)
systemctl status chronyd

# Check synchronization status
chronyc tracking
```

If chrony is not installed:

```
apt install chrony
systemctl enable --now chronyd
```

Creating a Cluster

A Proxmox cluster provides:

- **Unified management** — Manage all nodes from a single web UI
- **Live migration** — Move running VMs/containers between nodes (requires shared storage)
- **High availability** — Automatic restart of VMs/containers on surviving nodes if one fails
- **Replicated configuration** — Corosync distributes cluster config to all nodes

Step 1: Create the Cluster on node-01

SSH into node-01 (or use the web UI shell):

```
pvecm create homelab-cluster
```

Verify:

```
pvecm status
```

You should see a single-node cluster with quorum.

Step 2: Join node-02 to the Cluster

SSH into node-02:

```
pvecm add 192.168.1.10
```

You will be prompted for the root password of node-01. After joining, node-02 appears in the web UI under the cluster tree.

Step 3: Join node-03 to the Cluster

SSH into node-03:

```
pvecm add 192.168.1.10
```

Verify the Cluster

From any node:

```
pvecm status
```

Expected output shows:

```
Cluster information
-------------------
Name:             homelab-cluster
Config Version:   3
Transport:        knet
...

Membership information
----------------------
    Nodeid      Votes Name
         1          1 node-01 (local)
```

```
2          1 node-02
3          1 node-03
```

In the web UI, you should now see all three nodes under "Datacenter" in the left sidebar.

> **Common mistake:** Trying to join a node that already has containers or VMs. Always join nodes to the cluster before creating any workloads. If you already have containers, back them up, destroy them, join the cluster, then restore.

> **Common mistake:** Cluster nodes cannot reach each other. Verify with `ping 192.168.1.11` from node-01 before attempting to join. Clustering uses ports 5405-5412 (Corosync). If you have a firewall in front of the nodes, ensure these ports are open between cluster members.

Storage Configuration

Default Storage Layout

After installation, each node has:

Storage	Type	Content	Location
`local`	Directory	ISOs, templates, backups, snippets	`/var/lib/vz/`
`local-lvm`	LVM-thin	Container/VM root disks	LVM thin pool on install disk

This is sufficient for most setups.

Adding a Second Drive (SATA SSD or HDD)

If node-03 has an additional drive for bulk storage (backups, media):

```
# Identify the drive
lsblk

# Example output:
# sda      8:0    0  1.8T  0 disk   ← This is your additional drive
# nvme0n1  ...                      ← This is your OS drive

# Create a partition
fdisk /dev/sda
# n (new partition) → p (primary) → 1 → Enter → Enter → w (write)

# Format as ext4
mkfs.ext4 /dev/sda1

# Create mount point
```

```
mkdir -p /mnt/storage

# Get the UUID
blkid /dev/sda1
```

Add to `/etc/fstab`:

```
echo "UUID=<your-uuid>  /mnt/storage  ext4  defaults,noatime  0  2" >> /etc/fstab

# Mount it
mount -a

# Verify
df -h /mnt/storage
```

Add as Proxmox storage:

In the web UI: 1. Datacenter > Storage > Add > Directory 2. ID: `storage-sata` 3. Directory: `/mnt/storage` 4. Content: Select what this storage should hold (Disk image, Container, Backup, ISO image, etc.) 5. Nodes: Select which nodes can see this storage (just node-03 for local storage)

ZFS Pool Setup

If you chose ZFS during installation or want to add a ZFS pool to a second drive:

```
# Single disk ZFS pool (no redundancy, but checksums + compression)
zpool create -f -o ashift=12 tank /dev/sda

# Enable compression
zfs set compression=lz4 tank

# Create datasets
zfs create tank/containers
zfs create tank/backups
```

Add ZFS storage to Proxmox:

In the web UI: 1. Datacenter > Storage > Add > ZFS 2. ID: `tank-containers` 3. ZFS Pool: `tank/containers` 4. Content: Container, Disk image 5. Nodes: Appropriate node(s)

Post-Install Hardening

SSH Key Authentication

Password-based SSH is a liability. Set up key-based authentication from your workstation to all nodes.

On your workstation (if you don't have a key pair):

```
ssh-keygen -t ed25519 -C "your-email@example.com"
```

Copy your public key to each node:

```
ssh-copy-id root@192.168.1.10
ssh-copy-id root@192.168.1.11
ssh-copy-id root@192.168.1.12
```

Verify key-based login works:

```
ssh root@192.168.1.10    # Should not prompt for password
```

Disable password authentication for SSH (on each node):

```
# Edit SSH config
sed -i 's/^#PermitRootLogin.*/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
sed -i 's/^PermitRootLogin.*/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config

# Ensure pubkey auth is enabled
sed -i 's/^#PubkeyAuthentication.*/PubkeyAuthentication yes/' /etc/ssh/sshd_config

# Restart SSH
systemctl restart sshd
```

> **Why this matters:** SSH brute-force attacks are constant, even on private networks. `prohibit-password` means only SSH keys can authenticate as root. Password login for non-root users is still allowed (useful for initial setup of non-root accounts).

SSH Config on Your Workstation

Create or update `~/.ssh/config` to make connections easy:

```
Host node-01
    HostName 192.168.1.10
    User root
    IdentityFile ~/.ssh/id_ed25519

Host node-02
    HostName 192.168.1.11
    User root
    IdentityFile ~/.ssh/id_ed25519

Host node-03
    HostName 192.168.1.12
    User root
    IdentityFile ~/.ssh/id_ed25519
```

Now you can connect with just:

```
ssh node-01
```

Basic Firewall

Proxmox includes a built-in firewall. We will configure it properly in Chapter 7, but for now, enable basic protection:

In the web UI:

1. Datacenter > Firewall > Options
2. Set "Firewall" to **Yes**
3. Datacenter > Firewall > Rules > Add:
   - Direction: IN
   - Action: ACCEPT
   - Source: `192.168.1.0/24` (your LAN)
   - Protocol: tcp
   - Destination Port: `8006` (Web UI)
   - Comment: "Allow PVE web UI from LAN"
4. Add another rule:
   - Direction: IN
   - Action: ACCEPT
   - Source: `192.168.1.0/24`
   - Protocol: tcp
   - Destination Port: `22` (SSH)
   - Comment: "Allow SSH from LAN"

The default policy is DROP for inbound traffic, so only explicitly allowed traffic gets through.

> **Common mistake:** Enabling the firewall without adding rules first and locking yourself out of the web UI. Always add your LAN access rules BEFORE enabling the firewall. If you do lock yourself out, connect a monitor and keyboard directly to the node and run `pve-firewall stop` to disable it.

Update All Nodes

After cluster creation, update everything:

```
# Run on each node (or use a loop from your workstation)
apt update && apt full-upgrade -y
```

Or from your workstation, update all nodes at once:

```
for node in node-01 node-02 node-03; do
    echo "=== Updating $node ==="
    ssh $node 'apt update && apt full-upgrade -y'
done
```

Verify Your Installation

Before moving on, verify everything is working:

Cluster Health

```
# On any node
pvecm status            # All 3 nodes listed, quorate
pvecm nodes             # All nodes show as online
```

Storage

```
# On any node
pvesm status           # All storage pools listed and active
```

Web UI

- Navigate to `https://192.168.1.10:8006`
- All three nodes visible in the left sidebar
- No error messages in the cluster summary
- Each node shows correct CPU, RAM, and storage

SSH

```
# From your workstation
ssh node-01 'hostname'
ssh node-02 'hostname'
ssh node-03 'hostname'
```

Time Sync

```
for node in node-01 node-02 node-03; do
    echo "=== $node ==="
    ssh $node 'timedatectl | grep "System clock"'
done
```

All nodes should show "System clock synchronized: yes".

Troubleshooting

"Host key verification failed" when joining cluster

```
# On the node trying to join, clear old host keys
ssh-keygen -R 192.168.1.10
```

Web UI shows "connection error" after firewall enable

Connect directly to the node (monitor + keyboard or IPMI):

```
pve-firewall stop       # Disable firewall immediately
pve-firewall status     # Verify it's stopped
```

Then fix your firewall rules and re-enable.

Cluster join fails with "authentication failure"

- Verify you are typing the root password of the **existing cluster node** (node-01), not the joining node
- Verify node-01 is reachable: `ping 192.168.1.10` from the joining node
- Check that `/etc/hosts` on both nodes resolves their hostname correctly

Corosync errors in cluster status

```
# Check corosync ring status
pvecm status

# If ring shows errors, verify network connectivity between nodes
# Corosync uses UDP ports 5405-5412
```

Chapter Checkpoint

You just stood up a 3-node Proxmox cluster from bare metal. Before you move on, take 60 seconds to verify it actually works.

Before you move on, verify:

- ☐ Proxmox web UI loads at `https://<node-ip>:8006`
- ☐ No-subscription repo is configured and `apt update` completes without errors
- ☐ Timezone is correct (`timedatectl`)
- ☐ SSH works from your workstation to each node

You now have a functioning 3-node Proxmox cluster. The next chapter covers deploying containers — the lightweight workhorses that will run all your services.

Next: Chapter 3 — LXC Containers

Chapter 3 — LXC Containers

LXC containers are the backbone of a Proxmox homelab. They are lightweight, fast, and efficient — consuming a fraction of the resources a full virtual machine would need. This chapter covers everything from creating your first container to managing a fleet of them.

Why LXC Over VMs

This is the single most impactful decision in your homelab architecture, so let's be clear about it.

What LXC Containers Are

LXC (Linux Containers) are OS-level virtualization. They share the host kernel but have their own filesystem, network stack, process space, and user space. Think of them as very isolated chroot environments with their own init system.

What VMs Are

Full virtual machines emulate entire hardware. Each VM runs its own kernel, its own bootloader, and its own complete operating system. They are heavier but offer stronger isolation.

The Comparison

Factor	LXC Container	VM (KVM/QEMU)
Boot time	1-3 seconds	15-60 seconds
RAM overhead	~30-50 MB base	~256-512 MB base
Disk overhead	~200-500 MB	2-8 GB (full OS)
CPU overhead	Near zero	Low (hardware virtualization)
Isolation	Process/namespace isolation	Full hardware isolation
Kernel	Shared with host	Own kernel
GPU passthrough	Limited	Full support
Docker inside	Possible (nested)	Native support
Windows guests	No	Yes

When to Use LXC

Use LXC containers for: - DNS servers (AdGuard, Pi-hole) - Reverse proxies (Traefik, Nginx) - Monitoring (Grafana, Prometheus, Uptime Kuma) - File sync (Syncthing) - Web applications (Node.js, Python, static sites) - Databases (PostgreSQL, MariaDB, SQLite-based apps) - Backup servers (PBS) - VPN endpoints (Tailscale, WireGuard)

This covers 90%+ of homelab workloads.

When to Use VMs

Use VMs for: - Windows-based workloads - Docker/Podman hosts (if you prefer VMs for container isolation) - GPU passthrough (AI inference, transcoding) - Workloads requiring a specific kernel version - Security-critical workloads that need full isolation (e.g., malware analysis)

> **Why this matters:** A mini PC with 32 GB RAM can comfortably run 15-20 LXC containers, but only 4-6 VMs. LXC lets you do more with less hardware, which is the entire point of a homelab. Use VMs only when LXC genuinely cannot do the job.

Downloading Templates

Before creating containers, you need OS templates. Proxmox uses pre-built rootfs images.

Via Web UI

1. Select a node in the sidebar
2. Click the node name > Local storage > CT Templates
3. Click "Templates" button
4. Select a template and click "Download"

Recommended templates:

Template	Use Case	Size
`debian-12-standard`	General purpose, most services	~180 MB
`ubuntu-22.04-standard`	When software requires Ubuntu	~220 MB
`alpine-3.19-default`	Ultra-lightweight, minimal	~4 MB

Via Command Line

```
# Update the template list
pveam update

# List available templates
pveam available --section system

# Download Debian 12
pveam download local debian-12-standard_12.7-1_amd64.tar.zst
```

```
# Download Ubuntu 22.04
pveam download local ubuntu-22.04-standard_22.04-1_amd64.tar.zst
```

My recommendation: Standardize on Debian 12 for everything unless a specific service requires Ubuntu. Fewer distributions means fewer things to remember during updates and troubleshooting.

Creating Containers

Via Web UI

1. Click "Create CT" in the top right
2. Fill in the form:

Field	Value	Notes
Node	Select the target node	Match your IP plan
CT ID	`100`	Match your numbering scheme
Hostname	`ct-dns`	Descriptive, matches your plan
Password	Strong password	Or use SSH key (recommended)
SSH Public Key	Paste your key	Same key as Proxmox hosts
Template	`debian-12-standard`	From local storage
Disk size	`4` GB	Right-size per service
CPU cores	`1`	Can increase later
Memory	`512` MB	Can increase later
Network	Bridge: `vmbr0`, IPv4: `192.168.1.20/24`, GW: `192.168.1.1`	Static IP from your plan
DNS	Domain: `lab`, DNS servers: `192.168.1.1`	Use router DNS initially

3. Review the summary and click "Finish"

Via Command Line

Command-line creation is faster and scriptable:

```
# Create a basic container on node-01
pct create 100 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-dns \
    --cores 1 \
    --memory 512 \
    --swap 256 \
    --rootfs local-lvm:4 \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.20/24,gw=192.168.1.1 \
    --nameserver 192.168.1.1 \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
```

```
    --unprivileged 1 \
    --features nesting=1 \
    --start 0 \
    --onboot 1 \
    --tags infra,dns
```

Important flags explained:

Flag	Purpose
`--unprivileged 1`	Runs container as non-root on the host. More secure. Default for new containers.
`--features nesting=1`	Required if the container will run Docker, LXC, or other nested virtualization.
`--onboot 1`	Container starts automatically when the host boots. Essential for infrastructure services.
`--tags infra,dns`	Organizational tags. Visible in the web UI and useful for filtering.
`--ssh-public-keys`	Copies your SSH public key into the container for key-based access.

Start the Container

```
pct start 100
```

Enter the Container

```
# From the Proxmox host
pct enter 100

# Or via SSH from your workstation
ssh root@192.168.1.20
```

Resource Allocation Best Practices

CPU

Proxmox uses CPU weights and limits, not hard reservations. Assigning 2 cores to a container means it can use up to 2 cores, not that 2 cores are reserved for it.

Service Type	Recommended Cores	Notes
DNS (AdGuard/Pi-hole)	1	Very lightweight
Reverse proxy (Traefik)	1-2	Increases with traffic
Monitoring (Grafana)	2	Dashboard rendering needs CPU
Monitoring (Prometheus)	2-4	Depends on scrape targets
Database	2-4	Depends on workload
Web application	1-2	Depends on complexity

Service Type	Recommended Cores	Notes
Backup (PBS)	2	Compression and verification

Common mistake: Over-allocating CPU cores "just in case." If you give every container 4 cores on a 6-core host, you are not reserving anything — CPU is shared. Start with the minimum and increase when you observe actual CPU contention (check with `pct monitor` or Grafana).

Memory

Unlike CPU, memory allocation is a hard limit. A container that hits its memory limit will trigger the OOM killer and processes will die.

Service Type	Recommended RAM	Notes
DNS	256-512 MB	Pi-hole can run on 256 MB
Reverse proxy	256-512 MB	Traefik is lean
Grafana	512 MB - 1 GB	More dashboards = more RAM
Prometheus	1-4 GB	Depends on metrics volume and retention
Uptime Kuma	256-512 MB	Lightweight
Syncthing	512 MB - 1 GB	Depends on file count
PBS	1-2 GB	Needs RAM for deduplication
Web application	512 MB - 2 GB	Depends on stack (Node.js, Python, etc.)

How to right-size:

1. Start with the minimum recommended above
2. Run the service for a few days
3. Check actual usage:

```
# From the Proxmox host, check container memory usage
pct exec 100 -- free -m

# Or use the Proxmox API
pvesh get /nodes/node-01/lxc/100/status/current
```

4. If usage consistently exceeds 80% of allocation, increase it:

```
# Increase memory (requires container restart if running)
pct set 100 --memory 1024
```

Disk

Container root disks can be resized up but not down. Start conservatively:

Service Type	Recommended Disk	Notes
DNS	2-4 GB	Tiny footprint
Reverse proxy	2-4 GB	Certificates and logs
Grafana	4-8 GB	Dashboards and database
Prometheus	10-50 GB	Metric data grows over time
Uptime Kuma	2-4 GB	SQLite database
Syncthing	4 GB + data volume	Separate mount for synced data
PBS	50-500 GB	Depends on backup volume
Web application	4-10 GB	Application + dependencies

Resize a disk:

```
# Increase container 100's root disk by 10 GB (online, no restart needed)
pct resize 100 rootfs +10G
```

Add a mount point for data separation:

```
# Add a second disk for data (e.g., Prometheus metrics)
pct set 201 --mp0 local-lvm:20,mp=/var/lib/prometheus
```

> **Why this matters:** Separating application data from the root filesystem makes backups more efficient, prevents a runaway log file from filling the root disk, and makes it easier to resize or migrate data independently.

Networking Containers

Bridge Networking (Default)

All containers connect to `vmbr0`, the Linux bridge that Proxmox creates during installation. This puts containers on the same network as the host — they get their own IP on your LAN and can communicate with everything.

```
Internet ← Router (.1) ← Switch ← vmbr0 ← Container (.20)
                                    ↑
                              Proxmox Host (.10)
```

This is the right configuration for most homelab setups.

Static IP Configuration

Static IPs are set during container creation or modified afterward:

```
# Set static IP on an existing container
pct set 100 --net0 name=eth0,bridge=vmbr0,ip=192.168.1.20/24,gw=192.168.1.1
```

```
# To use DHCP instead (combined with a DHCP reservation on your router)
pct set 100 --net0 name=eth0,bridge=vmbr0,ip=dhcp
```

Inside the container, verify networking:

```
# Check IP address
ip addr show eth0

# Test connectivity
ping -c 3 192.168.1.1     # Gateway
ping -c 3 8.8.8.8          # External IP
ping -c 3 google.com       # DNS resolution
```

DNS Configuration

Container DNS is configured in `/etc/resolv.conf` but managed by Proxmox. To change DNS settings:

```
# From the host
pct set 100 --nameserver "192.168.1.20 1.1.1.1" --searchdomain lab
```

After you set up your AdGuard DNS container in Chapter 4, you will point all containers to it:

```
# Point container DNS to your local DNS server
pct set 100 --nameserver "192.168.1.20"
```

Container Lifecycle Management

Starting and Stopping

```
pct start 100          # Start container
pct shutdown 100       # Graceful shutdown (sends SIGTERM, waits)
pct stop 100           # Forced stop (like pulling the power cord)
pct reboot 100         # Graceful reboot
```

> **Common mistake:** Using `pct stop` for routine shutdowns. This is a hard kill — processes do not get a chance to flush data or close connections. Always use `pct shutdown` unless the container is unresponsive.

Cloning

Cloning creates a copy of an existing container. Useful for testing changes or creating new instances from a base template.

```
# Full clone (independent copy)
pct clone 100 150 --full --hostname ct-dns-test --target node-01

# Linked clone (shares base image, uses less space, cannot be migrated)
pct clone 100 150 --hostname ct-dns-test
```

Snapshots

Snapshots capture the container state at a point in time. Take a snapshot before making risky changes:

```
# Create a snapshot
pct snapshot 100 before-upgrade --description "Before AdGuard upgrade"

# List snapshots
pct listsnapshot 100

# Rollback to a snapshot (container must be stopped)
pct rollback 100 before-upgrade

# Delete a snapshot (reclaims disk space)
pct delsnapshot 100 before-upgrade
```

> **Why this matters:** Snapshots are your "undo button." Before upgrading software, changing configuration, or doing anything you are not 100% sure about — take a snapshot. Rolling back takes seconds. Rebuilding from scratch takes hours.

Migration

Move containers between nodes:

```
# Offline migration (container must be stopped)
pct migrate 100 node-02

# Online migration (requires shared storage)
pct migrate 100 node-02 --online
```

For most homelab setups without shared storage, offline migration is the way to go. Stop the container, migrate, start on the new node. The downtime is typically under a minute.

Destroying Containers

```
# Container must be stopped first
pct stop 100

# Remove (with confirmation)
pct destroy 100

# Remove including all backups and snapshots
pct destroy 100 --purge
```

Tagging and Organization

Tags are metadata labels on containers. They cost nothing to add and make managing a growing fleet dramatically easier.

Tag Strategy

Use a two-axis tagging system:

Axis	Tags	Purpose
Role	`infra`, `app`, `monitoring`, `backup`	What category of service
Function	`dns`, `proxy`, `grafana`, `pbs`, `web`	What specific service
Criticality	`core`	Services that other services depend on

Applying Tags

```
# During creation
pct create 100 ... --tags infra,dns,core

# On an existing container
pct set 100 --tags infra,dns,core
```

In the web UI, tags appear as colored labels next to each container. You can filter the container list by tag — invaluable when you have 15+ containers.

Example Tag Assignments

CTID	Hostname	Tags
100	ct-dns	`infra`, `dns`, `core`
101	ct-proxy	`infra`, `proxy`, `core`
200	ct-grafana	`monitoring`, `grafana`
201	ct-prometheus	`monitoring`, `prometheus`, `core`
202	ct-uptime	`monitoring`, `uptime`
300	ct-pbs	`backup`, `pbs`, `core`
301	ct-sync	`infra`, `sync`
302	ct-git	`infra`, `git`

Privileged vs Unprivileged Containers

Unprivileged (Default and Recommended)

Unprivileged containers map UIDs inside the container to unprivileged UIDs on the host. Root inside the container (UID 0) maps to an unprivileged user on the host (e.g., UID 100000). This means even if a process escapes the container, it has no host privileges.

```
# Create unprivileged (default in Proxmox 8+)
pct create 100 ... --unprivileged 1
```

Privileged (When Required)

Some workloads require privileged containers because they need direct access to host resources:

- NFS servers (need kernel NFS support)
- Certain hardware passthrough scenarios
- Legacy applications that require specific kernel features

```
# Create privileged
pct create 100 ... --unprivileged 0
```

> **Why this matters:** Always start with unprivileged. Only switch to privileged if you hit a specific error that requires it, and understand the security implications. In practice, 95% of homelab services run fine in unprivileged containers.

Container Templates: Building Your Own

After configuring a base container the way you like it (with your SSH keys, common packages, timezone, etc.), you can turn it into a template for rapid deployment.

Create a Base Container

```
# Create a base container
pct create 9000 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname template-debian12 \
    --cores 1 \
    --memory 512 \
    --rootfs local-lvm:4 \
    --net0 name=eth0,bridge=vmbr0,ip=dhcp \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --start 0

# Start it and install your base packages
pct start 9000
pct enter 9000
```

Inside the container:

```
# Update packages
apt update && apt full-upgrade -y

# Install common utilities
apt install -y \
    curl \
    wget \
    vim \
    htop \
```

```
    git \
    unzip \
    ca-certificates \
    gnupg \
    lsb-release \
    sudo

# Set timezone
timedatectl set-timezone America/New_York

# Clean up
apt clean
rm -rf /var/lib/apt/lists/*

# Exit
exit
```

Convert to Template

```
# Stop the container
pct shutdown 9000

# Convert to template (irreversible — the container becomes read-only)
pct template 9000
```

Deploy From Template

```
# Clone from template to create a new container
pct clone 9000 100 --full \
    --hostname ct-dns \
    --target node-01

# Customize the new container
pct set 100 --net0 name=eth0,bridge=vmbr0,ip=192.168.1.20/24,gw=192.168.1.1
pct set 100 --nameserver "192.168.1.1"
pct set 100 --tags infra,dns,core
pct set 100 --onboot 1
```

This gives you a consistent base across all containers: same packages, same SSH keys, same timezone.

Batch Operations

As your lab grows, you will want to manage containers in bulk.

Update All Containers

```bash
#!/bin/bash
# update-all-containers.sh
# Run from a Proxmox host

for ctid in $(pct list | tail -n +2 | awk '{print $1}'); do
    status=$(pct status $ctid | awk '{print $2}')
    if [ "$status" = "running" ]; then
        echo "=== Updating container $ctid ==="
        pct exec $ctid -- bash -c 'apt update && apt full-upgrade -y && apt autoremove -y'
    else
        echo "=== Skipping $ctid (not running) ===="
    fi
done
```

Check Container Resource Usage

```bash
#!/bin/bash
# container-status.sh

printf "%-6s %-20s %-8s %-12s %-12s %-8s\n" "CTID" "HOSTNAME" "STATUS" "MEM USED" "MEM MAX" "DISK"
echo "----------------------------------------------------------------------"

for ctid in $(pct list | tail -n +2 | awk '{print $1}'); do
    hostname=$(pct config $ctid | grep "^hostname:" | awk '{print $2}')
    status=$(pct status $ctid | awk '{print $2}')

    if [ "$status" = "running" ]; then
        mem_used=$(pct exec $ctid -- free -m | awk '/^Mem:/{print $3}')
        mem_total=$(pct exec $ctid -- free -m | awk '/^Mem:/{print $2}')
        disk=$(pct exec $ctid -- df -h / | awk 'NR==2{print $3"/"$2}')
        printf "%-6s %-20s %-8s %-12s %-12s %-8s\n" "$ctid" "$hostname" "$status" "${mem_used}M" "${mem
    else
        printf "%-6s %-20s %-8s %-12s %-12s %-8s\n" "$ctid" "$hostname" "$status" "-" "-" "-"
    fi
done
```

Troubleshooting

Container Won't Start: "Permission denied"

Usually means an unprivileged container is trying to do something privileged:

```bash
# Check the task log in the web UI, or:
journalctl -u pve-container@100 --no-pager | tail -20
```

If the error involves `/dev/` access or specific capabilities, you may need to add features:

```
pct set 100 --features nesting=1
# or for specific capabilities:
# pct set 100 --features nesting=1,keyctl=1
```

Container Has No Network

```
# Verify bridge exists on host
ip link show vmbr0

# Check container network config
pct config 100 | grep net

# Inside the container
ip addr show
ip route show
```

Common causes: - Wrong bridge name in container config - Missing gateway - DHCP server not responding (if using DHCP)

Container Runs Out of Disk

```
# Check disk usage inside container
pct exec 100 -- df -h

# Resize from the host (online, no restart)
pct resize 100 rootfs +5G
```

"Too many open files" or "No space left on device" (but disk isn't full)

Check inode usage:

```
pct exec 100 -- df -i
```

If inodes are exhausted (100% usage), you have too many tiny files. Common with some applications that create excessive temp files.

Chapter Checkpoint

You just learned to create and manage the building blocks of your entire lab. Before you move on, take 60 seconds to verify you can actually work with them.

Before you move on, verify:

- ☐ You can create, start, stop, and enter an LXC container
- ☐ Containers get the IP address you assigned
- ☐ DNS resolves from inside a container (`nslookup google.com`)
- ☐ You have a template container ready to clone

You now know how to create, configure, and manage LXC containers. The next chapter puts them to work — deploying the core services that make your homelab actually useful.

Next: Chapter 4 — Core Services

Chapter 4 — Core Services

This chapter deploys the foundational services that every production homelab needs: DNS for name resolution, a reverse proxy for clean URLs and TLS, and file synchronization for keeping data consistent across devices. These are the services that every other service depends on — get them right, and everything else is easier.

DNS: AdGuard Home

Why You Need Local DNS

Without local DNS, you access services by IP address: `http://192.168.1.30:3000`. With local DNS, you access them by name: `http://grafana.lab`. This is not just convenience — it is how production infrastructure works, and it saves you from memorizing IP addresses.

AdGuard Home vs Pi-hole:

Feature	AdGuard Home	Pi-hole
DNS-over-HTTPS/TLS	Built-in	Requires extra setup
Web UI	Modern, responsive	Functional but dated
Local DNS rewrites	Straightforward UI	Requires editing config files
DHCP server	Built-in	Built-in
Resource usage	~50 MB RAM	~100 MB RAM
Configuration format	YAML (easy to back up)	Multiple files

My recommendation: AdGuard Home. The DNS rewrite feature alone makes it the better choice for a homelab — you can add `grafana.lab → 192.168.1.30` in the web UI in seconds.

Create the DNS Container

```
# On node-01
pct create 100 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-dns \
    --cores 1 \
    --memory 512 \
    --swap 256 \
```

```
    --rootfs local-lvm:4 \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.20/24,gw=192.168.1.1 \
    --nameserver 1.1.1.1 \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 1 \
    --tags infra,dns,core

pct start 100
```

> **Note:** We set the nameserver to `1.1.1.1` (Cloudflare) for this container because it IS the DNS server — it cannot point to itself during initial setup.

Install AdGuard Home

```
pct enter 100

# Install AdGuard Home
curl -s -S -L https://raw.githubusercontent.com/AdguardTeam/AdGuardHome/master/scripts/install.sh | sh

# Verify it's running
systemctl status AdGuardHome
```

AdGuard Home is now accessible at `http://192.168.1.20:3000` for initial setup.

Initial Configuration

1. Open `http://192.168.1.20:3000` in your browser
2. Click "Get Started"
3. **Admin interface:** Listen on all interfaces, port 3000 (or change to 80 if you prefer)
4. **DNS server:** Listen on all interfaces, port 53
5. Set admin username and password
6. Complete the wizard

Configure Upstream DNS

In the AdGuard Home web UI:

1. Settings > DNS Settings
2. Upstream DNS servers:

```
https://dns.cloudflare.com/dns-query
https://dns.google/dns-query
1.1.1.1
8.8.8.8
```

3. Bootstrap DNS servers:

```
1.1.1.1
8.8.8.8
```

4. Click "Test upstreams" to verify, then "Apply"

Add Local DNS Rewrites

This is where AdGuard Home shines for homelab use. Go to Filters > DNS Rewrites > Add:

Domain	Answer
`node-01.lab`	`192.168.1.10`
`node-02.lab`	`192.168.1.11`
`node-03.lab`	`192.168.1.12`
`dns.lab`	`192.168.1.20`
`proxy.lab`	`192.168.1.21`
`grafana.lab`	`192.168.1.21`
`prometheus.lab`	`192.168.1.21`
`uptime.lab`	`192.168.1.21`
`sync.lab`	`192.168.1.21`
`pbs.lab`	`192.168.1.40`

Notice that `grafana.lab`, `prometheus.lab`, `uptime.lab`, and `sync.lab` all point to the **reverse proxy** IP (`192.168.1.21`), not to the individual services. The reverse proxy will route traffic to the correct backend based on the hostname. More on that in the Traefik section.

Point Your Network at AdGuard

Option A: Router-level (recommended)

Change your router's DHCP settings to hand out `192.168.1.20` as the DNS server. Every device on your network will automatically use it.

On most routers: 1. LAN > DHCP Settings 2. DNS Server: `192.168.1.20` 3. Secondary DNS: `1.1.1.1` (fallback if AdGuard is down)

Option B: Per-container

Update each Proxmox container to use the DNS server:

```
# On the Proxmox host, for each container
pct set 101 --nameserver "192.168.1.20"
pct set 200 --nameserver "192.168.1.20"
# ... etc
```

Option C: Per-host

On each Proxmox node, add DNS to your network interface configuration in `/etc/network/interfaces`:

```
# Add these lines to your bridge interface (vmbr0) stanza:
    dns-nameservers 192.168.1.20 1.1.1.1
    dns-search lab
```

Then apply:

```
systemctl restart networking
```

Do not edit `/etc/resolv.conf` directly on Proxmox hosts. It gets overwritten on reboot or network restart. The `/etc/network/interfaces` method persists across reboots.

Common mistake: Not setting a fallback DNS server. If your AdGuard container goes down and it is the only DNS server, everything on your network loses DNS resolution — including the Proxmox web UI. Always have a fallback (`1.1.1.1` or `8.8.8.8`).

Verify DNS

```
# Test from any machine on the network
nslookup grafana.lab 192.168.1.20

# Should return:
# Name:    grafana.lab
# Address: 192.168.1.21

# Test external resolution
nslookup google.com 192.168.1.20
```

Back Up the Configuration

AdGuard Home stores its configuration in a single YAML file:

```
# Inside the container
cat /opt/AdGuardHome/AdGuardHome.yaml
```

Back this file up. If you ever need to rebuild the DNS container, restoring this file restores your entire configuration — upstream servers, rewrites, blocklists, everything.

Reverse Proxy: Traefik

Why a Reverse Proxy

Without a reverse proxy: - `http://192.168.1.30:3000` — Grafana - `http://192.168.1.32:3001` — Uptime Kuma - `http://192.168.1.41:8384` — Syncthing

With a reverse proxy: - `https://grafana.lab` — Grafana - `https://uptime.lab` — Uptime Kuma - `https://sync.lab` — Syncthing

A reverse proxy gives you: - **Clean URLs** — No port numbers to remember - **TLS termination** — HTTPS for everything, even internal services - **Single entry point** — One IP handles all web traffic - **Access control** — Middleware for authentication, rate limiting, etc.

Traefik vs Nginx Proxy Manager:

Feature	Traefik	Nginx Proxy Manager
Configuration	File-based (YAML/TOML)	Web UI (click-based)
Learning curve	Moderate	Low

Feature	Traefik	Nginx Proxy Manager
Dynamic configuration	Excellent (file watcher)	Manual through UI
Professional relevance	Industry standard	Not commonly used in enterprise
Middleware	Rich (headers, auth, rate limiting)	Basic
Resource usage	~30 MB RAM	~100 MB RAM

My recommendation: Traefik. The configuration-as-code approach is a professional skill, the file-based config is easy to back up and version control, and it is what you will encounter in production environments. If you strongly prefer a GUI, Nginx Proxy Manager works fine — but you will not learn as much.

Create the Proxy Container

```
# On node-01
pct create 101 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-proxy \
    --cores 2 \
    --memory 512 \
    --swap 256 \
    --rootfs local-lvm:4 \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.21/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 1 \
    --tags infra,proxy,core

pct start 101
pct enter 101
```

Install Traefik

```
# Download Traefik binary
TRAEFIK_VERSION="3.2"
wget -q "https://github.com/traefik/traefik/releases/download/v${TRAEFIK_VERSION}.0/traefik_v${TRAEFIK_
tar xzf traefik_v${TRAEFIK_VERSION}.0_linux_amd64.tar.gz
mv traefik /usr/local/bin/
chmod +x /usr/local/bin/traefik
rm traefik_v${TRAEFIK_VERSION}.0_linux_amd64.tar.gz

# Create directory structure
mkdir -p /etc/traefik/dynamic
mkdir -p /etc/traefik/certs
```

Generate Local TLS Certificates with mkcert

For internal services, you do not need Let's Encrypt. Use `mkcert` to create locally-trusted certificates:

```
# Install mkcert
apt install -y libnss3-tools

wget -q "https://dl.filippo.io/mkcert/latest?for=linux/amd64" -O /usr/local/bin/mkcert
chmod +x /usr/local/bin/mkcert

# Install the local CA
mkcert -install

# Generate a wildcard certificate for your lab domain
cd /etc/traefik/certs
mkcert "*.lab" "lab"

# This creates two files:
# _wildcard.lab+1.pem     (certificate)
# _wildcard.lab+1-key.pem (private key)
```

Install the CA on your workstation:

The root CA is at `/root/.local/share/mkcert/rootCA.pem` inside the container. Copy it to your workstation and install it:

```
# From your workstation
scp root@192.168.1.21:/root/.local/share/mkcert/rootCA.pem ~/lab-ca.pem
```

- **Windows:** Double-click the `.pem` file > Install Certificate > Local Machine > Trusted Root Certification Authorities
- **macOS:** `sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain ~/lab-ca.pem`
- **Linux (Chrome):** Settings > Privacy and Security > Security > Manage Certificates > Authorities > Import

After installing the CA, your browser will trust `https://*.lab` without warnings.

Configure Traefik

Static configuration (`/etc/traefik/traefik.yml`):

```
# /etc/traefik/traefik.yml
global:
  checkNewVersion: false
  sendAnonymousUsage: false

api:
  dashboard: true
  insecure: false
```

```yaml
entryPoints:
  web:
    address: ":80"
    http:
      redirections:
        entryPoint:
          to: websecure
          scheme: https
  websecure:
    address: ":443"

providers:
  file:
    directory: /etc/traefik/dynamic
    watch: true

log:
  level: INFO
  filePath: /var/log/traefik/traefik.log

accessLog:
  filePath: /var/log/traefik/access.log
```

```bash
mkdir -p /var/log/traefik
```

Dynamic configuration — routes (`/etc/traefik/dynamic/routes.yml`):

```yaml
# /etc/traefik/dynamic/routes.yml
http:
  routers:
    grafana:
      rule: "Host(`grafana.lab`)"
      service: grafana
      entryPoints:
        - websecure
      tls: {}

    prometheus:
      rule: "Host(`prometheus.lab`)"
      service: prometheus
      entryPoints:
        - websecure
      tls: {}

    uptime:
      rule: "Host(`uptime.lab`)"
      service: uptime
      entryPoints:
        - websecure
```

```yaml
      tls: {}

    sync:
      rule: "Host(`sync.lab`)"
      service: sync
      entryPoints:
        - websecure
      tls: {}

    traefik-dashboard:
      rule: "Host(`proxy.lab`)"
      service: api@internal
      entryPoints:
        - websecure
      tls: {}

  services:
    grafana:
      loadBalancer:
        servers:
          - url: "http://192.168.1.30:3000"

    prometheus:
      loadBalancer:
        servers:
          - url: "http://192.168.1.31:9090"

    uptime:
      loadBalancer:
        servers:
          - url: "http://192.168.1.32:3001"

    sync:
      loadBalancer:
        servers:
          - url: "http://192.168.1.41:8384"

tls:
  certificates:
    - certFile: /etc/traefik/certs/_wildcard.lab+1.pem
      keyFile: /etc/traefik/certs/_wildcard.lab+1-key.pem
```

Why TLS is in the dynamic config: TLS certificates belong in the dynamic configuration, not the static `traefik.yml`. This lets you update or rotate certificates without restarting Traefik. The `tls: {}` on each router tells it to use HTTPS; the certificate block at the bottom tells Traefik *which* certificate to serve.

Why this matters: The dynamic configuration file is watched by Traefik. Adding a new service means adding a few lines to this file — no restart required. Traefik picks up

changes within seconds. This is the "infrastructure as code" approach applied to your homelab.

Create a Systemd Service

```
cat > /etc/systemd/system/traefik.service << 'EOF'
[Unit]
Description=Traefik Reverse Proxy
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/local/bin/traefik --configFile=/etc/traefik/traefik.yml
Restart=on-failure
RestartSec=5
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now traefik
systemctl status traefik
```

Verify Traefik

1. Navigate to `https://proxy.lab` (the Traefik dashboard)
2. You should see your configured routers and services
3. Test a route: `https://grafana.lab` should proxy to Grafana (once Grafana is installed)

Adding New Services

Whenever you deploy a new service, add it to `/etc/traefik/dynamic/routes.yml`:

```yaml
# Add router
new-service:
  rule: "Host(`newservice.lab`)"
  service: new-service
  entryPoints:
    - websecure
  tls: {}

# Add service
new-service:
  loadBalancer:
    servers:
      - url: "http://192.168.1.XX:PORT"
```

Then add a DNS rewrite in AdGuard pointing `newservice.lab` to `192.168.1.21` (the proxy IP). No restarts needed for either service.

> **Interview angle:** "I run Traefik as a reverse proxy with file-based dynamic configuration — adding a new service is a few lines of YAML, no restart required. All internal traffic runs over TLS using a local CA managed with mkcert. DNS resolution is handled by AdGuard Home with local rewrites, so every service is accessible by name. It is the same pattern you would see in a production environment, just at homelab scale."

Alternative: Nginx Proxy Manager

If you prefer a GUI-based reverse proxy, Nginx Proxy Manager (NPM) is a popular alternative to Traefik. Instead of YAML configuration files, you manage everything through a web UI.

> **Recommendation:** We covered Traefik in depth because it is configuration-as-code — reproducible, version-controllable, and scriptable. NPM is easier to get started with but harder to automate, back up, and reproduce. If you are building a resume-worthy homelab, Traefik is the stronger choice. If you just want something working quickly, NPM is fine.

NPM is typically deployed as a Docker container (their recommended path), which does not fit this guide's LXC-first approach. If you want to run it anyway, see the official NPM installation guide — it walks you through the Docker Compose setup in minutes. The web UI runs on port 81 with default credentials `admin@example.com` / `changeme`.

Nginx Proxy Manager runs a web UI on port 81. You configure routes through the GUI by clicking "Add Proxy Host" and filling in the domain, target IP, and port.

The GUI approach is simpler to start with, but the configuration is stored in a database rather than a text file, making it harder to back up, version control, and reproduce.

File Synchronization: Syncthing

Why Syncthing

Syncthing synchronizes files between devices in real time, peer-to-peer, with no cloud dependency. Use cases in a homelab:

- Sync documents/notes between your workstation and a server
- Sync configuration files across devices
- Keep a knowledge base (Obsidian vault, wiki, etc.) synchronized
- Sync backups of important files to a second location

Syncthing vs alternatives:

Tool	How It Works	Pros	Cons
Syncthing	P2P sync, encrypted	No cloud, no cost, open source	Requires running on both ends
Nextcloud	Server-hosted sync	Full office suite, calendar, contacts	Heavy (PHP + database), complex
rsync + cron	One-way copy on schedule	Simple, built-in	Not real-time, one-directional
Resilio Sync	P2P sync	Fast, easy	Proprietary, paid for features

Create the Syncthing Container

```
# On node-03
pct create 301 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-sync \
    --cores 2 \
    --memory 1024 \
    --swap 512 \
    --rootfs local-lvm:4 \
    --mp0 local-lvm:50,mp=/data \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.41/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 1 \
    --tags infra,sync

pct start 301
pct enter 301
```

Note the `--mp0` flag — we added a 50 GB mount point at `/data` for synced files, separate from the root filesystem.

Install Syncthing

```
# Add the Syncthing release PGP key and repository
curl -L -o /etc/apt/keyrings/syncthing-archive-keyring.gpg \
    https://syncthing.net/release-key.gpg

echo "deb [signed-by=/etc/apt/keyrings/syncthing-archive-keyring.gpg] \
    https://apt.syncthing.net/ syncthing stable" \
    > /etc/apt/sources.list.d/syncthing.list

# Install
```

```
apt update
apt install -y syncthing
```

Configure Syncthing

Syncthing's web UI defaults to listening on `127.0.0.1:8384` — only accessible from localhost. Change it to listen on all interfaces:

```
# Generate initial config
syncthing generate --config=/root/.local/state/syncthing

# Edit the config to allow remote access
sed -i 's|<address>127.0.0.1:8384</address>|<address>0.0.0.0:8384</address>|' \
    /root/.local/state/syncthing/config.xml
```

Create a Systemd Service

```
cat > /etc/systemd/system/syncthing.service << 'EOF'
[Unit]
Description=Syncthing File Synchronization
After=network.target

[Service]
Type=simple
User=root
ExecStart=/usr/bin/syncthing serve --no-browser --config=/root/.local/state/syncthing --data=/root/.loc
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now syncthing
```

Initial Setup via Web UI

1. Access `http://192.168.1.41:8384` (or `https://sync.lab` once your proxy route is set up)
2. Set a GUI password when prompted (Settings > GUI > GUI Authentication User/Password)
3. The Device ID shown on the main screen is what you will use to pair other devices

Add a Shared Folder

In the Syncthing web UI:

1. Click "Add Folder"
2. Folder Label: `documents` (human-readable name)
3. Folder Path: `/data/documents`
4. Sharing: Select the devices to share with (after pairing)

Pair Devices

To sync with your workstation:

1. Install Syncthing on your workstation (syncthing.net/downloads)
2. Get your workstation's Device ID from its Syncthing web UI
3. On the server Syncthing UI: Add Remote Device > paste the workstation's Device ID
4. On the workstation: Accept the connection request
5. Share the folder with the new device

Syncthing handles the rest — files are synchronized in real time over your LAN.

Syncthing Security Notes

- Syncthing traffic between devices is encrypted (TLS 1.3)
- Discovery and relay servers are used only for NAT traversal; your data does not pass through them on a LAN
- If you want to disable global discovery (purely local sync): Settings > Connections > uncheck "Global Discovery" and "Enable Relaying"

Version Control: Nightly Git Push

Even if you do not run a local Git server, you should have your important configurations in version control. A simple nightly push script backs up your work to GitHub/GitLab.

Set Up a Repository

```
# On a container (e.g., ct-git on node-03)
mkdir -p /root/projects/homelab
cd /root/projects/homelab
git init
```

Create a basic structure:

```
homelab/
  state/
    cluster-state.md       # Current cluster config
    network-state.md       # IP scheme, DHCP, DNS
    services-state.md      # What runs where
    backup-state.md        # Backup configuration
  configs/
    traefik/               # Traefik config copies
    adguard/               # AdGuard backup YAML
    prometheus/            # Prometheus config
    grafana/               # Grafana dashboard JSON exports
  scripts/
    update-all.sh          # Container update script
    nightly-push.sh        # This script
```

Nightly Push Script

```bash
#!/bin/bash
# /root/scripts/nightly-push.sh
# Pushes homelab repo to remote every night

set -euo pipefail

LOG="/var/log/nightly-push.log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG"
}

log "Starting nightly push"

cd /root/projects/homelab

# Auto-commit any changes
if [ -n "$(git status --porcelain)" ]; then
    git add -A
    git commit -m "auto: nightly backup $(date '+%Y-%m-%d')"
    log "Changes committed"
else
    log "No changes to commit"
fi

# Push to remote
if git push origin main 2>> "$LOG"; then
    log "Push successful"
else
    log "ERROR: Push failed"
    # Optional: send email notification on failure
fi

log "Nightly push complete"
```

Cron Schedule

```bash
# Edit crontab
crontab -e

# Add nightly push at 3:15 AM
15 3 * * * /root/scripts/nightly-push.sh
```

Authentication

For GitHub/GitLab over HTTPS, use a personal access token or deploy key:

```
# Install GitHub CLI (optional but convenient)
apt install -y gh

# Authenticate
gh auth login

# Or set remote with token directly
git remote add origin https://<token>@github.com/your-username/homelab.git
```

For SSH-based authentication:

```
# Generate a deploy key
ssh-keygen -t ed25519 -f /root/.ssh/github_deploy -N ""

# Add the public key to your GitHub repo as a deploy key
cat /root/.ssh/github_deploy.pub

# Configure SSH to use this key for GitHub
cat >> /root/.ssh/config << 'EOF'
Host github.com
    IdentityFile /root/.ssh/github_deploy
    IdentitiesOnly yes
EOF

# Set remote to SSH
git remote add origin git@github.com:your-username/homelab.git
```

> **Why this matters:** Your homelab configuration is intellectual property — the accumulated knowledge of how your infrastructure works. Losing it to a disk failure means rebuilding from memory. A nightly git push costs nothing and takes seconds.

Service Dependency Map

Now that core services are deployed, here is how they relate:

```
All containers → DNS (ct-dns, .20) → External DNS
     ↓
All web UIs → Reverse Proxy (ct-proxy, .21) → Backend services
     ↓
Important files → Syncthing (ct-sync, .41) ↔ Your workstation
     ↓
Configurations → Git (ct-git, .42) → GitHub/GitLab
```

Startup order matters. When the cluster boots:

1. DNS should start first (other containers need name resolution)
2. Reverse proxy starts next (so services are accessible by name)
3. Everything else can start in any order

Set boot order in Proxmox:

```
# DNS starts first (order 1)
pct set 100 --startup order=1

# Proxy starts second
pct set 101 --startup order=2

# Everything else starts later
pct set 200 --startup order=3
pct set 201 --startup order=3
# ...
```

Chapter Checkpoint

You just deployed the three services everything else depends on — DNS, reverse proxy, and file sync. Before you move on, take 60 seconds to make sure the foundation is solid.

Before you move on, verify:

- ☐ AdGuard DNS resolves local `.lab` domains from any machine on the network
- ☐ Traefik routes traffic to at least one service via HTTPS
- ☐ You can add a new service to Traefik without restarting anything
- ☐ Syncthing shows "Up to Date" for at least one shared folder

---

Next: Chapter 5 — Monitoring and Alerting

Chapter 5 — Monitoring and Alerting

It's 2 AM. Your Grafana dashboard is green. You're asleep. That's the goal of this chapter.

Without monitoring, you find out something is broken when you try to use it — three days after it went down. With monitoring, your phone buzzes the moment disk hits 85%, a node drops offline, or a service stops responding. You glance at it, decide if it's urgent, and go back to sleep. That is the difference between running infrastructure and babysitting servers.

This chapter builds a complete monitoring stack: metrics collection, visualization, alerting, and uptime checking.

Architecture Overview

```
┌──────────────────────────────────────────────────────────┐
│                    Grafana (.30)                         │
│             Dashboards + Alert Rules                     │
│                   ▲                                      │
│                   │ queries                              │
│             Prometheus (.31)                             │
│             Scrapes metrics every 15s                    │
│          ▲         ▲         ▲         ▲               │
│          │         │         │         │               │
│    node_exporter  node_exporter  PVE exporter            │
│    (every CT)     (every host)   (PVE API)               │
│                                                          │
│             Uptime Kuma (.32)                            │
│             HTTP health checks                           │
│             ▼         ▼         ▼                        │
│           Services  Services  Services                   │
│                                                          │
│             Notifications → Email / Discord / Webhook    │
└──────────────────────────────────────────────────────────┘
```

The stack has three components:

1. **Prometheus** — Time-series database that pulls ("scrapes") metrics from targets
2. **Grafana** — Visualization and alerting platform that queries Prometheus
3. **Uptime Kuma** — Lightweight HTTP/TCP uptime monitor with its own alerting

---

Prometheus

Why Prometheus

Prometheus is the de facto standard for infrastructure monitoring. It uses a pull-based model — Prometheus reaches out to each target and collects metrics. This is simpler and more reliable than push-based systems where every host needs to know where to send data.

Create the Prometheus Container

```
# On node-02
pct create 201 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-prometheus \
    --cores 2 \
    --memory 2048 \
    --swap 512 \
    --rootfs local-lvm:8 \
    --mp0 local-lvm:30,mp=/var/lib/prometheus \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.31/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 1 \
    --tags monitoring,prometheus,core

pct start 201
pct enter 201
```

Note the separate mount point for Prometheus data (`/var/lib/prometheus`). Metric data grows over time — separating it from the root filesystem prevents a full data disk from crashing the OS.

Install Prometheus

```
# Create prometheus user
useradd --no-create-home --shell /bin/false prometheus

# Download Prometheus
PROM_VERSION="2.54.1"
wget -q "https://github.com/prometheus/prometheus/releases/download/v${PROM_VERSION}/prometheus-${PROM_
tar xzf prometheus-${PROM_VERSION}.linux-amd64.tar.gz
cd prometheus-${PROM_VERSION}.linux-amd64

# Install binaries
cp prometheus promtool /usr/local/bin/
cp -r consoles console_libraries /etc/prometheus/
```

```bash
# Set ownership
chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus
chown prometheus:prometheus /usr/local/bin/prometheus /usr/local/bin/promtool

# Clean up
cd ..
rm -rf prometheus-${PROM_VERSION}.linux-amd64*
```

Configure Prometheus

```bash
mkdir -p /etc/prometheus
```

Create `/etc/prometheus/prometheus.yml`:

```yaml
# /etc/prometheus/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  # Prometheus monitors itself
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  # Proxmox hosts (node_exporter)
  - job_name: 'proxmox-hosts'
    static_configs:
      - targets:
          - '192.168.1.10:9100'
          - '192.168.1.11:9100'
          - '192.168.1.12:9100'
        labels:
          group: 'proxmox-hosts'

  # LXC containers (node_exporter)
  - job_name: 'containers'
    static_configs:
      - targets:
          - '192.168.1.20:9100'   # ct-dns
          - '192.168.1.21:9100'   # ct-proxy
          - '192.168.1.30:9100'   # ct-grafana
          - '192.168.1.32:9100'   # ct-uptime
          - '192.168.1.40:9100'   # ct-pbs
          - '192.168.1.41:9100'   # ct-sync
        labels:
          group: 'containers'
```

```yaml
  # Proxmox VE exporter (Proxmox-specific metrics)
  - job_name: 'pve'
    static_configs:
      - targets: ['192.168.1.10:9221']
    metrics_path: /pve
    params:
      module: [default]
      cluster: ['1']
      node: ['1']
```

Create Systemd Service

```bash
cat > /etc/systemd/system/prometheus.service << 'EOF'
[Unit]
Description=Prometheus Monitoring System
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=prometheus
Group=prometheus
ExecStart=/usr/local/bin/prometheus \
    --config.file=/etc/prometheus/prometheus.yml \
    --storage.tsdb.path=/var/lib/prometheus \
    --storage.tsdb.retention.time=30d \
    --web.listen-address=:9090 \
    --web.enable-lifecycle
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now prometheus
```

Key flags: `- --storage.tsdb.retention.time=30d` — Keep 30 days of metrics. Adjust based on your disk space. `- --web.enable-lifecycle` — Allows configuration reload via HTTP POST (useful for automation).

Verify Prometheus

Open `http://192.168.1.31:9090` and check:

1. Status > Targets — All scrape targets should show "UP"
2. Try a query: `up` — Should return 1 for all targets
3. Try: `node_memory_MemTotal_bytes` — Should show memory for each host

---

node_exporter: Metrics on Every Host

node_exporter exposes system metrics (CPU, memory, disk, network) in a format Prometheus can scrape. Install it on every Proxmox host and every container you want to monitor.

Install on Proxmox Hosts

```
# SSH to each host and run:
apt install -y prometheus-node-exporter

# Enable and start
systemctl enable --now prometheus-node-exporter

# Verify
curl -s http://localhost:9100/metrics | head -20
```

Install on LXC Containers

```
# Inside each container:
apt install -y prometheus-node-exporter
systemctl enable --now prometheus-node-exporter
```

Customize node_exporter for Full Disk Coverage

By default, node_exporter excludes many filesystem mount points. If you have additional drives mounted (e.g., SATA drives at `/mnt/storage`), override the excluded mount points:

```
# Edit the defaults file
cat > /etc/default/prometheus-node-exporter << 'EOF'
ARGS="--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|run)($|/) --collector.diskstats.devic
EOF

systemctl restart prometheus-node-exporter
```

> **Why this matters:** The default exclusion regex for mount points includes `/mnt`, which means your SATA drives and additional storage mounts will not show up in Grafana. Overriding it to only exclude virtual filesystems (`/sys`, `/proc`, `/dev`, `/run`) ensures all real storage is monitored.

Verify node_exporter Targets

After installing node_exporter on all hosts and containers, check Prometheus:

1. Go to `http://192.168.1.31:9090`
2. Status > Targets
3. All targets should show "UP" with the last scrape time

If a target shows "DOWN": - Check if node_exporter is running on that host: `systemctl status prometheus-node-exporter` - Check if the port is accessible: `curl http://<target-ip>:9100/metrics` - Check firewall rules between the Prometheus container and the target

---

PVE Exporter: Proxmox-Specific Metrics

node_exporter gives you OS-level metrics. PVE exporter gives you Proxmox-specific metrics: container status, cluster health, storage pool usage, etc.

Create a Proxmox API Token

In the Proxmox web UI:

1. Datacenter > Permissions > API Tokens
2. Add:
   - User: `root@pam`
   - Token ID: `prometheus`
   - Privilege Separation: **unchecked** (the token inherits root permissions)
3. Copy the token value — you will need it in the next step

Install PVE Exporter

Install on one Proxmox host (not in a container — it needs access to the PVE API):

```
# On node-01
apt install -y python3-pip python3-venv

# Create a virtual environment
python3 -m venv /opt/pve-exporter
/opt/pve-exporter/bin/pip install prometheus-pve-exporter
```

Configure PVE Exporter

```
mkdir -p /etc/pve-exporter

cat > /etc/pve-exporter/pve.yml << 'EOF'
default:
    user: root@pam
    token_name: prometheus
    token_value: "your-token-value-here"
    verify_ssl: false
EOF

chmod 600 /etc/pve-exporter/pve.yml
```

Create Systemd Service

```
cat > /etc/systemd/system/pve-exporter.service << 'EOF'
[Unit]
Description=Prometheus PVE Exporter
After=network-online.target

[Service]
Type=simple
ExecStart=/opt/pve-exporter/bin/pve_exporter /etc/pve-exporter/pve.yml --listen-address 0.0.0.0 --liste
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now pve-exporter
```

Verify PVE Exporter

```
curl -s http://localhost:9221/pve | head -30
```

You should see metrics like `pve_up`, `pve_node_info`, `pve_guest_info`, etc.

Grafana

Grafana is where your monitoring comes to life — dashboards that show the health of your entire lab at a glance.

Create the Grafana Container

```
# On node-02
pct create 200 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-grafana \
    --cores 2 \
    --memory 1024 \
    --swap 512 \
    --rootfs local-lvm:8 \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.30/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 1 \
```

```
    --tags monitoring,grafana

pct start 200
pct enter 200
```

Install Grafana

```
# Install prerequisites
apt install -y apt-transport-https software-properties-common gnupg2 curl

# Add Grafana GPG key and repository
curl -fsSL https://apt.grafana.com/gpg.key | gpg --dearmor -o /etc/apt/keyrings/grafana.gpg
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" \
    > /etc/apt/sources.list.d/grafana.list

# Install
apt update
apt install -y grafana

# Enable and start
systemctl enable --now grafana-server
```

Configure Grafana

Edit `/etc/grafana/grafana.ini`:

```
[server]
http_port = 3000
domain = grafana.lab
root_url = https://grafana.lab

[security]
admin_user = admin
admin_password = your-secure-password-here

[smtp]
enabled = true
host = smtp.gmail.com:587
user = your-email@gmail.com
password = your-app-password
from_address = your-email@gmail.com
from_name = Homelab Grafana

[alerting]
enabled = true
```

```
systemctl restart grafana-server
```

> **Note on SMTP:** Gmail requires an "App Password" (not your regular password) when 2FA is enabled. Go to Google Account > Security > 2-Step Verification > App

passwords to generate one. Alternatively, use any SMTP relay service.

Add Prometheus as a Data Source

1. Log into Grafana at `https://grafana.lab` (default: admin/admin, or whatever you set)
2. Go to Connections > Data Sources > Add data source
3. Select "Prometheus"
4. URL: `http://192.168.1.31:9090`
5. Click "Save & Test" — should show "Data source is working"

Import Dashboards

Grafana has a library of community-created dashboards. The best ones for a homelab:

Node Exporter Full (Dashboard ID: 1860):

1. Dashboards > New > Import
2. Enter dashboard ID: `1860`
3. Select your Prometheus data source
4. Click Import

This gives you comprehensive host monitoring: CPU, memory, disk, network, filesystem, and more.

Proxmox VE (Dashboard ID: 10347):

1. Import dashboard ID: `10347`
2. Select your Prometheus data source

This shows Proxmox-specific metrics: guest status, cluster health, storage pools.

Create Custom Dashboard: Lab Overview

For a high-level "is everything healthy" view, create a custom dashboard:

1. Dashboards > New > New Dashboard
2. Add the following panels:

Panel 1 — Cluster Uptime (Stat)

Query:

```
up{job="proxmox-hosts"}
```

- Panel type: Stat
- Value mappings: 1 = "UP" (green), 0 = "DOWN" (red)

Panel 2 — Host CPU Usage (Time Series)

Query:

```
100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle",job="proxmox-hosts"}[5m])) * 100)
```

Panel 3 — Host Memory Usage (Gauge)

Query:

```
(1 - node_memory_MemAvailable_bytes{job="proxmox-hosts"} / node_memory_MemTotal_bytes{job="proxmox-host
```

- Thresholds: 0-70 green, 70-85 yellow, 85-100 red

Panel 4 — Disk Usage by Mount (Table)

Query:

```
100 - (node_filesystem_avail_bytes{fstype="ext4"} / node_filesystem_size_bytes{fstype="ext4"} * 100)
```

Panel 5 — Container Up/Down (Stat)

Query:

```
up{job="containers"}
```

Alert Rules

Dashboards show you what is happening now. Alerts tell you when something is wrong — even when you are not looking.

File-Provisioned Alerts

The most maintainable way to manage Grafana alerts is through file provisioning. This keeps your alert rules in version-controllable YAML files.

> **Important:** The `datasourceUid` values below (`prometheus`, `__expr__`) must match the UIDs of the datasources in *your* Grafana instance. Check **Configuration → Data Sources** in Grafana and look at the URL — the last path segment is the UID. If yours differs (e.g., an auto-generated hash like `ae4tgh12k`), update the YAML accordingly or provisioning will silently fail.

Create `/etc/grafana/provisioning/alerting/homelab-alerts.yml`:

```yaml
# /etc/grafana/provisioning/alerting/homelab-alerts.yml
apiVersion: 1

groups:
  - orgId: 1
    name: Homelab Alerts
    folder: Homelab
    interval: 1m
    rules:
      # Disk space critical (>85% full)
      - uid: disk-critical
        title: "Disk Space Critical"
        condition: C
        data:
          - refId: A
            relativeTimeRange:
              from: 300
              to: 0
            datasourceUid: prometheus
```

```yaml
      model:
        expr: >
          100 - (node_filesystem_avail_bytes{fstype="ext4"}
          / node_filesystem_size_bytes{fstype="ext4"} * 100)
        intervalMs: 1000
        maxDataPoints: 43200
    - refId: B
      relativeTimeRange:
        from: 300
        to: 0
      datasourceUid: __expr__
      model:
        conditions:
          - evaluator:
              params: [85]
              type: gt
            operator:
              type: and
            reducer:
              type: last
        type: classic_conditions
    - refId: C
      datasourceUid: __expr__
      model:
        expression: B
        type: classic_conditions
  noDataState: OK
  execErrState: Alerting
  for: 5m
  annotations:
    summary: "Disk usage above 85% on {{ $labels.instance }}"
    description: "Mount {{ $labels.mountpoint }} is {{ $value }}% full"
  labels:
    severity: critical

# Disk space warning (>70% full)
- uid: disk-warning
  title: "Disk Space Warning"
  condition: C
  data:
    - refId: A
      relativeTimeRange:
        from: 300
        to: 0
      datasourceUid: prometheus
      model:
        expr: >
          100 - (node_filesystem_avail_bytes{fstype="ext4"}
```

```yaml
          / node_filesystem_size_bytes{fstype="ext4"} * 100)
      - refId: B
        datasourceUid: __expr__
        model:
          conditions:
            - evaluator:
                params: [70]
                type: gt
          type: classic_conditions
      - refId: C
        datasourceUid: __expr__
        model:
          expression: B
          type: classic_conditions
    noDataState: OK
    execErrState: Alerting
    for: 10m
    annotations:
      summary: "Disk usage above 70% on {{ $labels.instance }}"
    labels:
      severity: warning

  # Node down
  - uid: node-down
    title: "Node Down"
    condition: C
    data:
      - refId: A
        datasourceUid: prometheus
        model:
          expr: up{job="proxmox-hosts"}
      - refId: B
        datasourceUid: __expr__
        model:
          conditions:
            - evaluator:
                params: [1]
                type: lt
          type: classic_conditions
      - refId: C
        datasourceUid: __expr__
        model:
          expression: B
          type: classic_conditions
    noDataState: Alerting
    execErrState: Alerting
    for: 1m
    annotations:
```

```yaml
    summary: "Node {{ $labels.instance }} is down"
  labels:
    severity: critical

# High CPU (>90% for 5 minutes)
- uid: high-cpu
  title: "High CPU Usage"
  condition: C
  data:
    - refId: A
      datasourceUid: prometheus
      model:
        expr: >
          100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
    - refId: B
      datasourceUid: __expr__
      model:
        conditions:
          - evaluator:
              params: [90]
              type: gt
        type: classic_conditions
    - refId: C
      datasourceUid: __expr__
      model:
        expression: B
        type: classic_conditions
  noDataState: OK
  execErrState: Alerting
  for: 5m
  annotations:
    summary: "CPU above 90% on {{ $labels.instance }}"
  labels:
    severity: warning

# High memory (>90%)
- uid: high-memory
  title: "High Memory Usage"
  condition: C
  data:
    - refId: A
      datasourceUid: prometheus
      model:
        expr: >
          (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100
    - refId: B
      datasourceUid: __expr__
      model:
```

```yaml
            conditions:
              - evaluator:
                  params: [90]
                  type: gt
            type: classic_conditions
          - refId: C
            datasourceUid: __expr__
            model:
              expression: B
              type: classic_conditions
        noDataState: OK
        execErrState: Alerting
        for: 5m
        annotations:
          summary: "Memory above 90% on {{ $labels.instance }}"
        labels:
          severity: warning
```

```bash
# Restart Grafana to load alerts
systemctl restart grafana-server
```

Configure Notification Channels

Email (contact point):

1. Alerting > Contact Points > Add contact point
2. Name: `email-alerts`
3. Type: Email
4. Addresses: `your-email@example.com`
5. Test and Save

Discord webhook (alternative):

1. In Discord: Server Settings > Integrations > Webhooks > New Webhook
2. Copy the webhook URL
3. In Grafana: Add contact point > Type: Discord
4. Paste webhook URL
5. Test and Save

Notification policy:

1. Alerting > Notification Policies
2. Default policy > Edit
3. Contact point: `email-alerts`
4. Group wait: `30s`
5. Group interval: `5m`
6. Repeat interval: `4h`
7. Save

Why this matters: `noDataState: OK` on disk and CPU alerts prevents false alarms when a container is intentionally stopped. `noDataState: Alerting` on node-down is cor-

rect — if we cannot collect data from a node, something is wrong. These are production patterns that prevent alert fatigue.

Interview angle: "I built a monitoring stack with Prometheus for metrics collection, Grafana for visualization and alerting, and Uptime Kuma for availability checks. I have five file-provisioned alert rules covering disk usage, CPU, memory, and node availability — each with tuned thresholds and noDataState settings to prevent alert fatigue. When an alert fires, I get an email with the specific metric and instance that triggered it. I tested every alert by simulating the failure condition."

Uptime Kuma

Prometheus monitors metrics. Uptime Kuma monitors availability — is this HTTP endpoint responding? Is this TCP port open?

Create the Uptime Kuma Container

```
# On node-02
pct create 202 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-uptime \
    --cores 1 \
    --memory 512 \
    --swap 256 \
    --rootfs local-lvm:4 \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.32/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 1 \
    --tags monitoring,uptime

pct start 202
pct enter 202
```

Install Uptime Kuma

```
# Install Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs git

# Clone and install Uptime Kuma
# Clone a specific stable release (check https://github.com/louislam/uptime-kuma/releases for latest)
git clone --depth 1 --branch 1.23.16 https://github.com/louislam/uptime-kuma.git /opt/uptime-kuma
cd /opt/uptime-kuma
npm run setup
```

Create Systemd Service

```
cat > /etc/systemd/system/uptime-kuma.service << 'EOF'
[Unit]
Description=Uptime Kuma Monitoring
After=network.target

[Service]
Type=simple
WorkingDirectory=/opt/uptime-kuma
ExecStart=/usr/bin/node server/server.js
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now uptime-kuma
```

Configure Monitors

Access `http://192.168.1.32:3001` (or `https://uptime.lab`). On first visit, create an admin account.

Add monitors for each service:

Monitor Type	Name	URL/Host	Interval
HTTP(s)	AdGuard Home	`http://192.168.1.20:3000`	60s
HTTP(s)	Traefik	`https://proxy.lab`	60s
HTTP(s)	Grafana	`https://grafana.lab`	60s
HTTP(s)	Prometheus	`http://192.168.1.31:9090`	60s
HTTP(s)	Syncthing	`http://192.168.1.41:8384`	60s
HTTP(s)	PVE node-01	`https://192.168.1.10:8006`	60s
HTTP(s)	PVE node-02	`https://192.168.1.11:8006`	60s
HTTP(s)	PVE node-03	`https://192.168.1.12:8006`	60s
TCP Port	PBS	`192.168.1.40:8007`	60s

For the Proxmox web UI monitors, you may need to accept invalid certificates (self-signed) in the monitor settings.

Set up notifications:

1. Settings > Notifications > Setup Notification
2. Add email or Discord webhook
3. Apply to all monitors or select specific ones

Status Page (Optional)

Uptime Kuma can create a public status page:

1. Status Pages > New Status Page
2. Add your monitors to groups (Infrastructure, Services, Monitoring)
3. Publish at a path like `/status`

This gives you a clean, single-page view of your entire lab's health.

Log Aggregation with Loki

Prometheus tells you *what* is happening — CPU is at 90%, disk is full. Loki tells you *why* — the backup script threw an error at 2:47 AM, and here's the exact log line.

Architecture

The Loki stack has three pieces, and you already have one of them:

- **Loki** — stores and indexes logs (runs in your monitoring container)
- **Promtail** — ships logs from each container to Loki (lightweight agent)
- **Grafana** — you already have it. Loki just adds a new datasource.

Install Loki

On the monitoring container (or alongside Grafana):

```
# Download Loki binary
curl -LO https://github.com/grafana/loki/releases/download/v3.0.0/loki-linux-amd64.zip
unzip loki-linux-amd64.zip
mv loki-linux-amd64 /usr/local/bin/loki
chmod +x /usr/local/bin/loki
```

Loki Configuration

Create `/etc/loki/loki-config.yml`:

```
# /etc/loki/loki-config.yml
auth_enabled: false

server:
  http_listen_port: 3100

common:
  path_prefix: /var/lib/loki
  storage:
    filesystem:
      chunks_directory: /var/lib/loki/chunks
      rules_directory: /var/lib/loki/rules
  replication_factor: 1
  ring:
    kvstore:
      store: inmemory
```

```yaml
schema_config:
  configs:
    - from: 2024-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h
```

Loki Systemd Service

```bash
mkdir -p /var/lib/loki /etc/loki

useradd --no-create-home --shell /bin/false loki
chown -R loki:loki /var/lib/loki

cat > /etc/systemd/system/loki.service << 'EOF'
[Unit]
Description=Loki Log Aggregation
After=network-online.target

[Service]
Type=simple
User=loki
ExecStart=/usr/local/bin/loki -config.file=/etc/loki/loki-config.yml
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now loki
```

Install Promtail on Each Container

Promtail is the agent that tails your log files and ships them to Loki. Install it on each container you want logs from:

```bash
curl -LO https://github.com/grafana/loki/releases/download/v3.0.0/promtail-linux-amd64.zip
unzip promtail-linux-amd64.zip
mv promtail-linux-amd64 /usr/local/bin/promtail
chmod +x /usr/local/bin/promtail
```

Promtail Configuration

Create `/etc/promtail/promtail-config.yml`:

```yaml
# /etc/promtail/promtail-config.yml
server:
  http_listen_port: 9080

positions:
  filename: /var/lib/promtail/positions.yml

clients:
  - url: http://192.168.1.30:3100/loki/api/v1/push

scrape_configs:
  - job_name: syslog
    static_configs:
      - targets:
          - localhost
        labels:
          job: syslog
          host: ${HOSTNAME}
          __path__: /var/log/syslog
```

Promtail Systemd Service

```bash
mkdir -p /var/lib/promtail /etc/promtail

cat > /etc/systemd/system/promtail.service << 'EOF'
[Unit]
Description=Promtail Log Shipper
After=network-online.target

[Service]
Type=simple
ExecStart=/usr/local/bin/promtail -config.file=/etc/promtail/promtail-config.yml
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now promtail
```

Add Loki as a Grafana Datasource

1. In Grafana, go to Configuration > Data Sources > Add data source

2. Select "Loki"
3. URL: `http://localhost:3100` (if Loki runs on the same container as Grafana, otherwise use the Loki container's IP)
4. Click "Save & Test"

Querying Logs

Now you can search all your logs from one place. In Grafana's Explore view, select the Loki datasource and try:

```
{host="ct-proxy"} |= "error"
```

This shows every log line containing "error" from your proxy container. You can filter by host, job, time range, or any label you configured in Promtail.

Combined with your Prometheus metrics, you have full observability — what happened, when, and why. Prometheus tells you a disk filled up at 2:47 AM. Loki tells you which process wrote 40 GB of logs to `/var/log` because you forgot to set up log rotation.

Putting It All Together

Verify the Complete Stack

After deploying everything in this chapter:

1. **Prometheus targets** (`http://192.168.1.31:9090/targets`): All targets should show "UP"
2. **Grafana dashboards** (`https://grafana.lab`): Dashboards should show data for all hosts
3. **Grafana alerts** (`https://grafana.lab` > Alerting > Alert rules): All rules should show "Normal" (green)
4. **Uptime Kuma** (`https://uptime.lab`): All monitors should show "Up"

Monitoring the Monitoring

There is a recursive problem: who monitors the monitoring stack? If Prometheus goes down, Grafana stops showing data. If Grafana goes down, you stop getting alerts.

Solution: Layer your monitoring.

1. Grafana alerts handle metric-based conditions (disk full, CPU high, node down)
2. Uptime Kuma handles availability checks (is Grafana itself reachable?)
3. Uptime Kuma sends its own notifications independently of Grafana
4. If you are truly paranoid, add an external uptime service (e.g., UptimeRobot free tier) monitoring your Uptime Kuma instance

For a homelab, having both Grafana and Uptime Kuma with independent notification channels covers 99% of failure scenarios.

What Good Monitoring Looks Like

After running for a week, you should have:

- Baseline CPU/memory/disk graphs showing normal patterns
- No false-positive alerts (tune thresholds if you get them)
- At least one real alert that caught something before you noticed it
- Confidence that if you walk away for a week, you will know if something breaks

If you are getting woken up by false alarms, your thresholds are too aggressive. If you miss a real outage, your monitoring has gaps. Iterate until your monitoring is useful without being noisy.

Chapter Checkpoint

You just built the eyes and ears of your lab. Before you move on, take 60 seconds to make sure they actually see and hear.

Before you move on, verify:

- ☐ Prometheus scrapes all nodes (check Targets page — all should be UP)
- ☐ Grafana dashboard shows CPU, memory, and disk for every node
- ☐ At least one alert rule fires when you simulate the condition (fill a temp disk to 90%)
- ☐ Uptime Kuma shows green for all monitored services

Next: Chapter 6 — Backups and Recovery

Chapter 6 — Backups and Recovery

Everyone has a backup strategy. Few people have a tested restore strategy. The difference matters at 3 AM when a disk dies and you're staring at a restore command you've never actually run before.

A backup you have never tested restoring is not a backup — it is a prayer. This chapter covers the full backup lifecycle: automated snapshots, application-level backups, verification, alerting, and — critically — how to recover when things go wrong.

> **Common mistake:** Setting up nightly backups and never once testing a restore. Schedule a quarterly "fire drill" — pick a container, restore it to a temporary ID, verify it works, then delete it. If you cannot restore, you do not have backups. You have a cron job that wastes disk space.

Backup Strategy Overview

Production backup strategy follows the 3-2-1 rule:

- **3** copies of your data
- **2** different storage media/locations
- **1** offsite (or at least off-node)

For a homelab, "offsite" might mean a different physical machine rather than a different geographic location. That is fine — the goal is surviving a single hardware failure.

What We Are Building

Layer	Tool	What It Backs Up	Where	Schedule
Infrastructure	PBS	Full container/VM snapshots	Storage node	Nightly
Application	Restic	App data, configs, databases	Local + remote	Nightly
Configuration	Git	Scripts, configs, state docs	GitHub/GitLab	Nightly

```
Layer 1: PBS snapshots (full container restore)
   └── Nightly at 01:00
   └── Retention: 7 daily, 4 weekly

Layer 2: Restic backups (application data)
```

```
   └── Nightly at 02:30
   └── Local repo on storage node
   └── Remote repo via SFTP to second node

Layer 3: Git push (configurations)
   └── Nightly at 03:15
   └── Remote repository (GitHub/GitLab)
```

Proxmox Backup Server (PBS)

PBS is purpose-built for backing up Proxmox VMs and containers. It provides deduplication, encryption, and fast incremental backups.

Install PBS

You have two options:

Option A: Dedicated PBS installation (recommended for production)

Install PBS on a separate machine or in a VM with dedicated storage.

Option B: PBS as an LXC container (simpler for a homelab)

Run PBS in a privileged container on your storage node. This is less isolated but perfectly functional.

```
# On node-03, create a privileged container for PBS
pct create 300 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-pbs \
    --cores 2 \
    --memory 2048 \
    --swap 512 \
    --rootfs local-lvm:8 \
    --mp0 local-lvm:200,mp=/mnt/datastore \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.40/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 0 \
    --onboot 1 \
    --tags backup,pbs,core

pct start 300
pct enter 300
```

> **Note:** PBS in an LXC container must be privileged because it needs direct filesystem access for its datastore. This is one of the valid use cases for privileged containers.

> **Warning:** If your Proxmox storage uses LVM thin pools (the default `local-lvm`), the PBS datastore mount can silently exhaust the thin pool when it grows. Monitor thin

pool usage with `lvs` and consider using a dedicated disk or thick-provisioned LV for the PBS datastore in production. Thin pool exhaustion causes I/O errors across *all* containers on that storage, not just PBS.

Install PBS inside the container:

```
# Add PBS repository
echo "deb http://download.proxmox.com/debian/pbs bookworm pbs-no-subscription" \
    > /etc/apt/sources.list.d/pbs.list

# Add Proxmox GPG key
wget -q "https://enterprise.proxmox.com/debian/proxmox-release-bookworm.gpg" \
    -O /etc/apt/trusted.gpg.d/proxmox-release-bookworm.gpg

# Install
apt update
apt install -y proxmox-backup-server

# Access the web UI at https://192.168.1.40:8007
```

Configure PBS Datastore

1. Log into the PBS web UI at `https://192.168.1.40:8007` (default user: `root@pam`)
2. Administration > Storage/Disks
3. Add Datastore:
   - Name: `lab-backups`
   - Backing Path: `/mnt/datastore`
4. Set retention (Prune settings):
   - Keep Last: 3
   - Keep Daily: 7
   - Keep Weekly: 4
   - Keep Monthly: 2

Connect Proxmox to PBS

On each Proxmox node, add PBS as a storage backend:

In the Proxmox web UI:

1. Datacenter > Storage > Add > Proxmox Backup Server
2. ID: `pbs`
3. Server: `192.168.1.40`
4. Port: `8007`
5. Username: `root@pam`
6. Password: (your PBS root password)
7. Datastore: `lab-backups`
8. Fingerprint: Copy from PBS web UI (Dashboard shows the fingerprint)
9. Content: VZDump backup file
10. Nodes: All

Automated Backup Schedule

Create a backup job in Proxmox:

Via Web UI:

1. Datacenter > Backup > Add
2. Storage: `pbs`
3. Schedule: `01:00` daily
4. Selection mode: All (or select specific containers)
5. Mode: Snapshot
6. Compression: ZSTD
7. Retention: Use PBS datastore defaults (or override here)

Via command line:

```
# Create backup job in /etc/pve/jobs.cfg
cat >> /etc/pve/jobs.cfg << 'EOF'
vzdump: nightly-backup
    enabled 1
    schedule 01:00
    storage pbs
    all 1
    mode snapshot
    compress zstd
    mailnotification failure
    mailto your-email@example.com
EOF
```

Verify PBS Backups

After the first backup runs:

1. Check the PBS web UI — Datastore > Content should list backups
2. Check backup sizes and deduplication ratios
3. Verify no errors in the task log

```
# From a Proxmox host, list backups on PBS
pvesh get /nodes/node-01/storage/pbs/content
```

Restic: Application-Level Backups

PBS snapshots capture entire containers. Restic backups up specific files and directories — application data, databases, configurations — with encryption and deduplication.

Why Both PBS and Restic?

Feature	PBS	Restic
Backs up	Entire container	Specific files/directories

Feature	PBS	Restic
Restore	Full container restore	Individual file restore
Speed	Fast (block-level dedup)	Fast (content-defined chunking)
Encryption	Optional	Always encrypted
Flexibility	Proxmox-specific	Any system, any storage backend

PBS is your "bare metal restore" — rebuild a container from scratch. Restic is your "I accidentally deleted that config file" or "restore this database to yesterday's version" tool.

Install Restic

```
# On the node that will run backups (e.g., node-01 or a dedicated container)
apt install -y restic
```

Create Restic Repositories

Create two repositories for redundancy:

Local repository (on the storage node):

```
# Create directory on storage node
ssh node-03 'mkdir -p /mnt/storage/restic-backups'

# Initialize the repository
export RESTIC_PASSWORD="your-strong-restic-password"
restic init -r sftp:root@192.168.1.12:/mnt/storage/restic-backups
```

Remote repository (on a different node):

```
# Initialize on a second node for redundancy
ssh node-02 'mkdir -p /mnt/restic-remote'
restic init -r sftp:root@192.168.1.11:/mnt/restic-remote
```

Password Management

Store the Restic password securely:

```
# Create password file (readable only by root)
echo "your-strong-restic-password" > /root/.restic-password
chmod 600 /root/.restic-password
```

Backup Script

Create a comprehensive backup script:

```
#!/bin/bash
# /opt/restic/backup-lab.sh
# Nightly application-level backup of critical data

set -euo pipefail
```

```
# Configuration
RESTIC_PASSWORD_FILE="/root/.restic-password"
REPO_LOCAL="sftp:root@192.168.1.12:/mnt/storage/restic-backups"
REPO_REMOTE="sftp:root@192.168.1.11:/mnt/restic-remote"
LOG="/var/log/restic-backup.log"
MAILTO="your-email@example.com"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG"
}

send_alert() {
    echo "$1" | mail -s "BACKUP FAILURE: $2" "$MAILTO" 2>/dev/null || true
}

backup_repo() {
    local repo="$1"
    local repo_name="$2"

    log "Backing up to $repo_name..."

    # Backup application data
    restic -r "$repo" \
        --password-file "$RESTIC_PASSWORD_FILE" \
        backup \
        --verbose \
        --tag "nightly" \
        /root/projects/ \
        /etc/traefik/ \
        /opt/AdGuardHome/AdGuardHome.yaml \
        /etc/prometheus/ \
        /etc/grafana/ \
        /var/lib/grafana/grafana.db \
        2>> "$LOG"

    if [ $? -ne 0 ]; then
        log "ERROR: Backup to $repo_name failed"
        send_alert "Restic backup to $repo_name failed. Check $LOG" "$repo_name"
        return 1
    fi

    # Prune old backups
    restic -r "$repo" \
        --password-file "$RESTIC_PASSWORD_FILE" \
        forget \
        --keep-daily 7 \
        --keep-weekly 4 \
        --keep-monthly 6 \
```

```
        --prune \
        2>> "$LOG"

    log "Backup to $repo_name complete"
}

# Main
log "=== Starting nightly backup ==="

ERRORS=0

backup_repo "$REPO_LOCAL" "LOCAL" || ((ERRORS++))
backup_repo "$REPO_REMOTE" "REMOTE" || ((ERRORS++))

if [ $ERRORS -gt 0 ]; then
    log "=== Backup completed with $ERRORS errors ==="
else
    log "=== Backup completed successfully ==="
fi
```

```
chmod +x /opt/restic/backup-lab.sh
```

Schedule the Backup

```
# Add to crontab
crontab -e

# Nightly at 02:30 (after PBS snapshots at 01:00)
30 2 * * * /opt/restic/backup-lab.sh
```

Database Backups

If you run databases (PostgreSQL, MariaDB, SQLite), dump them before the Restic backup runs:

```
#!/bin/bash
# /opt/restic/pre-backup-dumps.sh
# Run before restic backup to create consistent database dumps

DUMP_DIR="/root/backups/db-dumps"
mkdir -p "$DUMP_DIR"

# SQLite databases (copy the file — SQLite is single-file)
cp /data/app/database.db "$DUMP_DIR/app-database.db" 2>/dev/null || true

# PostgreSQL (if applicable)
# ssh ct-database 'pg_dump -U postgres mydb' > "$DUMP_DIR/mydb.sql" 2>/dev/null || true

# MariaDB (if applicable)
# ssh ct-database 'mysqldump -u root --all-databases' > "$DUMP_DIR/all-databases.sql" 2>/dev/null || tr
```

Add this script to the backup chain:

```
# In crontab:
25 2 * * * /opt/restic/pre-backup-dumps.sh
30 2 * * * /opt/restic/backup-lab.sh
```

Backup Verification

An untested backup is no backup at all. Set up automated verification.

Weekly Integrity Check

```bash
#!/bin/bash
# /opt/restic/check-repos.sh
# Weekly integrity check of all restic repositories

set -euo pipefail

RESTIC_PASSWORD_FILE="/root/.restic-password"
REPOS=(
    "sftp:root@192.168.1.12:/mnt/storage/restic-backups|LOCAL"
    "sftp:root@192.168.1.11:/mnt/restic-remote|REMOTE"
)
LOG="/var/log/restic-check.log"
MAILTO="your-email@example.com"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG"
}

log "=== Starting weekly integrity check ==="

ERRORS=0
for entry in "${REPOS[@]}"; do
    IFS='|' read -r repo name <<< "$entry"
    log "Checking $name..."

    if restic -r "$repo" --password-file "$RESTIC_PASSWORD_FILE" check 2>> "$LOG"; then
        log "$name: OK"
    else
        log "$name: FAILED"
        echo "Restic integrity check failed for $name. Check $LOG" | \
            mail -s "BACKUP INTEGRITY: $name FAILED" "$MAILTO" 2>/dev/null || true
        ((ERRORS++))
    fi
done
```

```bash
if [ $ERRORS -eq 0 ]; then
    log "=== All repositories passed integrity check ==="
else
    log "=== $ERRORS repositories FAILED integrity check ==="
fi

chmod +x /opt/restic/check-repos.sh

# Schedule weekly on Sunday at 04:00
echo "0 4 * * 0 /opt/restic/check-repos.sh" >> /var/spool/cron/crontabs/root
```

Backup Watchdog

Detect when the nightly backup did not run at all (cron failure, node was down, script crashed before logging):

```bash
#!/bin/bash
# /opt/restic/backup-watchdog.sh
# Alerts if the nightly backup didn't produce a log entry today

LOG="/var/log/restic-backup.log"
MAILTO="your-email@example.com"
TODAY=$(date '+%Y-%m-%d')

if ! grep -q "$TODAY.*Starting nightly backup" "$LOG" 2>/dev/null; then
    echo "No backup log entry found for $TODAY. The nightly backup may not have run." | \
        mail -s "BACKUP WATCHDOG: No backup detected for $TODAY" "$MAILTO" 2>/dev/null || true
fi

chmod +x /opt/restic/backup-watchdog.sh

# Run at 05:00, after backups should have completed
echo "0 5 * * * /opt/restic/backup-watchdog.sh" >> /var/spool/cron/crontabs/root
```

Email Alerting with msmtp

Many of the scripts above send email on failure. Set up a lightweight SMTP relay:

```bash
# mailutils provides the 'mail' command; msmtp replaces sendmail as the MTA
apt install -y mailutils msmtp msmtp-mta
```

Configure `/etc/msmtprc`:

```
# /etc/msmtprc
defaults
auth           on
tls            on
tls_trust_file /etc/ssl/certs/ca-certificates.crt
logfile        /var/log/msmtp.log
```

```
account        default
host           smtp.gmail.com
port           587
from           your-email@gmail.com
user           your-email@gmail.com
password       your-app-password
```

```
chmod 600 /etc/msmtprc

# Test
echo "Test email from homelab" | mail -s "Homelab Alert Test" your-email@example.com
```

> **Common mistake:** Using your main Gmail password. Gmail blocks this. You need an App Password: Google Account > Security > 2-Step Verification (enable it first) > App passwords > Generate.

Disaster Recovery

The whole point of backups is recovery. Here is how to restore from various failure scenarios.

Scenario 1: Container Corruption / Accidental Deletion

Restore from PBS:

1. In the Proxmox web UI, go to the node where you want to restore
2. Select the PBS storage in the sidebar
3. Find the backup for the container
4. Click "Restore"
5. Choose the target node and container ID
6. Restore

Via command line:

```
# List available backups
pvesh get /nodes/node-01/storage/pbs/content

# Restore container 100 from the latest backup
pct restore 100 pbs:backup/ct-100/2024-01-15T01:00:00Z --storage local-lvm
```

Scenario 2: Need a Specific File

Restore from Restic:

```
# List snapshots
restic -r sftp:root@192.168.1.12:/mnt/storage/restic-backups \
    --password-file /root/.restic-password \
    snapshots
```

```
# Browse a snapshot
restic -r sftp:root@192.168.1.12:/mnt/storage/restic-backups \
    --password-file /root/.restic-password \
    ls latest

# Restore a specific file
restic -r sftp:root@192.168.1.12:/mnt/storage/restic-backups \
    --password-file /root/.restic-password \
    restore latest \
    --target /tmp/restore \
    --include "/etc/traefik/dynamic/routes.yml"

# Restore a specific snapshot (not just latest)
restic -r sftp:root@192.168.1.12:/mnt/storage/restic-backups \
    --password-file /root/.restic-password \
    restore abc1234 \
    --target /tmp/restore
```

Scenario 3: Complete Node Failure

If a Proxmox node dies entirely:

1. **Replace the hardware** (or repurpose another machine)
2. **Install Proxmox VE** following Chapter 2
3. **Join the cluster** (`pvecm add 192.168.1.10`)
4. **Restore containers from PBS** — all containers that were on the failed node can be restored to any surviving node

If ALL nodes fail (catastrophic):

1. Install Proxmox on the first node
2. Connect to PBS storage (if it survived) and restore everything
3. If PBS is also gone, restore from Restic to a new installation
4. Configurations are in Git — clone the repo and reapply

Scenario 4: Restic Repository Recovery

If you lose the Restic password, the data is unrecoverable (encryption). Store the password securely:

```
# Print the password and store it somewhere safe (password manager, printed paper)
cat /root/.restic-password
```

Consider storing the Restic password in your password manager AND as a physical backup (printed, in a safe).

> **Interview angle:** "My backup strategy follows the 3-2-1 rule: three copies, two storage types, one off-node. I run nightly PBS snapshots for full container recovery and Restic for granular file-level restores to two separate repositories. I have automated integrity checks, a backup watchdog that alerts if the nightly job didn't run, and I test restores quarterly. I've actually restored containers from backup — not just backed them up."

Backup Schedule Summary

Time	Task	Tool	Location
01:00	Container snapshots	PBS	Storage node
02:25	Database dumps	Scripts	Local (pre-backup)
02:30	Application backup (local)	Restic	Storage node SATA
02:30	Application backup (remote)	Restic	Second node SFTP
03:15	Config git push	Git	GitHub/GitLab
04:00 (Sun)	Integrity check	Restic	All repos
05:00	Backup watchdog	Script	Email alert

Why staggered timing? Running everything at once creates I/O contention and can cause backups to fail or take much longer. Staggering by 30-60 minutes ensures each task gets full disk and network bandwidth.

Backup Verification Checklist

Run this monthly:

- ☐ PBS backups are being created nightly (check PBS web UI)
- ☐ PBS retention policy is pruning old backups (disk usage should be stable, not growing forever)
- ☐ Restic integrity checks are passing (check `/var/log/restic-check.log`)
- ☐ Test restore a container from PBS to a temporary ID
- ☐ Test restore a specific file from Restic
- ☐ Verify email alerts are working (send a test)
- ☐ Restic password is stored somewhere outside the lab
- ☐ Git push is running (check remote repository last push date)

Chapter Checkpoint

You just built a multi-layer backup system. Before you move on, take 60 seconds to make sure it does more than just run — make sure it can actually save you.

Before you move on, verify:

- ☐ PBS runs a backup job and completes without errors
- ☐ Restic backup script runs manually and succeeds
- ☐ You have tested a restore — not just a backup (restore one container to verify)
- ☐ Email alerting sends a test message to your inbox

Next: Chapter 7 — Security Hardening

Chapter 7 — Security Hardening

A homelab without security hardening is a botnet node waiting to happen. It is not a question of if your exposed SSH port gets probed — it is a question of how many times per hour.

"It is just a homelab" is the most dangerous sentence in self-hosting. Default passwords, open ports, and unpatched systems are how real breaches start. And the habits you build here are the habits you carry into your career. This chapter hardens your lab to a standard that would pass a basic security audit.

> **Common mistake:** Assuming your homelab is safe because it is behind a consumer router. NAT is not a firewall. One compromised IoT device, one forwarded port you forgot about, one UPnP rule your router auto-created — and your unpatched Proxmox node is reachable. Harden it anyway.

Security Principles

Before we touch a single configuration file, internalize these:

1. **Least privilege.** Every service, user, and process gets the minimum permissions it needs. Nothing more.
2. **Defense in depth.** Multiple layers of protection. If one fails, the next catches the threat.
3. **Assume breach.** Design as if an attacker is already on your network. Limit lateral movement.
4. **Audit and log.** If you cannot see what happened, you cannot respond to it.

These are not academic principles. They are how every serious organization approaches security. Building them into your homelab makes them second nature.

SSH Hardening

SSH is the primary management interface for your entire lab. It must be locked down.

Key-Only Authentication (Review)

You set this up in Chapter 2. Verify it is still in effect on every host and container:

```bash
# Check on each node/container
grep -E "^(PermitRootLogin|PasswordAuthentication|PubkeyAuthentication)" /etc/ssh/sshd_config
```

Expected output:

```
PermitRootLogin prohibit-password
PubkeyAuthentication yes
```

If `PasswordAuthentication` is present and set to `yes`, fix it:

```bash
sed -i 's/^PasswordAuthentication yes/PasswordAuthentication no/' /etc/ssh/sshd_config
echo "PasswordAuthentication no" >> /etc/ssh/sshd_config  # If not present at all
systemctl restart sshd
```

Additional SSH Hardening

Edit `/etc/ssh/sshd_config` on each host:

```bash
# Disable unused authentication methods
ChallengeResponseAuthentication no
KerberosAuthentication no
GSSAPIAuthentication no

# Disable X11 forwarding (not needed for server management)
X11Forwarding no

# Disable TCP forwarding unless you specifically need it
AllowTcpForwarding no

# Set a login grace period (seconds before disconnecting unauthenticated sessions)
LoginGraceTime 30

# Limit authentication attempts per connection
MaxAuthTries 3

# Disable empty passwords
PermitEmptyPasswords no

# Log level (VERBOSE gives you more audit data)
LogLevel VERBOSE

# Restrict SSH to specific users (optional — useful if you create non-root accounts)
# AllowUsers root admin

# Validate config before restarting (prevents lockout from typos)
sshd -t

# If no errors, restart
systemctl restart sshd
```

Common mistake: Editing sshd_config and restarting without testing the config first.

If you introduce a syntax error, SSH will refuse to start and you are locked out. Always run `sshd -t` before `systemctl restart sshd`. Keep a console session open as a fallback.

SSH Key Rotation

Good practice: rotate SSH keys annually. Generate new keys, distribute them, then remove old ones from `authorized_keys`.

```
# Generate new key
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_2025 -C "homelab-2025"

# Copy to all nodes
for node in node-01 node-02 node-03; do
    ssh-copy-id -i ~/.ssh/id_ed25519_2025.pub root@$node
done

# After verifying the new key works, remove the old one from authorized_keys
# on each node
```

Firewall Configuration

Proxmox Cluster Firewall

Proxmox includes a built-in firewall managed through the web UI and configuration files. It operates at three levels:

1. **Datacenter level** — Rules that apply to all nodes
2. **Node level** — Rules specific to a host
3. **Container/VM level** — Rules specific to a guest

Create IP Sets

IP Sets group addresses for use in firewall rules. Create them for management and LAN access:

In the Proxmox web UI:

1. Datacenter > Firewall > IPSet > Create

Management IPSET:

Name	CIDR	Comment
`management`		
	`192.168.1.200/32`	Workstation
	`192.168.1.10/32`	node-01
	`192.168.1.11/32`	node-02
	`192.168.1.12/32`	node-03
	`100.64.0.0/10`	Tailscale (if using)

LAN IPSET:

Name	CIDR	Comment
lan		
	192.168.1.0/24	Entire LAN

Datacenter Firewall Rules

Default policy: **DROP** for inbound. This means everything is blocked unless explicitly allowed.

```
# Datacenter > Firewall > Rules

Direction  Action   Source          Protocol  Dest Port   Comment
IN         ACCEPT   +management      tcp       8006        PVE Web UI
IN         ACCEPT   +management      tcp       22          SSH
IN         ACCEPT   +management      tcp       3128        SPICE Proxy
IN         ACCEPT   +lan             tcp       80,443      HTTP/HTTPS (services)
IN         ACCEPT   +lan             udp       5405:5412   Corosync cluster
IN         ACCEPT   +lan             tcp       111,2049    NFS (if using)
```

Enable the Firewall

1. Datacenter > Firewall > Options
2. Set "Firewall" to **Yes**
3. Set "Input Policy" to **DROP**
4. Set "Output Policy" to **ACCEPT**

   Why this matters: A default-deny firewall means any new service you deploy is blocked by default until you explicitly allow it. This prevents accidental exposure. In production, this is the only acceptable firewall posture.

Container-Level Firewall Rules

For extra security, you can add firewall rules at the container level. This restricts traffic even within your LAN.

Example: Restrict the Prometheus container to only accept connections from Grafana:

```
# Container 201 (Prometheus) > Firewall > Rules

Direction  Action   Source            Protocol  Dest Port   Comment
IN         ACCEPT   192.168.1.30/32   tcp       9090        Grafana → Prometheus
IN         DROP     (default)                                Block all else
```

This is defense in depth — even if someone compromises another container on your LAN, they cannot reach Prometheus directly.

Fail2ban

Fail2ban monitors log files and bans IPs that show malicious behavior (failed login attempts, authentication errors).

Install on Proxmox Hosts

```
apt install -y fail2ban
```

Configure Jails

Create `/etc/fail2ban/jail.local`:

```
# /etc/fail2ban/jail.local
[DEFAULT]
bantime  = 3600
findtime = 600
maxretry = 3
banaction = iptables-multiport
backend = systemd

# Email notification (optional, requires msmtp or equivalent)
# destemail = your-email@example.com
# sender = fail2ban@homelab
# action = %(action_mwl)s

[sshd]
enabled = true
port    = ssh
filter  = sshd
logpath = /var/log/auth.log
maxretry = 3

[proxmox]
enabled = true
port    = https,http,8006
filter  = proxmox
logpath = /var/log/daemon.log
maxretry = 3
```

Create Proxmox Filter

Fail2ban does not include a Proxmox filter by default. Create one:

```
cat > /etc/fail2ban/filter.d/proxmox.conf << 'EOF'
[Definition]
failregex = pvedaemon\[.*authentication failure; rhost=<HOST> user=.* msg=.*
ignoreregex =
EOF
```

Start Fail2ban

```
systemctl enable --now fail2ban

# Check status
fail2ban-client status

# Check specific jail
fail2ban-client status sshd
```

Verify Fail2ban

```
# Check banned IPs
fail2ban-client status sshd

# Manually unban an IP (if you accidentally ban yourself)
fail2ban-client set sshd unbanip 192.168.1.200
```

> **Common mistake:** Banning yourself by testing with too many failed logins. Add your workstation IP to the `ignoreip` directive in `jail.local`:
>
> ```
> [DEFAULT]
> ignoreip = 127.0.0.1/8 192.168.1.200/32
> ```
>
> **Interview angle:** "I hardened the environment with a defense-in-depth approach: SSH is key-only with password authentication disabled, the Proxmox cluster firewall runs default-deny with IP set-based rules, fail2ban monitors for brute-force attempts on both SSH and the Proxmox web UI, and unattended-upgrades handles security patches automatically across all containers. Remote access goes through a Tailscale subnet router — no ports exposed to the internet."

Unattended Security Upgrades

Every container and host should automatically install security patches.

Install on All Containers and Hosts

```
apt install -y unattended-upgrades apt-listchanges
```

Configure

Edit `/etc/apt/apt.conf.d/50unattended-upgrades`:

```
Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    "${distro_id}:${distro_codename}-updates";
};

// Remove unused dependencies after upgrade
```

```
Unattended-Upgrade::Remove-Unused-Dependencies "true";

// Auto-reboot if required (optional — consider carefully for production services)
// Unattended-Upgrade::Automatic-Reboot "true";
// Unattended-Upgrade::Automatic-Reboot-Time "04:00";

// Email notification
Unattended-Upgrade::Mail "your-email@example.com";
Unattended-Upgrade::MailReport "on-change";
```

Enable automatic updates:

```
cat > /etc/apt/apt.conf.d/20auto-upgrades << 'EOF'
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::Download-Upgradeable-Packages "1";
APT::Periodic::AutocleanInterval "7";
EOF
```

Deploy to All Containers

```
#!/bin/bash
# deploy-unattended-upgrades.sh
# Run from a Proxmox host to install on all running containers

for ctid in $(pct list | tail -n +2 | awk '{print $1}'); do
    status=$(pct status $ctid | awk '{print $2}')
    if [ "$status" = "running" ]; then
        echo "=== Installing on container $ctid ==="
        pct exec $ctid -- bash -c '
            apt install -y unattended-upgrades apt-listchanges
            cat > /etc/apt/apt.conf.d/20auto-upgrades << "APTEOF"
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::Download-Upgradeable-Packages "1";
APT::Periodic::AutocleanInterval "7";
APTEOF
        '
    fi
done
```

> **Why this matters:** Most breaches exploit known vulnerabilities with available patches. Unattended upgrades for security patches are a no-brainer. The risk of a security patch breaking something is far lower than the risk of running unpatched software.

Remote Access: Tailscale

You need secure access to your lab from outside your home network — from your phone, a coffee shop, or while traveling. Tailscale creates a mesh VPN using WireGuard that is simple to set up and rock-solid in practice.

Why Tailscale

Feature	Tailscale	Raw WireGuard	Twingate
Setup complexity	Very low	Moderate	Low
NAT traversal	Automatic	Manual	Automatic
Cost (homelab)	Free (3 users)	Free	Free (5 users)
Subnet routing	Built-in	Manual config	Agent-based
Device management	Web dashboard	Manual	Web dashboard
MagicDNS	Built-in	Not included	Not included

My recommendation: Tailscale for the primary VPN. It works. It is free. It takes 5 minutes to set up. If you want zero-trust (per-service access control), add Twingate alongside it — they serve different purposes and do not conflict.

Set Up Tailscale Subnet Router

Instead of installing Tailscale on every device, create one container that acts as a subnet router — it advertises your entire LAN to your Tailscale network.

```
# Create the Tailscale container on node-01
pct create 103 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-vpn \
    --cores 1 \
    --memory 256 \
    --swap 128 \
    --rootfs local-lvm:2 \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.23/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 0 \
    --features nesting=1,tun=1 \
    --onboot 1 \
    --tags infra,vpn

pct start 103
pct enter 103
```

> **Note:** The Tailscale container is privileged because it needs to manage network interfaces (TUN device).

Install and Configure Tailscale

```
# Install Tailscale
curl -fsSL https://tailscale.com/install.sh | sh

# Enable IP forwarding (required for subnet routing)
echo 'net.ipv4.ip_forward = 1' >> /etc/sysctl.conf
echo 'net.ipv6.conf.all.forwarding = 1' >> /etc/sysctl.conf
sysctl -p

# Start Tailscale with subnet routing
tailscale up --advertise-routes=192.168.1.0/24 --accept-dns=false
```

Tailscale will print a URL. Open it in your browser to authenticate and register the device.

Approve the Subnet Route

1. Go to admin.tailscale.com
2. Find your subnet router machine
3. Click the "…" menu > Edit route settings
4. Approve the `192.168.1.0/24` route

Configure Split DNS (Optional but Recommended)

To resolve your `.lab` domain names over Tailscale:

1. In the Tailscale admin console > DNS
2. Add a nameserver: `192.168.1.20` (your AdGuard DNS)
3. Restrict to domain: `lab`

Now, when connected via Tailscale from your phone or laptop, `grafana.lab` resolves correctly to `192.168.1.30` and routes through the subnet router.

Install Tailscale on Your Devices

Install the Tailscale app on your phone, laptop, or any device you want to access the lab from. Once connected to the same Tailscale network, you can access `https://grafana.lab` from anywhere in the world — encrypted, authenticated, no port forwarding.

Verify Remote Access

From a device on Tailscale (not on your home network):

```
# Test connectivity to a lab service
curl -k https://192.168.1.30:3000    # Grafana
ping 192.168.1.10                     # node-01
```

Certificate Management

Why Internal TLS

Even on a private LAN, TLS matters:

- Prevents credential sniffing if a device on your network is compromised
- Required for HTTP/2 and many modern web features
- Builds the habit of never running services over plain HTTP
- Browsers increasingly flag HTTP as insecure, even locally

mkcert (Already Configured)

You set up mkcert in Chapter 4. Here is a maintenance checklist:

Check certificate expiration:

```
# On the Traefik container
openssl x509 -in /etc/traefik/certs/_wildcard.lab+1.pem -noout -enddate
```

mkcert certificates are valid for ~2 years. Set a calendar reminder to regenerate before expiration.

Regenerate certificates:

```
cd /etc/traefik/certs
mkcert "*.lab" "lab"
# Traefik watches the certificate files and reloads automatically
```

Adding new SANs (Subject Alternative Names):

If you add a new domain (e.g., `*.home` in addition to `*.lab`):

```
mkcert "*.lab" "lab" "*.home" "home"
```

Install the CA on All Devices

For browsers to trust your internal TLS, the mkcert root CA must be installed:

Platform	Method
Windows	Double-click `.pem` > Install > Local Machine > Trusted Root
macOS	`sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain rootCA.pem`
Linux (Chrome)	Settings > Security > Manage Certificates > Import
Android	Settings > Security > Encryption > Install from storage
iOS	Email the CA file to yourself, tap to install, then trust in Settings > General > About > Certificate Trust Settings

---

Security Audit Checklist

Run this quarterly:

Authentication

- ☐ All SSH access is key-only (`PasswordAuthentication no`)
- ☐ Proxmox web UI has a strong password
- ☐ All service web UIs have authentication enabled
- ☐ No default passwords remain anywhere
- ☐ SSH keys are current (rotated within the last year)

Network

- ☐ Proxmox firewall is enabled with default DROP
- ☐ Only necessary ports are open
- ☐ Fail2ban is running on all Proxmox hosts
- ☐ No services are exposed to the internet unintentionally
- ☐ VPN (Tailscale) is the only remote access path

Patching

- ☐ Unattended upgrades are running on all hosts and containers
- ☐ Check `/var/log/unattended-upgrades/` for errors
- ☐ Proxmox itself is updated (`apt full-upgrade`)
- ☐ Application software is current (Grafana, Prometheus, etc.)

Access

- ☐ Review who has Tailscale access (admin console)
- ☐ Review SSH `authorized_keys` on all hosts
- ☐ Review Proxmox user accounts and API tokens
- ☐ Remove any test accounts or tokens

Data

- ☐ Backups are running (Chapter 6 checklist)
- ☐ No credentials in Git repositories
- ☐ No credentials in container configuration files
- ☐ Restic password is stored securely outside the lab

What We Did Not Cover (and Why)

Topic	Status	Reason
IDS/IPS (Suricata, Snort)	Skipped	Overkill for a homelab; monitor logs instead
SIEM (Wazuh, ELK)	Skipped	Valuable but complex; add when you need centralized log analysis
Network segmentation (VLANs)	Mentioned	Requires managed switch; add later if desired
WAF	Skipped	Not needed unless you expose services to the internet
Vulnerability scanning	Skipped	Consider `lynis` for periodic host audits

For a homelab, the measures in this chapter provide a strong security posture. They cover the most common attack vectors and build professional-grade habits. More advanced security tooling is valuable but should be added when you have the basics solidly in place — not before.

Chapter Checkpoint

You just locked down your lab to a standard that would pass a basic security audit. Before you move on, take 60 seconds to verify the locks actually hold.

Before you move on, verify:

- ☐ SSH root login requires a key (password auth is disabled)
- ☐ Firewall rules are active on the cluster (`pve-firewall status`)
- ☐ Fail2ban is running and has at least the sshd jail active
- ☐ Tailscale connects from your phone and you can reach a `.lab` service

---

Next: Chapter 8 — Automation and Maintenance

Chapter 8 — Automation and Maintenance

Here is the line between a hobby and infrastructure: can you ignore it for a week and trust it is still running? If every service requires you to SSH in, check logs, rotate files, and restart things manually — you don't have a homelab. You have a part-time job that doesn't pay.

Automation is what turns "I built a thing" into "I run a thing." This chapter covers the patterns and routines that keep your lab healthy without constant babysitting.

The Cron Schedule

By now you have several scheduled tasks scattered across nodes and containers. Let's consolidate them into a coherent schedule.

Master Schedule

Time	Node/Container	Task	Script
01:00	Proxmox (all nodes)	PBS container snapshots	PVE backup job
02:25	node-01	Database dumps (pre-backup)	`/opt/restic/pre-backup-dumps.sh`
02:30	node-01	Restic backup (local + remote)	`/opt/restic/backup-lab.sh`
02:45	node-01	Lab health report	`/opt/lab-report/lab-report.sh`
03:15	ct-git	Nightly git push	`/root/scripts/nightly-push.sh`
04:00 Sun	node-01	Restic integrity check	`/opt/restic/check-repos.sh`
05:00	node-01	Backup watchdog	`/opt/restic/backup-watchdog.sh`

Time	Node/Container	Task	Script
Auto	All	Unattended security upgrades	`apt` / `unattended-upgrades`

Document this in your homelab repo. This table should live in your `state/` directory so anyone (including future-you) can understand what runs when.

> **Why staggered times?** Everything competing for disk I/O at once causes backups to take 10x longer and occasionally fail. Give each task its own time window.

Scripting Patterns

Every maintenance script in your lab should follow the same pattern. Consistency makes debugging easy and prevents you from reinventing the wheel for each new task.

The Template

```bash
#!/bin/bash
# /path/to/script.sh
# One-line description of what this script does
#
# Schedule: When this runs (e.g., "Nightly at 02:30 via cron")
# Dependencies: What must be running/available

set -euo pipefail

# === Configuration ===
LOG="/var/log/scriptname.log"
MAILTO="your-email@example.com"
HOSTNAME=$(hostname)

# === Functions ===
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [$HOSTNAME] $1" | tee -a "$LOG"
}

send_alert() {
    local subject="$1"
    local body="$2"
    echo "$body" | mail -s "$subject" "$MAILTO" 2>/dev/null || \
        log "WARNING: Failed to send email alert"
}

cleanup() {
```

```
    # Remove temp files, release locks, etc.
    rm -f /tmp/scriptname.lock 2>/dev/null || true
}
trap cleanup EXIT

# === Lock (prevent concurrent runs) ===
LOCKFILE="/tmp/scriptname.lock"
if [ -f "$LOCKFILE" ]; then
    log "ERROR: Lock file exists. Another instance may be running."
    exit 1
fi
touch "$LOCKFILE"

# === Main ===
log "=== Starting task ==="

# ... your actual work here ...

log "=== Task complete ==="
```

Key patterns explained:

Pattern	Why
`set -euo pipefail`	Exit on error, undefined variable, or pipe failure. Prevents silent failures.
Logging with timestamps	Every line is traceable. "When did this fail?" is the first question you ask.
`trap cleanup EXIT`	Cleanup runs even if the script crashes. Prevents stale lock files.
Lock file	Prevents two copies of the same script from running simultaneously (e.g., cron fires while the previous run is still going).
Email on failure	You do not check logs proactively. The script must tell you when it fails.

Log Rotation

Scripts that log to files will eventually fill your disk. Set up log rotation:

```
cat > /etc/logrotate.d/homelab << 'EOF'
/var/log/restic-backup.log
/var/log/restic-check.log
/var/log/nightly-push.log
```

```
/var/log/lab-report.log
{
    weekly
    rotate 4
    compress
    delaycompress
    missingok
    notifempty
    create 0640 root root
}
EOF
```

Lab Health Report

A daily report that summarizes the state of your entire lab. Run it after backups complete so it can include backup status.

```bash
#!/bin/bash
# /opt/lab-report/lab-report.sh
# Daily lab health report — runs at 02:45, emails summary

set -euo pipefail

LOG="/var/log/lab-report.log"
MAILTO="your-email@example.com"
REPORT="/tmp/lab-report-$(date '+%Y-%m-%d').txt"

NODES=("192.168.1.10" "192.168.1.11" "192.168.1.12")
NODE_NAMES=("node-01" "node-02" "node-03")

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG"
}

report() {
    echo "$1" >> "$REPORT"
}

# Start fresh
> "$REPORT"
report "=== HOMELAB DAILY REPORT — $(date '+%Y-%m-%d %H:%M') ==="
report ""

# --- Node Health ---
report "## NODE HEALTH"
report ""
```

```
for i in "${!NODES[@]}"; do
    ip="${NODES[$i]}"
    name="${NODE_NAMES[$i]}"

    if ssh -o ConnectTimeout=5 "root@$ip" 'true' 2>/dev/null; then
        uptime=$(ssh "root@$ip" 'uptime -p')
        load=$(ssh "root@$ip" 'cat /proc/loadavg | cut -d" " -f1-3')
        mem=$(ssh "root@$ip" 'free -h | awk "/^Mem:/{printf \"%s / %s\", \$3, \$2}"')
        disk=$(ssh "root@$ip" 'df -h / | awk "NR==2{printf \"%s / %s (%s)\", \$3, \$2, \$5}"')

        report "$name ($ip): UP"
        report "  Uptime:  $uptime"
        report "  Load:    $load"
        report "  Memory:  $mem"
        report "  Disk:    $disk"
    else
        report "$name ($ip): DOWN / UNREACHABLE"
    fi
    report ""
done

# --- Container Status ---
report "## CONTAINER STATUS"
report ""

# Run on node-01 (cluster member)
ssh root@192.168.1.10 'pvesh get /cluster/resources --type vm --output-format text 2>/dev/null' >> "$RE
    report "  ERROR: Could not query cluster resources"
report ""

# --- Disk Usage Warnings ---
report "## DISK USAGE (>60%)"
report ""

for i in "${!NODES[@]}"; do
    ip="${NODES[$i]}"
    name="${NODE_NAMES[$i]}"

    ssh "root@$ip" "df -h | awk 'NR>1{pct=\$5; gsub(/%/,\"\",pct); if(pct+0>60) print \"  $name: \" \$6
done
report ""

# --- Recent Backup Status ---
report "## BACKUP STATUS"
report ""

if [ -f /var/log/restic-backup.log ]; then
```

```
    today=$(date '+%Y-%m-%d')
    if grep -q "$today.*Backup completed successfully" /var/log/restic-backup.log 2>/dev/null; then
        report "  Restic: OK (completed today)"
    elif grep -q "$today.*errors" /var/log/restic-backup.log 2>/dev/null; then
        report "  Restic: ERRORS (check /var/log/restic-backup.log)"
    else
        report "  Restic: NO RUN DETECTED TODAY"
    fi
else
    report "  Restic: No log file found"
fi
report ""

# --- Security ---
report "## SECURITY"
report ""

# Check fail2ban bans
if command -v fail2ban-client &>/dev/null; then
    bans=$(fail2ban-client status sshd 2>/dev/null | grep "Currently banned" | awk '{print $NF}')
    report "  Fail2ban SSH bans: ${bans:-0}"
fi

# Check for pending security updates
pending=$(apt list --upgradable 2>/dev/null | grep -c "security" || echo "0")
report "  Pending security updates (this host): $pending"
report ""

report "=== END OF REPORT ===="

# Send the report
mail -s "Homelab Daily Report — $(date '+%Y-%m-%d')" "$MAILTO" < "$REPORT" 2>/dev/null || \
    log "WARNING: Could not send report email"

# Cleanup
rm -f "$REPORT"
log "Daily report sent"
```

```
chmod +x /opt/lab-report/lab-report.sh

# Schedule at 02:45
echo "45 2 * * * /opt/lab-report/lab-report.sh" >> /var/spool/cron/crontabs/root
```

> **Interview angle:** "I automated day-two operations with staggered cron jobs: nightly backups, daily health reports, weekly container updates, and a backup watchdog that alerts me if a scheduled job didn't run. Every script follows the same pattern — logging, lock files, error handling, and email on failure. I can walk away for a week and know the infrastructure is maintaining itself."

Container Update Strategy

The Wrong Way

SSH into each container individually and run `apt upgrade`. This does not scale, is easy to forget, and gives you no record of what was updated.

The Right Way

Centralized, scripted, logged, and on a schedule.

```bash
#!/bin/bash
# /root/scripts/update-all-containers.sh
# Update all running containers from the Proxmox host
# Schedule: Weekly, Sunday at 06:00

set -euo pipefail

LOG="/var/log/container-updates.log"
MAILTO="your-email@example.com"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG"
}

log "=== Starting container updates ==="

UPDATED=0
FAILED=0
SKIPPED=0

for ctid in $(pct list | tail -n +2 | awk '{print $1}'); do
    hostname=$(pct config $ctid | grep "^hostname:" | awk '{print $2}')
    status=$(pct status $ctid | awk '{print $2}')

    if [ "$status" != "running" ]; then
        log "SKIP: $ctid ($hostname) — not running"
        ((SKIPPED++))
        continue
    fi

    log "Updating $ctid ($hostname)..."

    if pct exec $ctid -- bash -c 'apt update -qq && apt full-upgrade -y -qq && apt autoremove -y -qq' >
        log "OK: $ctid ($hostname) updated"
        ((UPDATED++))
    else
```

```
        log "FAIL: $ctid ($hostname) update failed"
        ((FAILED++))
    fi
done

SUMMARY="Updated: $UPDATED, Failed: $FAILED, Skipped: $SKIPPED"
log "=== Update complete — $SUMMARY ==="

if [ $FAILED -gt 0 ]; then
    echo "Container update failures detected. $SUMMARY. Check $LOG" | \
        mail -s "CONTAINER UPDATES: $FAILED failures" "$MAILTO" 2>/dev/null || true
fi
```

Maintenance Windows

Even in a homelab, have a concept of maintenance windows:

- **Weekly (Sunday 06:00):** Container package updates
- **Monthly (1st Sunday):** Proxmox host updates (`apt full-upgrade`)
- **Quarterly:** Review and update application software (Grafana, Prometheus versions)

The key insight: scheduled maintenance prevents emergencies. A container that is updated weekly never falls so far behind that an update breaks things.

Infrastructure as Documentation

Why State Files

Your homelab's configuration lives in many places: Proxmox UI settings, container config files, application configs, cron jobs, firewall rules. If you lose a node and need to rebuild, where do you look?

State files are human-readable documents that describe your infrastructure. They are the single source of truth for "how is my lab configured right now?"

State File Structure

```
homelab/
  state/
    cluster-state.md       # Proxmox cluster configuration
    network-state.md       # IP scheme, DNS, firewall rules
    services-state.md      # What runs where, ports, dependencies
    backup-state.md        # Backup schedule, repos, retention
    cron-state.md          # Complete cron schedule across all nodes
  configs/
    traefik/               # Copy of Traefik config
    adguard/               # AdGuard backup YAML
    prometheus/            # Prometheus config
```

```
    grafana/               # Dashboard JSON exports
  scripts/
    ...
```

Example: `cluster-state.md`

```
# Cluster State

Last updated: 2024-01-15

## Nodes

| Hostname | IP | CPU | RAM | Storage | Role |
|----------|-----|-----|-----|---------|------|
| node-01 | 192.168.1.10 | i5-8500T (6C) | 32 GB | 512 GB NVMe | Compute, PVE exporter |
| node-02 | 192.168.1.11 | i5-8500T (6C) | 32 GB | 512 GB NVMe | Compute, monitoring |
| node-03 | 192.168.1.12 | i5-9500T (6C) | 32 GB | 512 GB NVMe + 2TB SATA | Storage, backups |

## Containers

| CTID | Hostname | Node | IP | Service | Cores | RAM | Disk | Tags |
|------|----------|------|----|---------|-------|-----|------|------|
| 100 | ct-dns | node-01 | .20 | AdGuard Home | 1 | 512 MB | 4 GB | infra,dns,core |
| 101 | ct-proxy | node-01 | .21 | Traefik | 2 | 512 MB | 4 GB | infra,proxy,core |
| 103 | ct-vpn | node-01 | .23 | Tailscale | 1 | 256 MB | 2 GB | infra,vpn |
| 200 | ct-grafana | node-02 | .30 | Grafana | 2 | 1 GB | 8 GB | monitoring,grafana |
| 201 | ct-prometheus | node-02 | .31 | Prometheus | 2 | 2 GB | 8+30 GB | monitoring,prometheus,core |
| 202 | ct-uptime | node-02 | .32 | Uptime Kuma | 1 | 512 MB | 4 GB | monitoring,uptime |
| 300 | ct-pbs | node-03 | .40 | PBS | 2 | 2 GB | 8+200 GB | backup,pbs,core |
| 301 | ct-sync | node-03 | .41 | Syncthing | 2 | 1 GB | 4+50 GB | infra,sync |
| 302 | ct-git | node-03 | .42 | Git/Scripts | 1 | 512 MB | 4 GB | infra,git |

## Cluster Configuration

- Cluster name: homelab-cluster
- Transport: knet
- Corosync: ports 5405-5412
- Firewall: enabled, default DROP inbound
- Storage: local-lvm (per node), pbs (backup)
```

Keeping State Files Current

The state files are only useful if they are accurate. Update them:

- **Immediately** when you add or remove a container
- **Immediately** when you change an IP address or port
- **Weekly** during your maintenance window (review and verify)
- **After every significant change** to any service

Commit changes to git. The nightly push ensures they are backed up remotely.

> **Why this matters:** State files are your disaster recovery playbook. If node-02 dies, you look at `cluster-state.md` to see exactly what was running there, then rebuild it from the state file and PBS backups. Without this, you are reconstructing your infrastructure from memory.

Day 2 Operations

"Day 1" is building the lab. "Day 2" is everything after. Here is what to check and when.

Daily (Automated)

These should run without your intervention:

- ☐ PBS snapshots complete (check email if configured for failure notification)
- ☐ Restic backups complete (check email)
- ☐ Lab report delivered (check email)
- ☐ Unattended upgrades applied (check `/var/log/unattended-upgrades/`)

If you are not getting daily report emails, something is already broken.

Weekly (5 Minutes)

- ☐ Glance at the Grafana overview dashboard — any anomalies?
- ☐ Check Uptime Kuma — all services green?
- ☐ Review disk usage trends — anything growing unexpectedly?
- ☐ Run container updates (or verify the automated script ran)
- ☐ Check fail2ban status — any unusual ban activity?

Monthly (30 Minutes)

- ☐ Update Proxmox hosts: `apt update && apt full-upgrade -y`
- ☐ Review and update state files
- ☐ Check Prometheus retention — data store growing as expected?
- ☐ Review Grafana alerts — any that keep firing? Tune or fix.
- ☐ Test restore one container from PBS
- ☐ Test restore one file from Restic
- ☐ Review Tailscale device list — remove any old devices

Quarterly (1-2 Hours)

- ☐ Full security audit (Chapter 7 checklist)
- ☐ Update application software to latest stable versions
- ☐ Review and rotate SSH keys
- ☐ Review backup retention — adjust if disk is filling or retention is too short
- ☐ Check TLS certificate expiration
- ☐ Review DNS rewrites — remove stale entries
- ☐ Review firewall rules — remove anything no longer needed

☐ Update this guide's planning documents if your lab has changed significantly

---

Automation Maturity Model

Where you are in your automation journey:

Level	Description	You Are Here If…
0: Manual	SSH in, run commands by hand	You have to remember everything
1: Scripted	Bash scripts for common tasks	Tasks are reproducible but manual to trigger
2: Scheduled	Cron jobs run scripts automatically	You get email when things fail
3: Monitored	Grafana + alerts verify automation ran	You know if cron missed a run
4: Self-healing	Scripts detect and fix common issues	Disk full? Log rotation triggers. Service down? Auto-restart.

After following this guide, you should be solidly at **Level 3**. Level 4 is achievable with experience — add self-healing scripts for common failure modes as you encounter them.

The goal is not zero-touch. The goal is that your lab requires 5 minutes of attention per day, not 5 hours. And when something does need manual intervention, you know about it immediately and have the tools to fix it quickly.

Configuration Management with Ansible

Scripts are great for one machine. But when you have 3 nodes and 10 containers, running the same script on each one by hand stops being automation and starts being a job.

Ansible solves this. It is agentless — it connects over SSH, which you already have. It is declarative — you describe the desired state, not the steps to get there. And it is idempotent — run the same playbook 10 times and you get the same result. No "oops, I installed that twice" problems.

A Minimal Playbook

Here is a playbook that applies a security baseline to every container in your lab. This does in one command what would otherwise take you 10 SSH sessions:

```yaml
# playbooks/security-baseline.yml
---
- name: Apply security baseline to all containers
  hosts: containers
  become: yes
  tasks:
    - name: Install unattended-upgrades
```

```yaml
    apt:
      name: unattended-upgrades
      state: present

  - name: Enable automatic security updates
    copy:
      dest: /etc/apt/apt.conf.d/20auto-upgrades
      content: |
        APT::Periodic::Update-Package-Lists "1";
        APT::Periodic::Unattended-Upgrade "1";

  - name: Ensure fail2ban is installed
    apt:
      name: fail2ban
      state: present

  - name: Ensure fail2ban is running
    service:
      name: fail2ban
      state: started
      enabled: yes
```

The Inventory File

Ansible needs to know what machines to target. That is the inventory file — a list of hosts organized into groups:

```ini
# inventory/homelab.ini
[nodes]
node-01 ansible_host=192.168.1.10
node-02 ansible_host=192.168.1.11
node-03 ansible_host=192.168.1.12

[containers]
ct-dns    ansible_host=192.168.1.20
ct-proxy  ansible_host=192.168.1.21
ct-mon    ansible_host=192.168.1.30
ct-sync   ansible_host=192.168.1.41
ct-kuma   ansible_host=192.168.1.32
```

Running It

```
ansible-playbook -i inventory/homelab.ini playbooks/security-baseline.yml
```

One command. Every container gets unattended-upgrades and fail2ban. If any of them already have it installed, Ansible skips that task. If one is missing it, Ansible installs it. That is idempotence in action.

This is the starting point, not the destination. Once you have Ansible working, every "I need to do X on all containers" becomes a 3-line task in a playbook instead of an SSH loop. Your future

self will thank you.

> **Common mistake:** Don't try to Ansible everything on day one. Start with one playbook that does one thing. Get it working. Then add more. The teams that fail with Ansible are the ones that try to automate their entire infrastructure in a weekend.

Chapter Checkpoint

You just turned a collection of services into self-maintaining infrastructure. Before you move on, take 60 seconds to verify it can actually run without you.

Before you move on, verify:

- ☐ At least one cron job is scheduled and has run successfully
- ☐ Your nightly git push script commits and pushes without manual intervention
- ☐ Lab report script generates output and you've read it once
- ☐ Unattended-upgrades is installed on all containers

Next: Chapter 9 — Advanced Topics

Chapter 9 — Advanced Topics

You have a production-grade homelab. This chapter covers what comes next: running AI workloads, deploying web applications, comparing VPN options, scaling your cluster, and — perhaps most importantly — turning your homelab into a career asset.

AI Inference Server with Ollama

Running local AI models is one of the most compelling homelab use cases. Ollama makes it straightforward to run open-source language models (Llama, Mistral, Qwen, Gemma) on your own hardware.

Hardware Requirements

Workload	Minimum	Recommended	Notes
Small models (1-3B params)	4 GB RAM, any CPU	8 GB RAM, modern CPU	Runs on mini PCs
Medium models (7-13B params)	16 GB RAM, 8+ cores	32 GB RAM, GPU with 8+ GB VRAM	Usable without GPU, slow
Large models (30-70B params)	32 GB RAM, GPU with 24+ GB VRAM	64 GB RAM, GPU with 48+ GB VRAM	GPU essentially required

CPU vs GPU inference:

Factor	CPU	GPU (NVIDIA)
Speed (tokens/sec)	5-15 (7B model)	30-100+ (7B model)
Cost	Free (use existing hardware)	$200-$2000+ (used GPUs)
Power draw	Low-moderate	High (200-400W under load)
Model size limit	Limited by RAM	Limited by VRAM
Setup complexity	Simple	Requires driver setup, GPU passthrough

CPU Inference (LXC Container)

For smaller models or when speed is not critical, CPU inference runs fine in a container:

```
# Create the AI container with generous resources
pct create 400 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-ai-cpu \
    --cores 8 \
    --memory 16384 \
    --swap 4096 \
    --rootfs local-lvm:20 \
    --mp0 local-lvm:50,mp=/root/.ollama \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.50/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 0 \
    --tags app,ai

pct start 400
pct enter 400
```

Note: `--onboot 0` because AI inference is not a core service. Start it when you need it.

Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh

# Start Ollama
systemctl enable --now ollama
```

Configure Ollama for Network Access

By default, Ollama only listens on `127.0.0.1:11434`. To access it from other devices:

```
# Create override directory
mkdir -p /etc/systemd/system/ollama.service.d

cat > /etc/systemd/system/ollama.service.d/override.conf << 'EOF'
[Service]
Environment="OLLAMA_HOST=0.0.0.0:11434"
EOF

systemctl daemon-reload
systemctl restart ollama
```

Pull and Run Models

```
# Pull a model
ollama pull llama3.2:3b      # Small, fast, good for testing
ollama pull qwen2.5:7b        # Medium, good quality
ollama pull mistral:7b         # Medium, popular general-purpose
```

```
# Interactive chat
ollama run llama3.2:3b

# API access (from any device on the network)
curl http://192.168.1.50:11434/api/generate -d '{
  "model": "llama3.2:3b",
  "prompt": "Explain Proxmox in one sentence.",
  "stream": false
}'
```

GPU Inference (VM with Passthrough)

For GPU inference, you need a VM (not a container) with GPU passthrough:

1. **Enable IOMMU** on the Proxmox host:

```
# Edit GRUB config
nano /etc/default/grub

# For Intel:
GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=on iommu=pt"

# For AMD:
GRUB_CMDLINE_LINUX_DEFAULT="quiet amd_iommu=on iommu=pt"

# Update GRUB and reboot
update-grub
reboot
```

2. **Blacklist host GPU drivers** (so the host doesn't claim the GPU):

```
cat >> /etc/modprobe.d/blacklist.conf << 'EOF'
blacklist nouveau
blacklist nvidia
blacklist nvidiafb
EOF

update-initramfs -u
reboot
```

3. **Create a VM and pass through the GPU:**

In the Proxmox web UI: - Create a new VM with your preferred OS (Ubuntu 22.04 recommended for NVIDIA drivers) - VM > Hardware > Add > PCI Device - Select your NVIDIA GPU - Enable "All Functions" and "PCI Express"

4. **Install NVIDIA drivers and Ollama inside the VM:**

```
# Inside the VM
sudo apt install -y nvidia-driver-545
sudo reboot
```

```bash
# Verify GPU
nvidia-smi

# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh
```

Model Storage

Models are large (3-40 GB each). Use the separate mount point (`/root/.ollama`) to avoid filling the root filesystem. If you have a SATA drive or NAS, mount it there:

```bash
# Store models on a larger drive
# Symlink or mount to /root/.ollama/models
```

Ollama Failover (Optional)

If you have both CPU and GPU nodes, set up a simple failover:

```bash
#!/bin/bash
# ollama-query.sh — try GPU first, fall back to CPU

GPU_HOST="192.168.1.60:11434"
CPU_HOST="192.168.1.50:11434"

# Health check GPU
if curl -sf "http://$GPU_HOST/api/tags" > /dev/null 2>&1; then
    HOST="$GPU_HOST"
else
    HOST="$CPU_HOST"
fi

curl "http://$HOST/api/generate" -d "{
  \"model\": \"$1\",
  \"prompt\": \"$2\",
  \"stream\": false
}"
```

Running Web Applications

Your homelab is a great platform for hosting web applications — personal projects, internal tools, or portfolio pieces.

Node.js Application with PM2

PM2 is a process manager for Node.js that handles restarts, logging, and clustering.

```bash
# Create an app container
pct create 102 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-app-01 \
    --cores 2 \
    --memory 1024 \
    --swap 512 \
    --rootfs local-lvm:8 \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.22/24,gw=192.168.1.1 \
    --nameserver "192.168.1.20" \
    --searchdomain lab \
    --ssh-public-keys /root/.ssh/authorized_keys \
    --unprivileged 1 \
    --features nesting=1 \
    --onboot 1 \
    --tags app,web

pct start 102
pct enter 102

# Install Node.js
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# Install PM2 globally
npm install -g pm2

# Deploy your application
cd /opt
git clone https://github.com/your-username/your-app.git
cd your-app
npm install

# Start with PM2
pm2 start server.js --name "my-app"

# Save PM2 process list (survives reboot)
pm2 save

# Generate startup script
pm2 startup systemd
# Follow the instructions PM2 prints
```

Add to Reverse Proxy

Add a route in `/etc/traefik/dynamic/routes.yml` on the Traefik container:

```yaml
    my-app:
      rule: "Host(`app.lab`)"
```

```yaml
      service: my-app
      entryPoints:
        - websecure
      tls: {}

    # In services:
    my-app:
      loadBalancer:
        servers:
          - url: "http://192.168.1.22:3000"
```

Add DNS rewrite in AdGuard: `app.lab → 192.168.1.21` (proxy IP).

Python Application with Gunicorn

```bash
# Inside an app container
apt install -y python3 python3-pip python3-venv

# Create a virtual environment
python3 -m venv /opt/myapp/venv
source /opt/myapp/venv/bin/activate
pip install gunicorn flask  # or django, fastapi, etc.

# Run with Gunicorn
gunicorn --workers 2 --bind 0.0.0.0:8000 app:app
```

Create a systemd service for persistence:

```bash
cat > /etc/systemd/system/myapp.service << 'EOF'
[Unit]
Description=My Python Application
After=network.target

[Service]
Type=simple
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/venv/bin/gunicorn --workers 2 --bind 0.0.0.0:8000 app:app
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl enable --now myapp
```

Database Containers

For applications that need a database:

```
# PostgreSQL container
pct create 110 local:vztmpl/debian-12-standard_12.7-1_amd64.tar.zst \
    --hostname ct-postgres \
    --cores 2 \
    --memory 2048 \
    --rootfs local-lvm:4 \
    --mp0 local-lvm:20,mp=/var/lib/postgresql \
    --net0 name=eth0,bridge=vmbr0,ip=192.168.1.25/24,gw=192.168.1.1 \
    --unprivileged 1 \
    --onboot 1 \
    --tags app,database

pct start 110
pct enter 110

# Install PostgreSQL
apt install -y postgresql postgresql-client

# Configure for network access
echo "listen_addresses = '*'" >> /etc/postgresql/15/main/postgresql.conf
echo "host all all 192.168.1.0/24 scram-sha-256" >> /etc/postgresql/15/main/pg_hba.conf

systemctl restart postgresql

# Create a database and user
sudo -u postgres psql -c "CREATE USER appuser WITH PASSWORD 'your-password';"
sudo -u postgres psql -c "CREATE DATABASE myapp OWNER appuser;"
```

> **Common mistake:** Running the database in the same container as the application. Separate them. This allows independent backups, scaling, and maintenance. If the app container needs a restart, the database keeps running.

VPN Options Comparison

You set up Tailscale in Chapter 7. Here is a deeper comparison for informed decisions.

Detailed Comparison

Feature	Tailscale	WireGuard	Twingate
Architecture	Mesh VPN, coordination server	Point-to-point VPN	Zero-trust, agent-based
Setup time	5 minutes	30-60 minutes	15 minutes
NAT traversal	Automatic (DERP relays)	Manual (port forward)	Automatic
Performance	Near-native WireGuard	Native WireGuard	Slight overhead (relay)

Feature	Tailscale	WireGuard	Twingate
Free tier	3 users, 100 devices	Unlimited (self-hosted)	5 users, unlimited devices
Access control	ACLs in admin console	Firewall rules	Per-resource policies
Split tunneling	Configurable	Configurable	Default (per-resource)
Self-hosted option	Headscale (community)	Native	No
Best for	Full network access	Maximum control	Granular per-service access

When to Use Each

Tailscale: You want VPN access to your entire lab with minimal setup. You are the only user, or you trust all users equally. This is the right choice for most homelabs.

WireGuard: You want full control, no external dependencies, or your use case requires a self-hosted VPN. You are comfortable managing keys and configurations manually.

Twingate: You want to give someone access to a specific service (e.g., Grafana) without exposing your entire network. Great for shared access or guest access.

Using multiple: Tailscale for your primary full-network access, Twingate for selective guest access. They coexist without conflict since they work at different layers.

WireGuard Quick Setup

If you want to try WireGuard as an alternative or supplement:

```
# On a container or host
apt install -y wireguard

# Generate keys
wg genkey | tee /etc/wireguard/server-private.key | wg pubkey > /etc/wireguard/server-public.key
chmod 600 /etc/wireguard/server-private.key
```

Create `/etc/wireguard/wg0.conf`:

```
[Interface]
Address = 10.0.0.1/24
ListenPort = 51820
PrivateKey = <server-private-key>
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE

[Peer]
# Your phone/laptop
PublicKey = <client-public-key>
AllowedIPs = 10.0.0.2/32
```

```
wg-quick up wg0
systemctl enable wg-quick@wg0
```

Client configuration — on your phone or laptop, create a config file (or import via QR code):

```
# Generate client keys (on the server or any machine)
wg genkey | tee client-private.key | wg pubkey > client-public.key
```

Save this as `homelab.conf` and import it into the WireGuard app:

```
[Interface]
Address = 10.0.0.2/32
PrivateKey = <client-private-key>
DNS = 192.168.1.20

[Peer]
PublicKey = <server-public-key>
Endpoint = <your-public-ip>:51820
AllowedIPs = 192.168.1.0/24, 10.0.0.0/24
PersistentKeepalive = 25
```

You will need to forward UDP port 51820 on your router to the WireGuard host. This is the main operational difference from Tailscale — Tailscale handles NAT traversal automatically, WireGuard requires a port forward.

Scaling Your Cluster

Starting with Two Nodes

The guide targets a 3-node cluster, but not everyone has three machines on day one. Two nodes will get you far — just understand the trade-off you're making.

The quorum problem. Proxmox uses Corosync for cluster communication, and Corosync requires a majority of nodes (a quorum) to agree before allowing changes. With 3 nodes, you can lose 1 and still have a majority (2 of 3). With 2 nodes, losing 1 means you have 1 of 2 — that's not a majority, so the cluster locks up. No starting containers, no stopping them, no migrations. The web UI will show "cluster not ready" and refuse to cooperate.

The manual workaround. If one of your two nodes goes down, you can force the surviving node to operate alone:

```
pvecm expected 1
```

This tells Corosync "I know I'm alone, let me work anyway." It works, but it's a manual step you have to remember at 2am when something breaks. Not ideal.

The better option: QDevice. A Corosync QDevice acts as a lightweight tie-breaker vote. It doesn't run any VMs or containers — it just participates in quorum decisions. You can run it on almost anything: a Raspberry Pi, an old laptop, even a small VM on a different machine outside the cluster.

```
# On an external Debian/Ubuntu machine (NOT a cluster member)
apt install corosync-qnetd
```

```
# On one of your Proxmox nodes
pvecm qdevice setup <qdevice-ip>
```

This gives you 2 nodes + 1 QDevice = 3 votes, which means you can lose a node without manual intervention. See the Proxmox QDevice documentation for the full setup procedure.

The practical recommendation: Start with 2 nodes if budget is tight, but plan to add a third. In the meantime, either set up a QDevice or at minimum bookmark the `pvecm expected 1` command so you're not scrambling when a node goes offline.

Running a single node? Everything in this guide works on one machine. Skip the cluster setup in Chapter 2 and run containers directly on your single host. You lose live migration and HA, but you keep monitoring, backups, security hardening, and all the operational practices. When you add a second machine later, come back to Chapter 2 and form the cluster.

Adding a New Node

When you outgrow three nodes:

1. **Install Proxmox** on the new hardware (Chapter 2)
2. **Join the cluster:**

```
# On the new node
pvecm add 192.168.1.10
```

3. **Update your documentation:**
   - IP plan
   - Container plan (assign CTID range for the new node)
   - Monitoring (add node_exporter targets in Prometheus)
   - DNS (add DNS rewrite for the new node)
   - Firewall (add to management IPSET if needed)
   - Backup (add to PBS backup job)
4. **Migrate workloads** if desired:

```
# Stop and migrate a container to the new node
pct shutdown 202
pct migrate 202 node-04
pct start 202
```

Migrating Containers

Offline migration (no shared storage required):

```
# Takes ~1-5 minutes depending on container size
pct shutdown 100
pct migrate 100 node-02
pct start 100
```

Online migration (requires shared storage — Ceph, NFS, etc.):

```
pct migrate 100 node-02 --online
```

When to Scale vs When to Optimize

Before adding a node, check if you actually need it:

```
# Check resource usage across the cluster
pvesh get /cluster/resources --type node
```

Metric	Add a Node	Optimize First
CPU consistently >80%	Yes	Review if any containers are over-allocated
RAM consistently >85%	Yes	Check for memory leaks, reduce swap usage
Disk full	Maybe	Add a disk to existing node, review retention policies
Want isolation	Yes	Separate workloads by security boundary
More availability	Yes	Third node enables HA with quorum

Storage Deep Dive: ZFS vs Ceph vs LVM-Thin

At some point, someone on r/homelab will tell you that you need Ceph. Let's talk about when they're right and when they're not.

LVM-Thin (what you have now)

This is what Proxmox gives you out of the box. Simple, fast, and good enough for 1-3 nodes.

Thin provisioning means you can overcommit storage — a container with a 20 GB disk only uses the space it actually writes to. This is how you run 10 containers on a 256 GB NVMe without sweating.

The downside is straightforward: no redundancy across nodes. If a disk dies, that data is gone unless you have backups. This is why Chapter 6 exists.

ZFS

ZFS is the storage nerd's dream. Every block gets a checksum, which means it catches silent data corruption — the kind of bit rot that slowly destroys your backups without you knowing. Snapshots are essentially free. Compression is built-in. And `zfs send/receive` gives you replication to another machine with one command.

The downsides are real. ZFS is RAM-hungry — budget roughly 1 GB of RAM per TB of storage for the ARC (adaptive replacement cache). You also cannot easily expand a pool by tossing in one more disk. ZFS wants you to plan your pool layout upfront.

Best for: NAS nodes, backup storage, anywhere data integrity matters most.

Ceph

Ceph is distributed storage across multiple nodes. Your data is replicated automatically — lose a node, and your data survives because copies live elsewhere. This is what makes live migration work without NFS: containers can move between nodes because the storage layer spans all of them.

The downsides are significant. Ceph is complex to set up and maintain. It needs a dedicated network (10GbE recommended, or at least a separate VLAN on your existing network). Three

nodes is the absolute minimum. And it will eat your RAM and CPU doing replication and recovery in the background.

Best for: 3+ node clusters where you want shared storage and live migration without a separate NFS server.

Decision table:

	LVM-Thin	ZFS	Ceph
Nodes needed	1+	1+	3+ (minimum)
Redundancy	None (use backups)	Mirror/RAIDZ (single node)	Across nodes
RAM overhead	Low	High (1GB/TB)	High
Complexity	Low	Medium	High
Best for	Getting started, small labs	NAS, backup storage	Multi-node shared storage
Live migration	Needs shared storage	Local only	Built-in

The recommendation: For a 3-node homelab? Start with LVM-Thin. Add ZFS on your storage node for backups. Consider Ceph only if you've outgrown this guide and want to simulate enterprise distributed storage. Do not add complexity before you need it.

Lab as a Resume Builder

This section might be the most valuable in the entire guide if you are building a career in IT.

What Employers Actually Look For

When a hiring manager sees "homelab" on a resume, they are not impressed by the hardware. They want to know:

1. **Can you design and maintain infrastructure?** — Your cluster architecture answers this.
2. **Do you understand monitoring and observability?** — Your Grafana + Prometheus stack answers this.
3. **Do you handle backups and disaster recovery?** — Your PBS + Restic + verification scripts answer this.
4. **Can you automate operational tasks?** — Your cron jobs, scripts, and maintenance routines answer this.
5. **Do you think about security?** — Your firewall, fail2ban, SSH hardening, and VPN answer this.

How to Talk About It in Interviews

Bad: "I have a homelab with Proxmox."

Good: "I designed and maintain a 3-node Proxmox cluster running 15 LXC containers. I implemented automated backups with Restic and PBS, built a Prometheus/Grafana monitoring stack

with alerting for disk, CPU, and availability, and hardened the environment with fail2ban, firewall rules, and key-only SSH. I treat it like production infrastructure — everything is monitored, backed up, and documented."

Resume Section Example

```
HOMELAB INFRASTRUCTURE

- Designed and operate a 3-node Proxmox VE cluster (96 GB RAM, 15+ LXC containers)
- Built monitoring pipeline: Prometheus metrics collection, Grafana dashboards,
  5 alert rules (disk, CPU, memory, node availability), Uptime Kuma health checks
- Implemented automated backup strategy: nightly PBS snapshots (7d/4w retention),
  Restic encrypted backups to multiple repos, weekly integrity verification
- Hardened security: SSH key-only auth, PVE cluster firewall (default-deny),
  fail2ban, unattended security upgrades, Tailscale VPN for remote access
- Automated operations: nightly backups, daily health reports, centralized
  container updates, infrastructure-as-docs with git-versioned state files
```

Specific Technologies to Highlight

Map your homelab skills to job postings:

Homelab Skill	Maps To Job Requirement
Proxmox (KVM/LXC)	Virtualization, containerization
Prometheus/Grafana	Monitoring, observability, SRE
Bash scripting + cron	Automation, CI/CD mindset
Firewall rules + fail2ban	Network security, hardening
Restic + PBS	Backup and disaster recovery
Traefik reverse proxy	Load balancing, service mesh concepts
Tailscale/WireGuard	VPN, zero-trust networking
Git for configs	Infrastructure as Code (IaC)
DNS management	Networking fundamentals

Blog About It

Write about what you build. A blog post explaining how you set up your monitoring stack demonstrates more than a certification ever will. It shows you can:

- Implement something end-to-end
- Document your work
- Explain technical concepts
- Think about trade-offs and design decisions

Employers Google candidates. A technical blog with homelab content is a significant differentiator.

Open Source Your Configs

Consider making your homelab documentation public (scrubbed of passwords and personal details). A GitHub repo with your Prometheus config, Grafana dashboards, backup scripts, and state files

is a living portfolio piece.

Troubleshooting

Quorum Loss

Symptoms: The Proxmox web UI shows "cluster not ready" or "no quorum." You can't start, stop, or migrate VMs or containers. Commands like `pct start` hang or return errors about quorum.

What happened: More than half your cluster nodes are offline. Corosync requires a majority of votes to function — if you have 3 nodes and 2 go down, the surviving node can't form a quorum by itself.

Immediate fix:

```
# On a surviving node — force single-node quorum
pvecm expected 1
```

This overrides the quorum requirement and lets the surviving node operate. You should regain access to the web UI and be able to manage containers immediately.

This is a temporary override, not a permanent fix. It tells Corosync "I'm the only one, let me work." If you leave it like this and bring the other nodes back, they might have a different view of the cluster state, which can cause split-brain issues.

Recovery:

1. Bring the failed nodes back online
2. Wait for them to rejoin the cluster (usually automatic)
3. Verify cluster health:

```
pvecm status
# Look for "Quorate: Yes" and all nodes listed
```

4. If a node doesn't rejoin automatically, check network connectivity and Corosync:

```
systemctl status corosync
journalctl -u corosync --since "1 hour ago"
```

Prevention:

- Run 3+ nodes so you can survive a single node failure without intervention
- If you only have 2 nodes, set up a QDevice (see "Starting with Two Nodes" above)
- At minimum, document this procedure somewhere you can find it at 2am — a sticky note on the rack, a bookmark in your browser, whatever works
- Consider setting up a Grafana alert for node availability (covered in Chapter 5) so you know immediately when a node drops, not when you try to use the cluster and it's locked

---

What to Build Next

With the foundation from this guide, here are natural next steps:

Project	What You Learn	Difficulty
Ceph storage cluster	Distributed storage, replication, failure modes	Hard
Ansible automation	Configuration management, idempotent deployments	Medium
Terraform for Proxmox	Infrastructure as Code, declarative provisioning	Medium
CI/CD pipeline	Jenkins/Gitea Actions, automated testing and deployment	Medium
Kubernetes (k3s)	Container orchestration, pods, services, ingress	Hard
Log aggregation (Loki)	Centralized logging, log queries, correlation	Medium
SIEM (Wazuh)	Security monitoring, threat detection, compliance	Hard
Mail server	SMTP, DKIM, SPF, DMARC (educational but painful to maintain)	Hard

Each of these builds on the platform you have already created. The Proxmox cluster, monitoring, and backup infrastructure you built in this guide supports all of them.

Next: Chapter 10 — Appendix

Chapter 10 — Appendix

Reference materials, templates, and checklists. Print these, bookmark them, or keep them in your homelab repo.

A. IP Planning Worksheet

Copy this template and fill it in for your network.

Infrastructure

IP Address	Hostname	Device/Service	MAC Address	Notes
192.168.1.1	gateway	Router		Default gateway, DHCP
192.168.1.2	switch	Network switch		Management (if managed)
192.168.1.10	node-01	Proxmox host		
192.168.1.11	node-02	Proxmox host		
192.168.1.12	node-03	Proxmox host + storage		
192.168.1.13		(reserved for node-04)		
192.168.1.14		(reserved for node-05)		

Containers — node-01 (.20-.29)

IP Address	CTID	Hostname	Service	Port(s)
192.168.1.20	100	ct-dns	AdGuard Home	53, 3000
192.168.1.21	101	ct-proxy	Traefik	80, 443
192.168.1.22	102	ct-app-01	Application	3000
192.168.1.23	103	ct-vpn	Tailscale	
192.168.1.24		(available)		
192.168.1.25	110	ct-postgres	PostgreSQL	5432
192.168.1.26		(available)		
192.168.1.27		(available)		
192.168.1.28		(available)		
192.168.1.29		(available)		

Containers — node-02 (.30-.39)

IP Address	CTID	Hostname	Service	Port(s)
192.168.1.30	200	ct-grafana	Grafana	3000
192.168.1.31	201	ct-prometheus	Prometheus	9090
192.168.1.32	202	ct-uptime	Uptime Kuma	3001
192.168.1.33		(available)		
192.168.1.34		(available)		
192.168.1.35		(available)		
192.168.1.36		(available)		
192.168.1.37		(available)		
192.168.1.38		(available)		
192.168.1.39		(available)		

Containers — node-03 (.40-.49)

IP Address	CTID	Hostname	Service	Port(s)
192.168.1.40	300	ct-pbs	Proxmox Backup Server	8007
192.168.1.41	301	ct-sync	Syncthing	8384, 22000
192.168.1.42	302	ct-git	Git / Scripts	
192.168.1.43		(available)		
192.168.1.44		(available)		
192.168.1.45		(available)		
192.168.1.46		(available)		
192.168.1.47		(available)		
192.168.1.48		(available)		
192.168.1.49		(available)		

AI / Specialized (.50-.59)

IP Address	CTID	Hostname	Service	Port(s)
192.168.1.50	400	ct-ai-cpu	Ollama (CPU)	11434
192.168.1.51		(available)		

Client Devices (.200-.254)

IP Address	Device	MAC Address	Notes
192.168.1.200	Workstation		Primary management station
192.168.1.201	Laptop		
192.168.1.202	Phone		
192.168.1.250-254	(reserved)		Temporary / testing

B. Container Inventory Template

Comprehensive inventory for tracking all containers.

CTID	Hostname	Node	IP	Service	Version	Cores	RAM	Root Disk	Data Disk	Priv?	Onboot	Tags	Depends On	Notes
100	ct-dns	node-01	.20	AdGuard	0.107	1	512 MB	4 GB	-	No	Yes	infra,dns,core	(none, external DNS)	Boot order 1
101	ct-proxy	node-01	.21	Traefik	3.2	2	512 MB	4 GB	-	No	Yes	infra,proxy,core	ct-dns	Boot order 2
102	ct-app-01	node-01	.22	App	-	2	1 GB	8 GB	-	No	Yes	app,web	ct-dns, ct-proxy	
103	ct-vpn	node-01	.23	Tailscale	-	1	256 MB	2 GB	-	Yes	Yes	infra,vpn	(internet)	Privileged for TUN
200	ct-grafana	node-02	.30	Grafana	11.x	2	1 GB	8 GB	-	No	Yes	monitoring,grafana	ct-prometheus	
201	ct-prometheus	node-02	.31	Prometheus	2.54	2	2 GB	8 GB	30 GB (/var/lib/prometheus)	No	Yes	monitoring,prometheus,core	ct-grafana (scrape targets)	30d retention
202	ct-uptime	node-02	.32	Uptime Kuma	1.23	1	512 MB	4 GB	-	No	Yes	monitoring,uptime	ct-dns,	
300	ct-pbs	node-03	.40	PBS	3.x	2	2 GB	8 GB	200 GB (/mnt/datastore)	Yes	Yes	backup,storage	(storage)	Privileged for datastore
301	ct-sync	node-03	.41	Syncthing	1.27	2	1 GB	4 GB	50 GB (/data)	No	Yes	infra,sync	ct-dns	
302	ct-git	node-03	.42	Git/Scripts	-	1	512 MB	4 GB	-	No	Yes	infra,git	(internet for push)	

C. Backup Verification Checklist

Run monthly. Check off each item.

PBS (Proxmox Backup Server)

Check	Status	Date
PBS web UI is accessible		
Nightly backups are visible in datastore		
Latest backup is from last night		
Datastore disk usage is reasonable (not 95%+)		
Pruning is working (old backups being removed)		
Test restore: picked a random container, restored to temp CTID		
Test restore: verified the restored container starts and runs		
Cleaned up test restore container after verification		

Restic

Check	Status	Date
LOCAL repo: `restic snapshots` shows recent backups		
REMOTE repo: `restic snapshots` shows recent backups		
Latest snapshot is from last night		
Weekly integrity check log shows no errors		
Test restore: extracted a specific file from latest snapshot		
Test restore: verified file contents are correct		
Restic password is stored outside the lab (password manager, paper)		

Git

Check	Status	Date
Remote repository shows recent pushes		
State files are current and accurate		
Config copies match actual running configs		

Alerting

Check	Status	Date
Sent a test email from msmtp		
Backup watchdog would have alerted if backup missed		
Grafana alert test (manually trigger one)		
Uptime Kuma notification test		

D. Monitoring Dashboard Export Template

Export your Grafana dashboards as JSON for backup and version control.

Export a Dashboard

1. Open the dashboard in Grafana
2. Click the share icon (or navigate to Dashboard Settings)
3. Go to JSON Model
4. Copy the JSON
5. Save to `homelab/configs/grafana/dashboard-name.json`

Import a Dashboard

1. Grafana > Dashboards > New > Import
2. Paste the JSON or upload the file
3. Select data sources
4. Import

Key Dashboards to Export

Dashboard	ID/Source	What It Shows
Node Exporter Full	Community ID 1860	Comprehensive host metrics
Proxmox VE	Community ID 10347	Proxmox-specific metrics
Lab Overview	Custom	High-level health summary

E. Recommended Hardware Links and Budget Builds

Budget Build: $250 Starter Lab

Item	Model	Qty	Price Each	Total
Mini PC	Lenovo M720q (i5-8500T, 16GB, 256GB) refurbished	2	$100	$200

Item	Model	Qty	Price Each	Total
Switch	TP-Link TL-SG108 8-port Gigabit	1	$20	$20
Ethernet cables	Cat6 3ft	3	$3	$9
USB drive	8 GB (for Proxmox installer)	1	$5	$5
Total				**$234**

Recommended Build: $600 Workhorse Lab

Item	Model	Qty	Price Each	Total
Mini PC	Lenovo M920q (i5-9500T, 32GB, 512GB NVMe) refurbished	3	$140	$420
Switch	TP-Link TL-SG108 8-port Gigabit	1	$20	$20
UPS	CyberPower CP850PFCLCD	1	$100	$100
SATA SSD (storage node)	1TB 2.5” SATA	1	$60	$60
Ethernet cables	Cat6 3ft	5	$3	$15
USB drive	8 GB	1	$5	$5
Total				**$620**

Premium Build: $1,200+ Production Lab

Item	Model	Qty	Price Each	Total
Mini PC	Intel NUC 12 (i5-1240P, 64GB, 1TB NVMe)	3	$300	$900
Switch	TP-Link TL-SG1016DE 16-port Managed Gigabit	1	$70	$70
UPS	CyberPower CP1500AVRLCD	1	$160	$160
NAS	Synology DS224+ (2-bay)	1	$300	$300
NAS Drives	4TB NAS HDD	2	$80	$160
Ethernet cables	Cat6 various lengths	10	$3	$30
Total				**$1,620**

Where to Buy Refurbished

Seller	What They Sell	Notes
eBay (various sellers)	Mini PCs, servers, enterprise gear	Largest selection, verify seller ratings
Amazon Renewed	Mini PCs, NAS units	Amazon return policy applies
ServerMonkey	Enterprise servers	Warranty included
SaveMyServer	Dell/HP servers	Good selection of PowerEdge/ProLiant
r/homelabsales (Reddit)	Everything	Community sales, often good deals

---

F. Quick Reference: Common Commands

Proxmox Cluster

```
pvecm status                  # Cluster status and quorum
pvecm nodes                   # List cluster nodes
pvesh get /cluster/resources   # All cluster resources
```

Container Management

```
pct list                      # List all containers on this node
pct status 100                # Status of container 100
pct start 100                 # Start container
pct shutdown 100              # Graceful shutdown
pct stop 100                  # Force stop
pct enter 100                 # Enter container shell
pct exec 100 -- <command>     # Run command in container
pct config 100                # Show container configuration
pct set 100 --memory 1024     # Change container setting
pct resize 100 rootfs +5G     # Resize container disk
pct snapshot 100 snap1        # Create snapshot
pct rollback 100 snap1        # Rollback to snapshot
pct migrate 100 node-02       # Migrate to another node
pct clone 100 150 --full      # Clone container
pct destroy 100               # Delete container (must be stopped)
```

Storage

```
pvesm status                  # List all storage pools
pvesm list local              # List contents of a storage pool
df -h                         # Disk usage (host)
zpool status                  # ZFS pool status (if using ZFS)
zpool list                    # ZFS pool capacity
```

Networking

```
ip addr show                    # Network interfaces
ip route show                   # Routing table
brctl show                      # Bridge configuration
ss -tlnp                        # Listening TCP ports
```

Monitoring

```
# Prometheus
curl -s http://localhost:9090/api/v1/targets | python3 -m json.tool  # Scrape targets
promtool check config /etc/prometheus/prometheus.yml                  # Validate config

# node_exporter
curl -s http://localhost:9100/metrics | head -50    # Raw metrics

# Grafana
grafana-cli plugins ls                              # List installed plugins
systemctl status grafana-server                     # Service status
```

Backups

```
# PBS
pvesh get /nodes/<node>/storage/pbs/content         # List PBS backups

# Restic
restic -r <repo> --password-file <file> snapshots   # List snapshots
restic -r <repo> --password-file <file> check        # Verify integrity
restic -r <repo> --password-file <file> stats         # Repository statistics
restic -r <repo> --password-file <file> ls latest     # Browse latest snapshot
restic -r <repo> --password-file <file> restore latest --target /tmp/restore  # Full restore
```

Security

```
fail2ban-client status                              # Fail2ban overview
fail2ban-client status sshd                         # SSH jail status
fail2ban-client set sshd unbanip <ip>               # Unban an IP
tailscale status                                    # Tailscale connection status
pve-firewall status                                 # Firewall status
pve-firewall stop                                   # Emergency: disable firewall
```

G. Glossary

Term	Definition
ARC	Adaptive Replacement Cache — ZFS's memory-based read cache

Term	Definition
Bridge (vmbr0)	Virtual network switch connecting containers/VMs to the physical network
Container (LXC)	OS-level virtualization — shares host kernel, isolated userspace
Corosync	Cluster communication layer used by Proxmox for node coordination
CTID	Container ID — numeric identifier for each LXC container
Datastore	PBS term for a storage location where backups are kept
DHCP Reservation	Router configuration that assigns a consistent IP to a specific MAC address
Exporter	A process that exposes metrics in Prometheus-compatible format
Fail2ban	Intrusion prevention software that bans IPs after failed login attempts
FQDN	Fully Qualified Domain Name — e.g., `node-01.lab`
Grafana	Open-source visualization and alerting platform
HA	High Availability — automatic failover when a node goes down
Hypervisor	Software that creates and manages virtual machines (Proxmox VE is a Type 1 hypervisor)
IOMMU	Input-Output Memory Management Unit — required for PCI/GPU passthrough
IPSET	A named group of IP addresses used in firewall rules
KVM	Kernel-based Virtual Machine — Linux kernel virtualization technology
LXC	Linux Containers — OS-level virtualization technology
mkcert	Tool for creating locally-trusted development certificates
NUT	Network UPS Tools — software for monitoring and managing UPS devices
OOM	Out of Memory — kernel mechanism that kills processes when memory is exhausted
PBS	Proxmox Backup Server — purpose-built backup solution for Proxmox
PM2	Process Manager 2 — Node.js process manager for production
Prometheus	Open-source time-series database and monitoring system
PromQL	Prometheus Query Language — used for querying metrics
PVE	Proxmox Virtual Environment
QEMU	Quick Emulator — hardware emulation used alongside KVM for VMs
Quorum	Minimum number of cluster nodes needed to make decisions (2 out of 3)
Restic	Fast, encrypted, deduplicated backup program
Scrape	Prometheus term for collecting metrics from a target
Subnet	A logical division of an IP network (e.g., `192.168.1.0/24`)
Tailscale	Mesh VPN service built on WireGuard
Template	A read-only container image used for cloning new containers

Term	Definition
TLS	Transport Layer Security — encryption protocol for HTTPS
Thin Pool	LVM storage that allocates disk space on demand rather than upfront
Unprivileged Container	LXC container where UIDs are mapped to non-root host UIDs (more secure)
Uptime Kuma	Open-source uptime monitoring tool with a web dashboard
VLAN	Virtual LAN — network segmentation technology
VM	Virtual Machine — full hardware emulation with its own kernel
WireGuard	Modern, fast VPN protocol
ZFS	Filesystem and volume manager with data integrity, compression, and snapshots

---

H. Troubleshooting Decision Tree

Container Will Not Start

```
Container won't start
├── Check: pct start <ctid> 2>&1
│   ├── "Permission denied" → Check if unprivileged + needs privileges
│   │   └── Try: pct set <ctid> --features nesting=1
│   │   └── Last resort: pct set <ctid> --unprivileged 0
│   ├── "Storage not available" → Check: pvesm status
│   │   └── Is the storage pool active? Is the disk full?
│   ├── "Lock" error → Check: ls /var/lock/lxc/
│   │   └── Remove stale lock: rm /var/lock/lxc/<ctid>.lock
│   └── Other → Check: journalctl -u pve-container@<ctid>
```

Service Not Accessible

```
Can't reach service
├── Can you ping the container IP?
│   ├── No → Check container is running: pct status <ctid>
│   │   └── Check bridge: ip link show vmbr0
│   │   └── Check container network: pct exec <ctid> -- ip addr
│   └── Yes → Is the service running?
│       ├── No → pct exec <ctid> -- systemctl status <service>
│       │   └── Check logs: pct exec <ctid> -- journalctl -u <service>
│       └── Yes → Is the port open?
│           ├── No → Check firewall: pve-firewall status
│           │   └── Check iptables: iptables -L -n
│           └── Yes → Check Traefik route configuration
│               └── Check DNS: nslookup <name>.lab <dns-ip>
```

Monitoring Gap

```
Grafana shows "No Data"
├── Check Prometheus targets: http://<prometheus>:9090/targets
│   ├── Target is DOWN → node_exporter not running on target
│   │   └── SSH to target, check: systemctl status prometheus-node-exporter
│   ├── Target is UP but no data → Wrong metric name in query
│   │   └── Test query directly in Prometheus web UI
│   └── All targets missing → Prometheus itself is down
│       └── Check: systemctl status prometheus
│
├── Check Grafana data source: Connections > Data Sources > Test
│   ├── "Error" → Prometheus URL wrong or unreachable
│   └── "Working" → Dashboard query issue
```

I. Next Steps After This Guide

You have built a production-grade homelab. Here is a prioritized list of what to explore next, ordered by practical value:

High Value (Do These)

1. **Ansible** — Automate everything you did manually in this guide. Replace "SSH into each node and run commands" with a playbook.
2. **Git-based workflow** — Store all configs in git. Changes go through pull requests, even if you are the only reviewer.
3. **Centralized logging (Loki + Promtail)** — You have metrics with Prometheus. Add logs with Loki. Same Grafana interface.
4. **Infrastructure as Code (Terraform)** — Define your containers in `.tf` files. Run `terraform apply` instead of clicking the web UI.

Medium Value (Do If Interested)

5. **Kubernetes (k3s)** — If your career path goes toward DevOps/SRE, Kubernetes experience is increasingly expected.
6. **CI/CD pipeline** — Gitea or Drone CI on your cluster. Auto-deploy applications on git push.
7. **SIEM (Wazuh)** — If your career path goes toward security, a SIEM is a significant lab addition.
8. **Network segmentation (VLANs)** — Add a managed switch and segment your network properly.

Low Urgency (Nice to Have)

9. **Ceph distributed storage** — Complex but teaches you distributed systems.
10. **Mail server** — Educational but painful to maintain. Not recommended unless you enjoy suffering.

11. **Public-facing services** — Expose a service to the internet (carefully, behind Cloudflare Tunnel or similar).

Closing Thoughts

You built something real. Not a toy, not a demo — a functioning infrastructure platform that monitors itself, backs itself up, alerts you when something breaks, and documents its own state.

The skills you developed doing this are the same skills used in every production environment on the planet. The tools are the same. The patterns are the same. The mindset — monitor everything, back up everything, document everything, automate everything — is the same.

The difference between a homelab and a data center is scale, not substance.

Keep building. Keep breaking things (intentionally). Keep fixing them. That is how you learn infrastructure.

Interview Readiness Rubric

Picture this. You're in a DevOps interview. The interviewer says, "Tell me about your infrastructure experience." Most candidates mumble about Docker containers on a laptop. You're about to describe a 3-node Proxmox cluster with monitoring, automated backups, security hardening, and documentation. Here's how to know you're ready.

Tier 1: "I Have a Homelab" (Minimum Viable Interview Story)

You can confidently discuss:

- ☐ What Proxmox is and why you chose it over ESXi or bare Docker
- ☐ Your network layout (IP scheme, DNS, subnets)
- ☐ At least 3 services you run and why each exists
- ☐ How you back up your data and how often
- ☐ One time something broke and how you fixed it

This gets you through a screening call.

Tier 2: "I Run Production-Grade Infrastructure" (Strong Junior Candidate)

Everything in Tier 1, plus:

- ☐ You can explain your monitoring stack (what you monitor, what triggers alerts, how you get notified)
- ☐ You can walk through your backup strategy: what's backed up, where, how often, and when you last tested a restore
- ☐ You can describe your security posture: firewall rules, SSH hardening, fail2ban, VPN access
- ☐ You can explain a change you made that required planning (not just "I installed X")
- ☐ You document your infrastructure and can show it

This makes you competitive for junior SRE/DevOps/sysadmin roles.

Tier 3: "I Think Like an Engineer" (Standout Candidate)

Everything in Tier 2, plus:

- ☐ You can explain trade-offs you made and why (e.g., "I chose Traefik over Nginx because…")
- ☐ You automate repetitive tasks and can show the scripts
- ☐ You think about failure modes: "What happens if this node dies at 2 AM?"
- ☐ You can tie your homelab work to the specific job you're interviewing for
- ☐ You contribute to or maintain documentation that someone else could follow

This puts you ahead of candidates with 1-2 years of professional experience who can't articulate what they do.

The lab is the proof. The interview is the presentation. This guide gave you the lab. Now practice presenting it — out loud, to a mirror, to a friend, to a rubber duck. The candidates who get hired are not the ones who know the most. They are the ones who can explain what they know clearly and confidently.

Return to: Introduction

About the Author

Steven J. Vik is a cybersecurity engineer and DevOps practitioner who builds and breaks infrastructure for a living.

He runs a production-grade Proxmox homelab — the same one documented in this guide — with a three-node cluster, automated backups to multiple targets, Grafana monitoring with real alerting, and a Tailscale overlay network for remote access. Everything in this guide was built, tested, broken, fixed, and documented on his own hardware.

Steve holds a B.S. in Cybersecurity and Information Assurance from Western Governors University and has spent years in roles spanning security operations, systems administration, and infrastructure automation. He's the kind of engineer who reads the man page before the blog post and prefers boring, proven tools over the shiny new thing.

When he's not hardening containers or writing alert rules, he runs a long-running D&D campaign, tinkers with self-hosted AI inference, and tries to keep his cat off the keyboard.

You can find more of his work at **SJVIK Labs**.

If this guide helped you build something cool, I'd love to hear about it. Drop a review on the product page — it means more than you'd think.

Also from SJVIK Labs

SOC Analyst Interview Kit

Everything you need to land your first SOC role

200+ flashcards, 25 real-world scenarios, SIEM query exercises,
behavioral frameworks, a mock interview scoring guide,
and a 90-day onboarding playbook.

Whether you're breaking into cybersecurity or preparing for
your next SOC interview, this kit gives you the edge.

Available now on Gumroad

gumroad.com/sjviklabs